



第4章 常用组合逻辑功能器件

本章将介绍几种常用的中规模集成电路(**MSI**), 这些中规模集成电路分别具有特定的**逻辑功能**, 称为**功能模块**, 用功能模块设计组合逻辑电路, 具有许多优点。



4.1 自顶向下的模块化设计方法

顶： 指系统功能，即系统总要求，较抽象。

向下： 指根据系统总要求，将系统分解为若干个子系统，再将每个子系统分解为若干个功能模块... ..，直至分成许多各具特定功能的基本模块为止。

例：设计一个数据检测系统，
功能表如下：

数据A、B分别来自**两个**
传感器

S_1	S_2	输出功能
0	0	$A + B$
0	1	$A - B$
1	0	$\text{Min}(A, B)$
1	1	$\text{Max}(A, B)$

T: 数据检测系统

顶层

T₁: 输入
传感器数据

T₂
计算值

T₃
选择输出

T₁₁
传感器A

T₁₂
传感器B

T₂₁
A+B

T₂₂
A-B

T₂₃
Min(A,B)

T₂₄
Max(A,B)

T₂₃₁
比较
A和B

T₂₃₂
选择
Min

T₂₄₁
比较
A和B

T₂₄₂
选择
Max

* : 叶结点

分层设计树

*

*

*

*

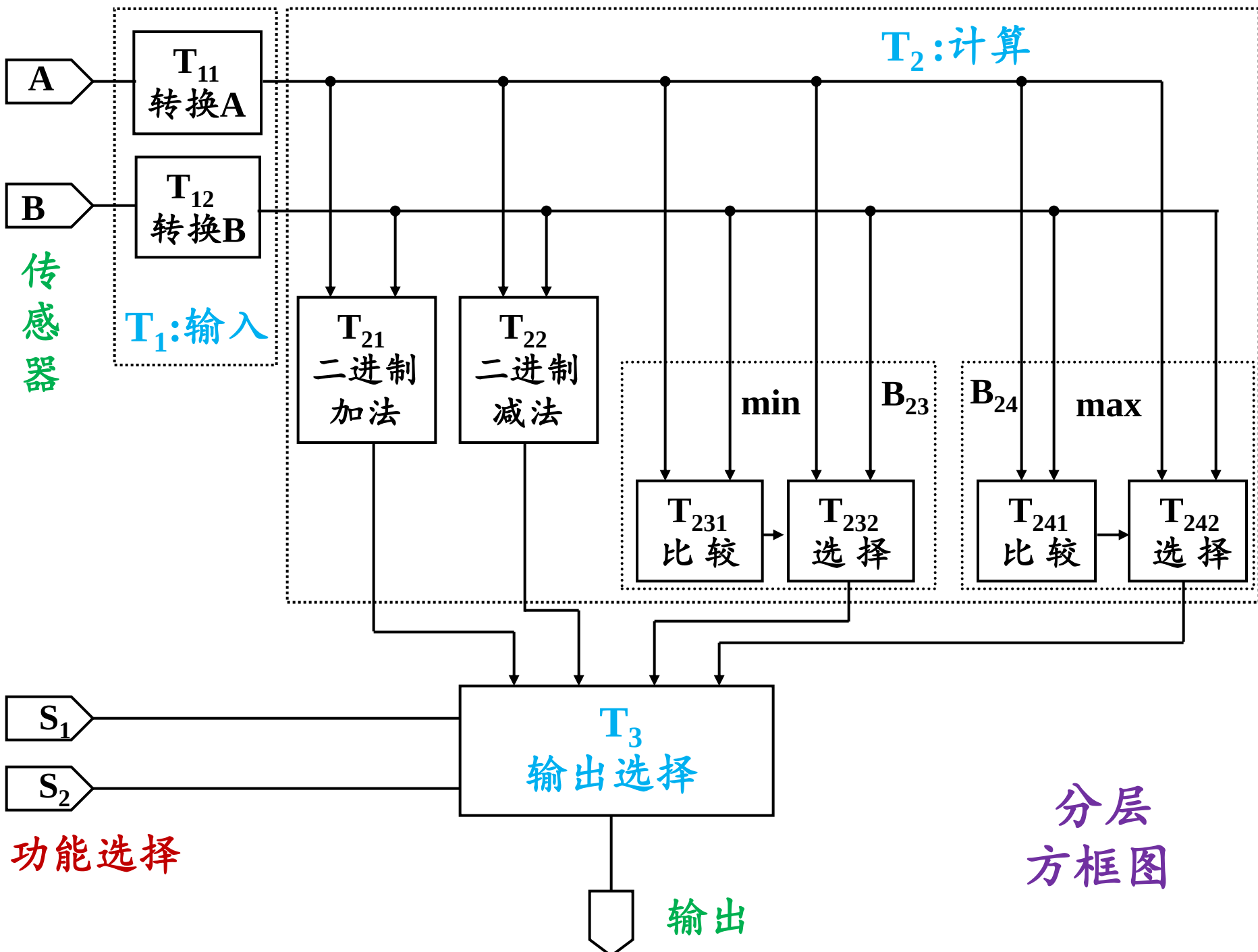
*

*

*

*

*





4.2 编码器

将信息（如数和字符等）转换成符合一定规则的二进制代码的过程称为**编码**，即编码就是用特定码表示特定信息的过程。完成编码逻辑功能的电路称为**编码器**。

4.2.1 二进制编码器

用 n 位二进制代码对 $N=2^n$ 个特定信息进行编码的逻辑电路。

设计方法：**以例说明**



例：设计一个输入互相排斥（即任何时刻只能有一个输入信号有效）的编码器。

设：输入 X_0 、 X_1 、 X_2 、 X_3 （设高电平为有效电平）
输出 A_1 、 A_0

解：输入信号和输出码之间的对应关系定义为：

输入	输出	
	A_1	A_0
X_0	0	0
X_1	0	1
X_2	1	0
X_3	1	1

X_3	X_2	X_1	X_0	A_1	A_0
0	0	0	0	×	×
0	0	0	1	0	0
0	0	1	0	0	1
0	0	1	1	×	×
0	1	0	0	1	0
0	1	0	1	×	×
0	1	1	0	×	×
0	1	1	1	×	×
1	0	0	0	1	1
1	0	0	1	×	×
1	0	1	0	×	×
1	0	1	1	×	×
1	1	0	0	×	×
1	1	0	1	×	×
1	1	1	0	×	×
1	1	1	1	×	×

$X_3X_2 \backslash X_1X_0$	00	01	11	10
00	×	0	×	0
01	1	×	×	×
11	×	×	×	×
10	1	×	×	×

$$A_1 = X_2 + X_3$$

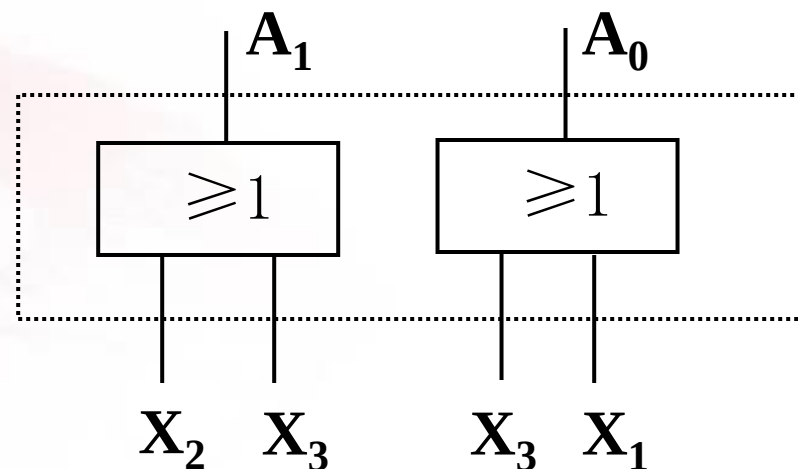
$X_3X_2 \backslash X_1X_0$	00	01	11	10
00	×	0	×	1
01	0	×	×	×
11	×	×	×	×
10	1	×	×	×

$$A_0 = X_1 + X_3$$



4线—2线普通编码器电路图：

- (1) 普通编码器在任何时候只允许有一个输入信号有效；
- (2) 电路无 X_0 输入端；
- (3) 电路无输入时，编码器的输出与 X_0 编码等效。





带输出**使能(Enable)**端的**优先编码器**:

输出**使能**端: 用于判别电路是否有信号输入。

优先: 对输入信号按轻重缓急**排序**, 当有多个信号同时输入时, 只对**优先权高**的一个信号进行编码。

下面把上例4线—2线普通编码器改成带输出**使能(Enable)**端的**优先编码器**, 假设输入信号优先级的次序:
 X_3 、 X_2 、 X_1 、 X_0 。

X_3	X_2	X_1	X_0	A_1	A_0	EO
0	0	0	0	0	0	1
0	0	0	1	0	0	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	1	0	0
0	1	0	1	1	0	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	1	1	0
1	0	0	1	1	1	0
1	0	1	0	1	1	0
1	0	1	1	1	1	0
1	1	0	0	1	1	0
1	1	0	1	1	1	0
1	1	1	0	1	1	0
1	1	1	1	1	1	0

X_3X_2	X_1X_0	00	01	11	10
00	00	0	0	0	0
01	00	1	1	1	1
11	00	1	1	1	1
10	00	1	1	1	1

$$A_1 = X_2 + X_3$$

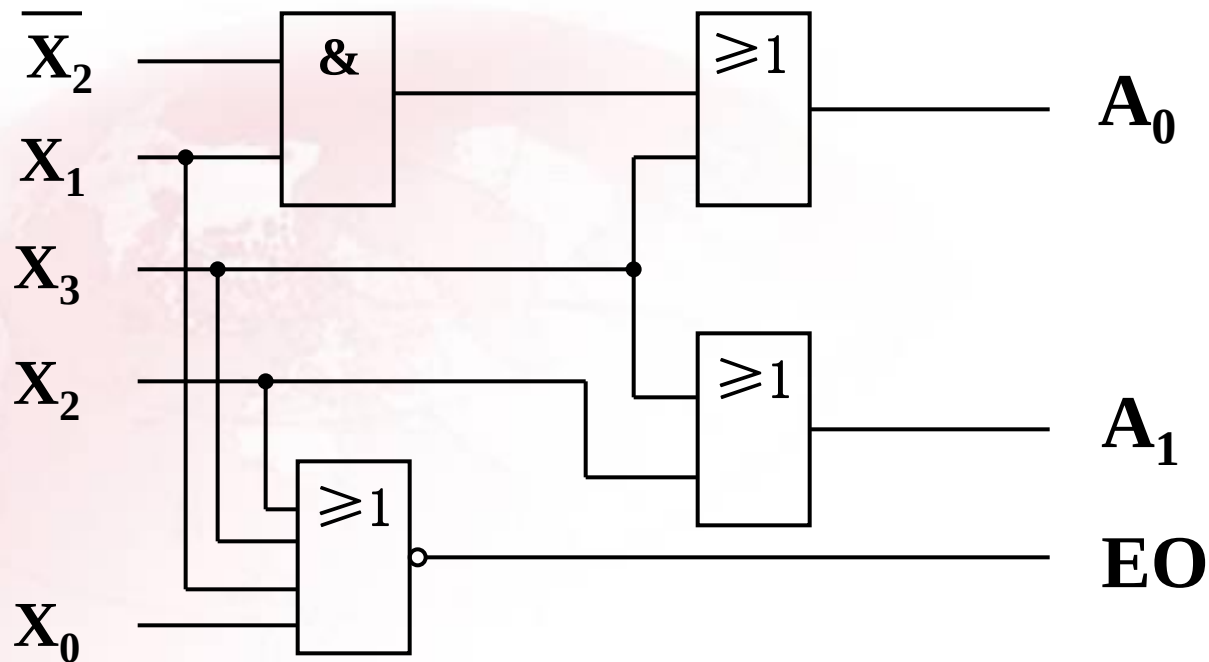
X_3X_2	X_1X_0	00	01	11	10
00	00	0	0	1	1
01	00	0	0	0	0
11	00	1	1	1	1
10	00	1	1	1	1

$$A_0 = X_3 + \overline{X_2}X_1$$



$$EO = \overline{X_3} \overline{X_2} \overline{X_1} \overline{X_0} = \overline{X_3 + X_2 + X_1 + X_0}$$

编码器
电路图





4.2.2 二—十进制编码器

输入： $I_0, I_1, I_2 \dots \dots I_9$ ，表示十个要求编码的信号。

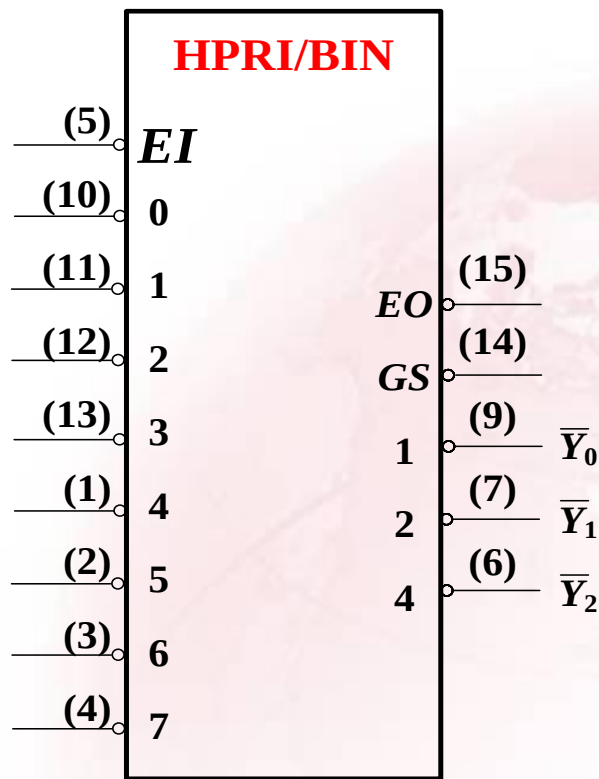
输出：BCD码。

电路有十根输入线，四根输出线，常称为**10线—4线**编码器。

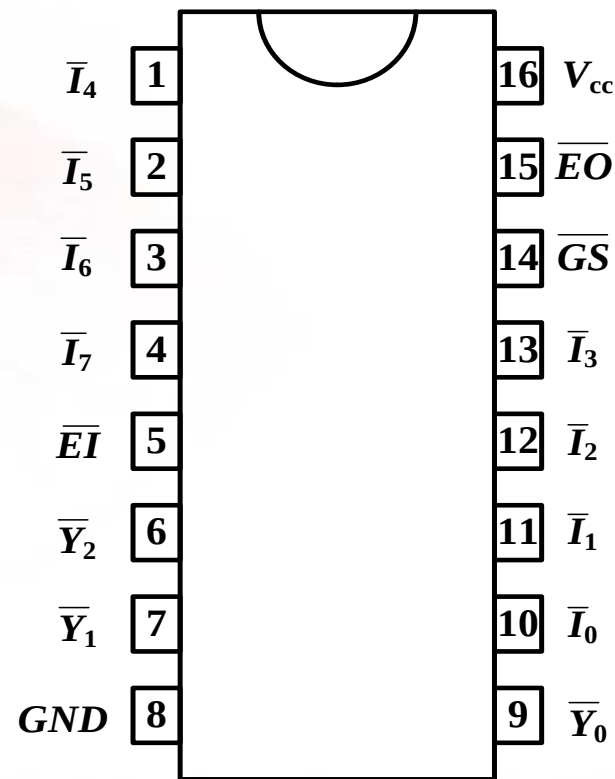


4.2.3 通用编码器集成电路

1. 8线—3线优先编码器74148



逻辑图



引脚图



74148功能说明:

- (1) 74148为**8线—3线**优先编码器，**HPRI**是最高位优先编码器的说明。
- (2) 编码器输入为**低**电平有效，输出为3位二进制**反码**。
- (3) **$\overline{EI}$** 端为输入使能端，当 **$\overline{EI} = 0$** 时，电路处于正常工作状态；当 **$\overline{EI} = 1$** 时，电路禁止工作， **$\overline{Y_2}\overline{Y_1}\overline{Y_0}=111$** ， **$\overline{EO}=1$** ， **$\overline{GS}=1$** 。



(4) $\overline{EO}$ 为选通输出端

$$\overline{EO} = EI \overline{I_0} \overline{I_1} \overline{I_2} \overline{I_3} \overline{I_4} \overline{I_5} \overline{I_6} \overline{I_7}$$

① 当 $\overline{EI}=1$ （即禁止工作时），则 $\overline{EO}=1$ 。

② 当 $\overline{EI}=0$ （即正常工作时），

若编码输入信号 $\overline{I_i}$ 均为 1（即无编码信号输入），则 $\overline{EO}=0$ ，这时 $\overline{Y_2} \overline{Y_1} \overline{Y_0}=111$ 。

若编码输入信号 $\overline{I_i}$ 有 0（即有编码信号输入），则 $\overline{EO}=1$ ，这时 $\overline{Y_2} \overline{Y_1} \overline{Y_0}$ 输出对应二进制码的反码。



(5) $\overline{GS}$ 为扩展输出端

$$\overline{GS} = \overline{EI(I_0 + I_1 + I_2 + I_3 + I_4 + I_5 + I_6 + I_7)}$$

① 当 $\overline{EI}=1$ （即禁止工作时），则 $\overline{GS}=1$ 。

② 当 $\overline{EI}=0$ （即正常工作时），

若编码输入信号 $\bar{I}_i$ 均为1（即无编码信号输入），则 $\overline{GS}=1$ ，这时 $\bar{Y}_2\bar{Y}_1\bar{Y}_0=111$ 。

若编码输入信号 $\bar{I}_i$ 有0（即有编码信号输入），则 $\overline{GS}=0$ ，这时 $\bar{Y}_2\bar{Y}_1\bar{Y}_0$ 输出对应二进制码的反码。



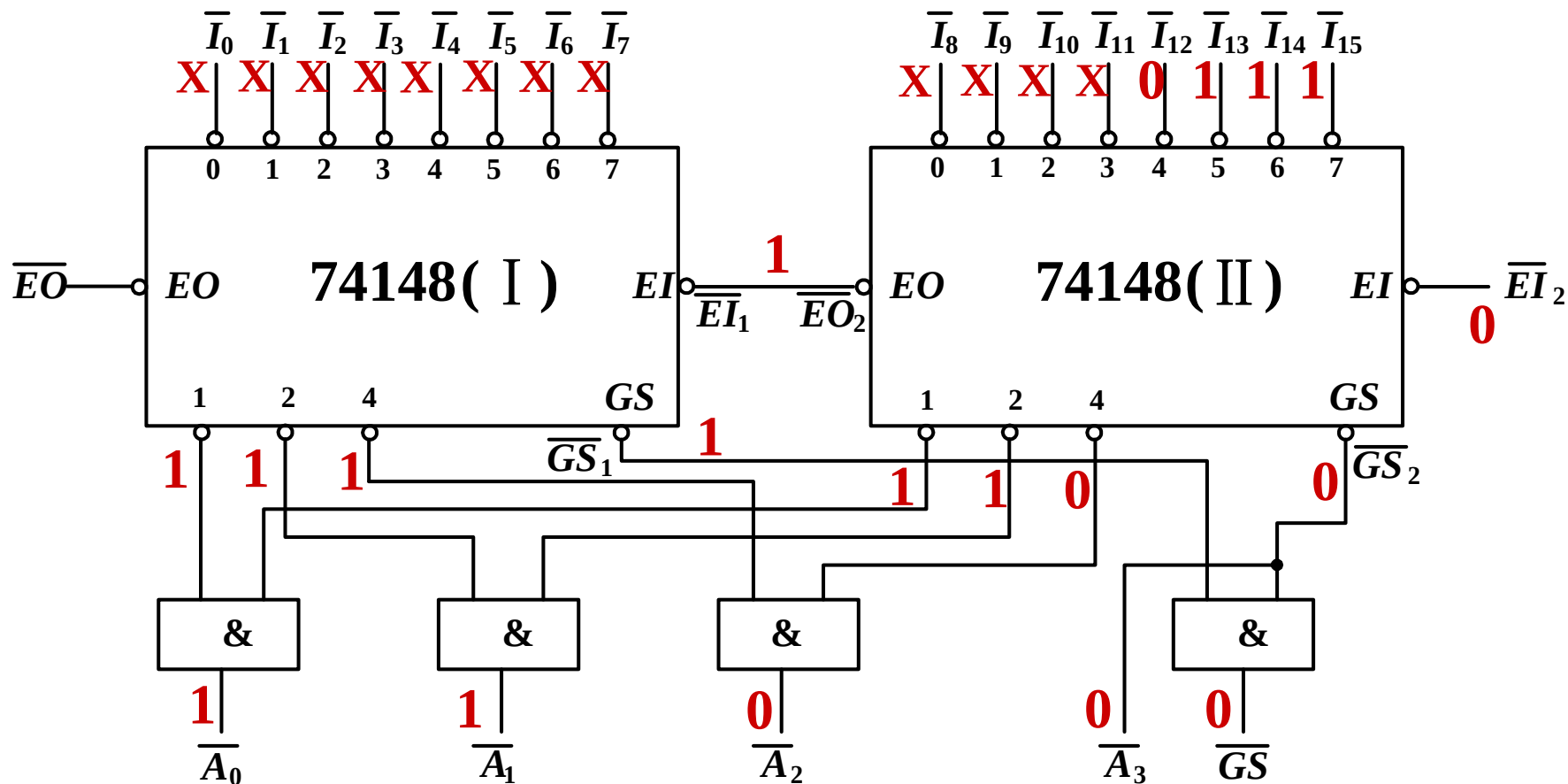
74148功能表

输 入									输 出				
$\overline{EI}$	$\overline{I_0}$	$\overline{I_1}$	$\overline{I_2}$	$\overline{I_3}$	$\overline{I_4}$	$\overline{I_5}$	$\overline{I_6}$	$\overline{I_7}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$	$\overline{GS}$	$\overline{EO}$
1	×	×	×	×	×	×	×	×	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	×	×	×	×	×	×	×	0	0	0	0	0	1
0	×	×	×	×	×	×	0	1	0	0	1	0	1
0	×	×	×	×	×	0	1	1	0	1	0	0	1
0	×	×	×	×	0	1	1	1	0	1	1	0	1
0	×	×	×	0	1	1	1	1	1	0	0	0	1
0	×	×	0	1	1	1	1	1	1	0	1	0	1
0	×	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1

例如: 若 $\overline{EI}=0$, 当输入 $\overline{I_5}$ 、 $\overline{I_3}$ 、 $\overline{I_2}$ 为**0** (有效), 其它输入为**1**。

则编码器对 $\overline{I_5}$ 进行编码, 输出 $\overline{Y_2}\overline{Y_1}\overline{Y_0}$ =**010** (**101**的反码)。

***例：用两片74148构成16线—4线优先编码器。**

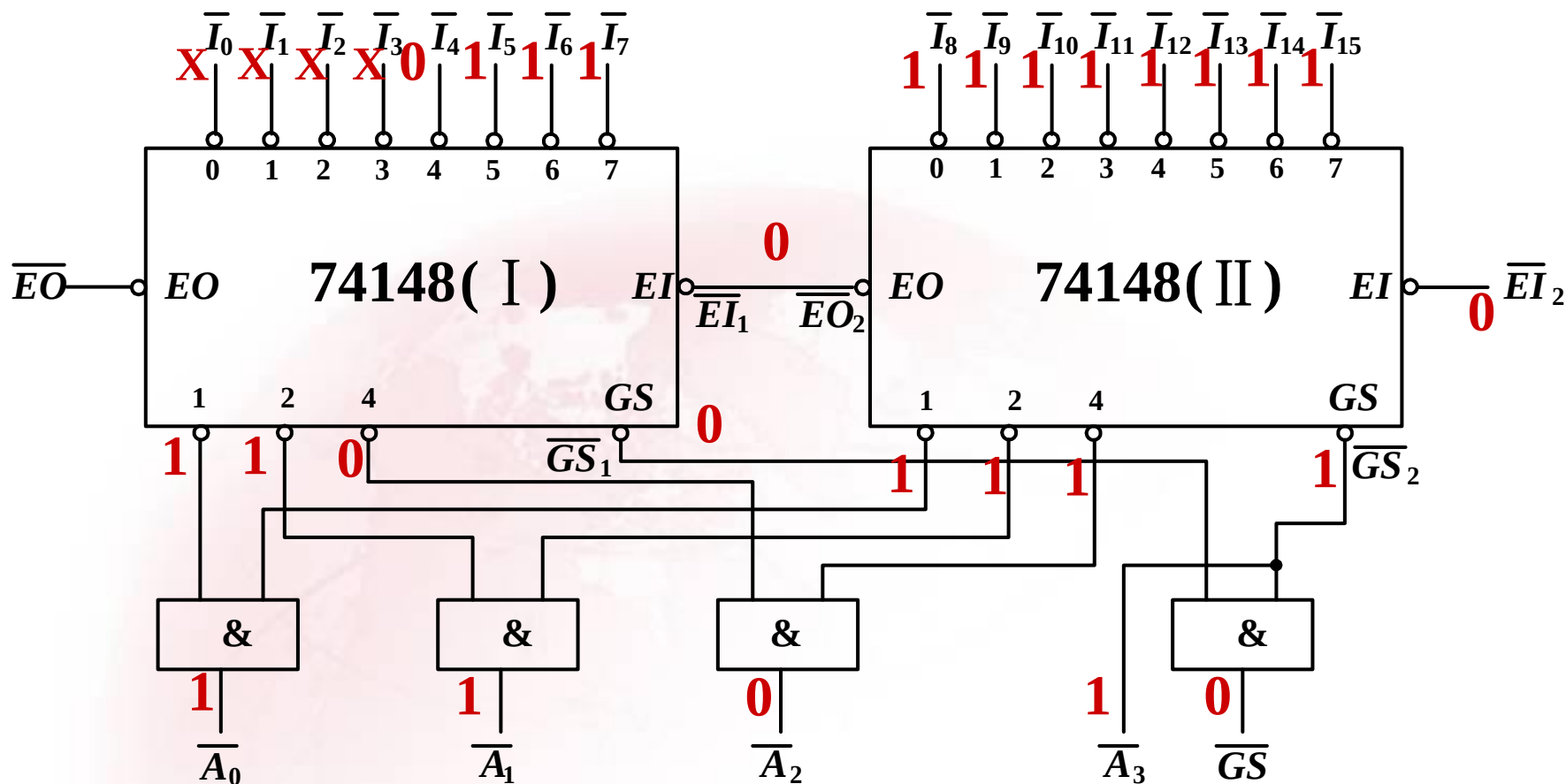


输入	$\bar{A}_3 \bar{A}_2 \bar{A}_1 \bar{A}_0$
$\bar{I}_{15} \sim \bar{I}_8$	0000~0111
$\bar{I}_7 \sim \bar{I}_0$	1000~1111

$\bar{GS}_2$	$\bar{GS}_1$	$\bar{A}_3$
0	0	×
0	1	0
1	0	1
1	1	1

$\bar{GS}_2 \backslash \bar{GS}_1$	0	1
0	×	0
1	1	1

$$\bar{A}_3 = \bar{GS}_2$$





*问题思考:

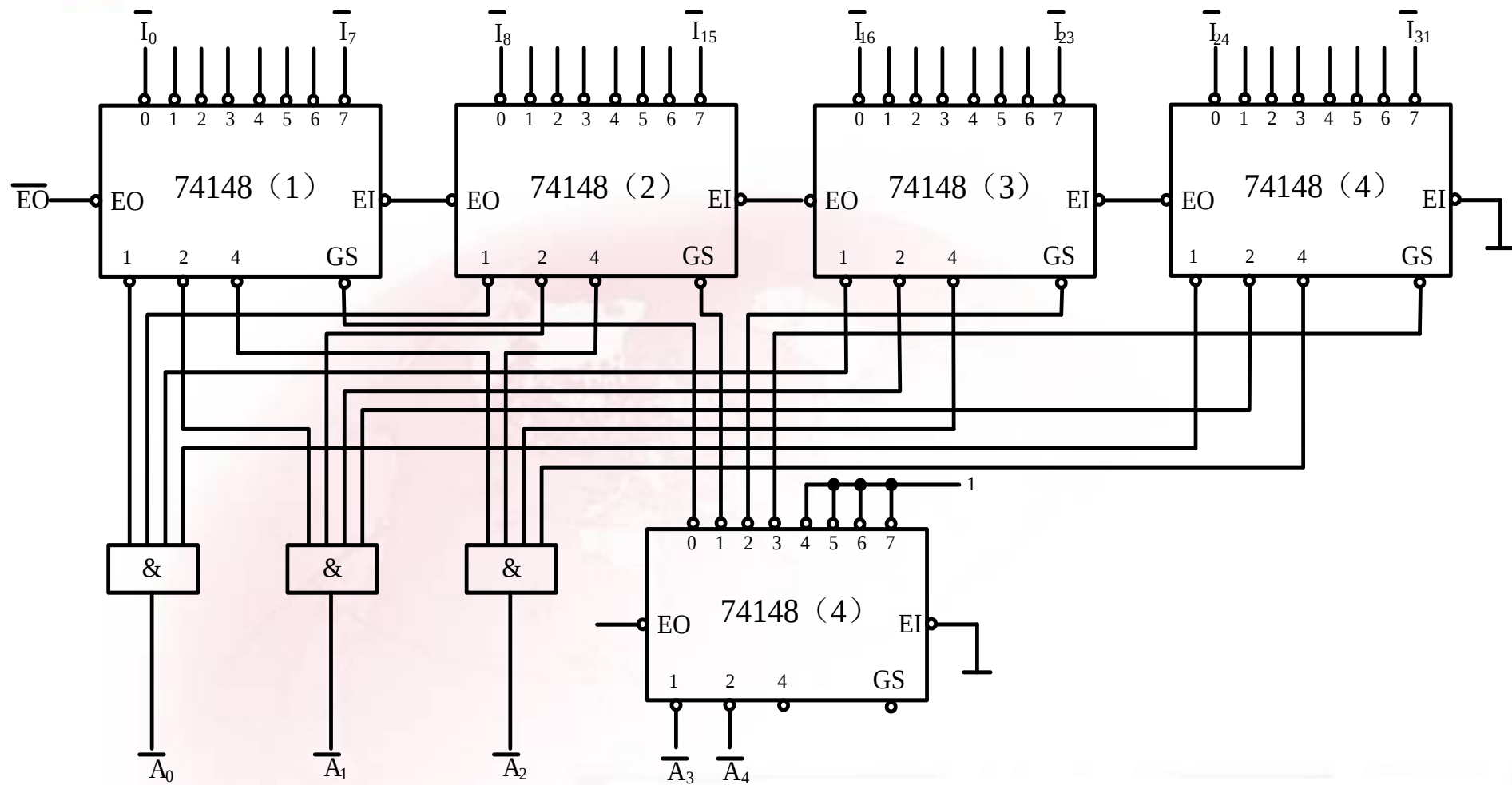
- 1) 若用5片74148构成一个32线—5线编码器，电路如何设计？
- 2) 若用9片74148构成一个64线—6线编码器，电路又如何设计？

*扩展电路设计提示:

- 1) 观察上例编码器低三位输出电路结构，并找出规律；
- 2) 分析高位输出和各 $\overline{GS}$ 之间的关系，将 $\overline{GS}$ 作为输入，高位信号作为输出，设计一输出电路。

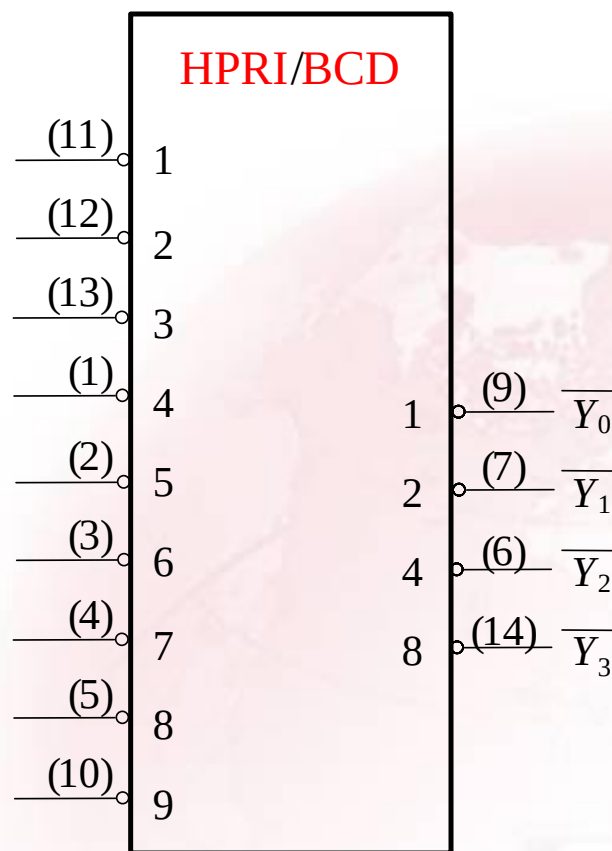


数字逻辑电路教学课程

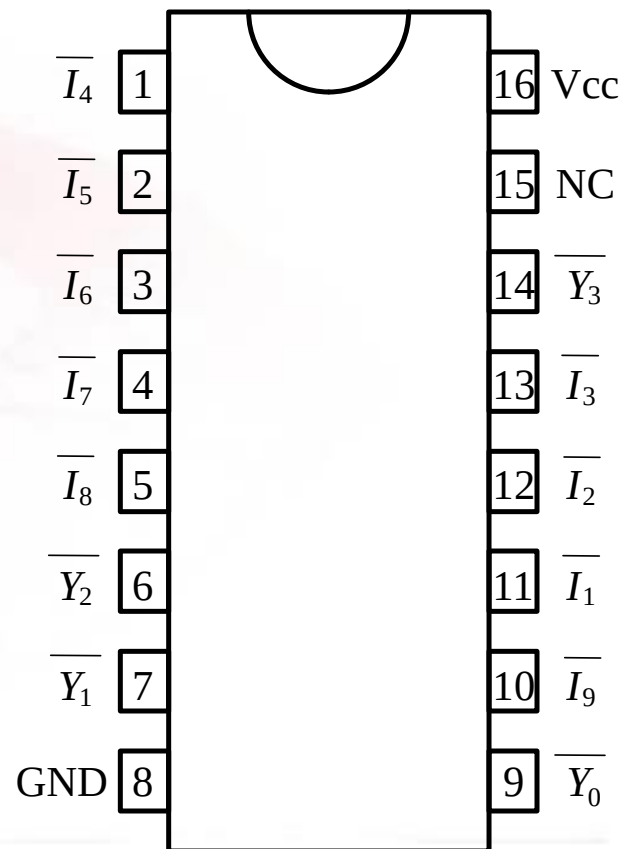




2. 10线—4线优先编码器74147



逻辑图



引脚图

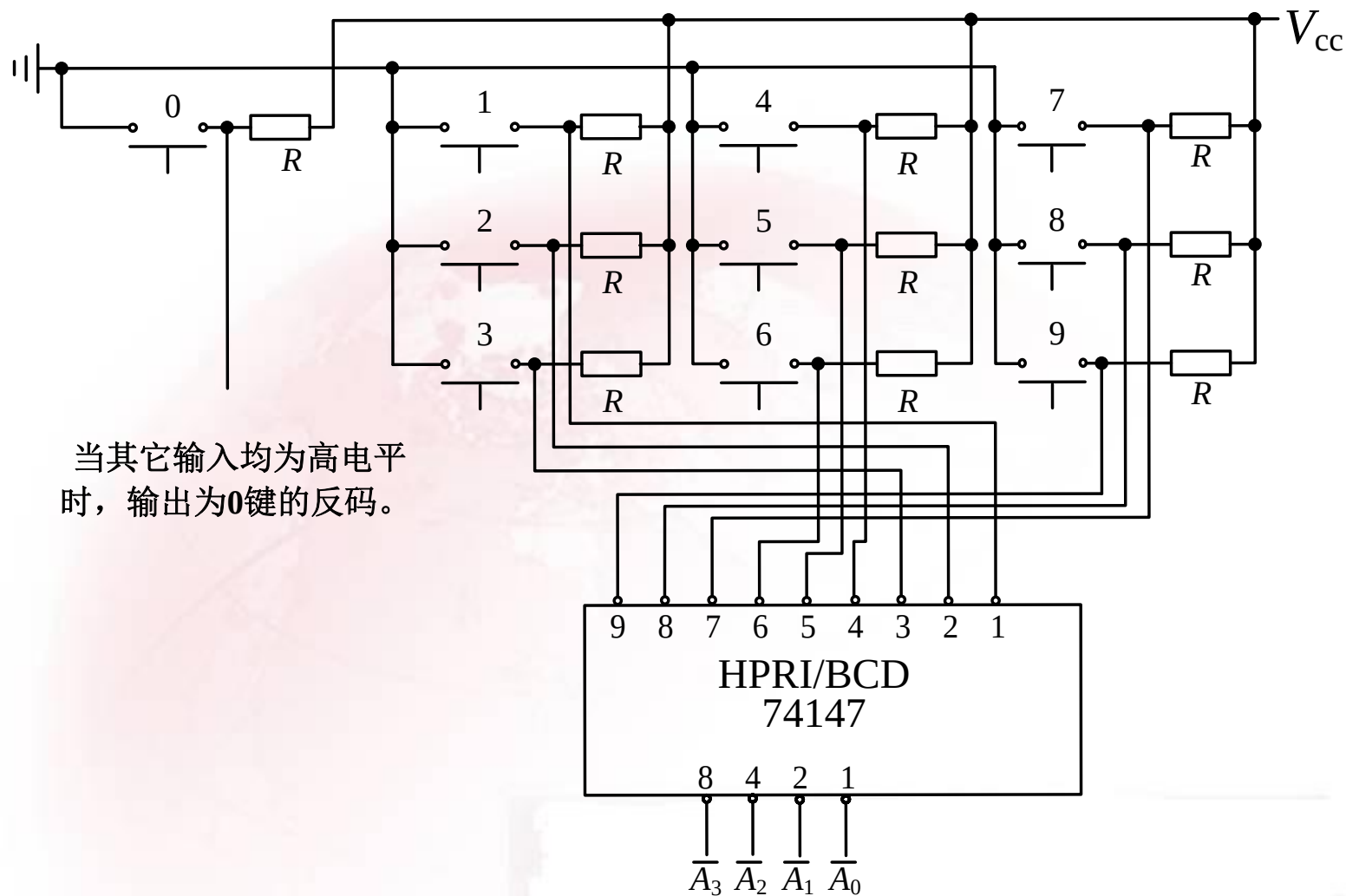


74147功能表

[illegible]



4.2.4 编码器应用举例





4.2.5 编码器的Verilog描述

1. 低电平输入有效的普通8线-3线编码器的Verilog程序代码。

```
module Vrencoder(I_L,Y);  
    input [7:0] I_L;  
    output [2:0] Y;  
    reg [7:0] I;  
    reg [2:0] Y;  
  
    always @ (I_L)  
    begin  
        I = ~I_L;  
        case(I)  
            128: Y=3'b111;
```



```
64: Y=3'b110;  
32: Y=3'b101;  
16: Y=3'b100;  
8: Y=3'b011;  
4: Y=3'b010;  
2: Y=3'b001;  
1: Y=3'b000;  
default: Y=3'b111;  
endcase  
end  
endmodule
```



2. 优先编码器的 Verilog程序代 码。

```
module Vrpriencoder(I_L,Y);  
    input [7:0] I_L;  
    output [2:0] Y;  
    reg [2:0] Y;  
  
    always@(I_L)  
    begin  
        if (I_L[7]==0) Y<=7;  
        else if (I_L[6]==0) Y<=6;  
        else if (I_L[5]==0) Y<=5;  
        else if (I_L[4]==0) Y<=4;  
        else if (I_L[3]==0) Y<=3;  
        else if (I_L[2]==0) Y<=2;  
        else if (I_L[1]==0) Y<=1;  
        else if (I_L[0]==0) Y<=0;  
        else Y<=0;  
    end  
endmodule
```



4.3 译码器/数据分配器

译码是编码的逆过程，作用是将一组码转换为确定信息。

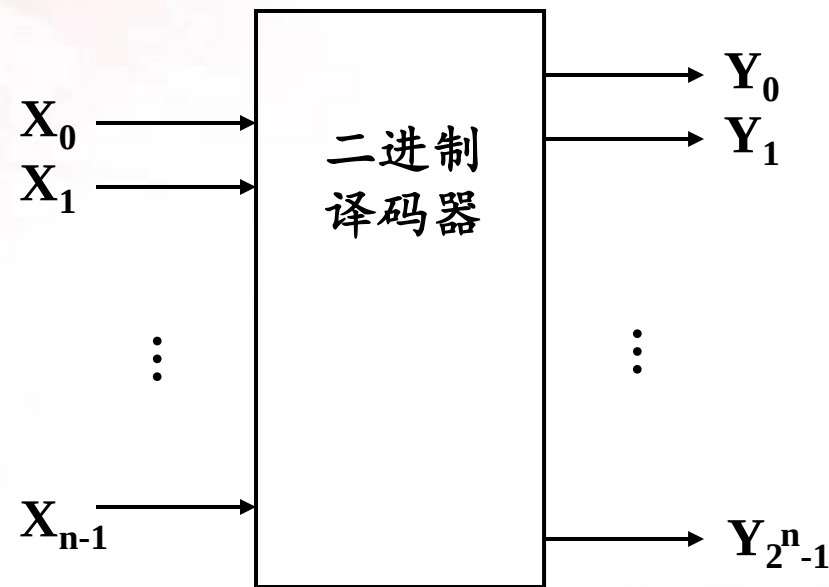
4.3.1 二进制译码器

输入：二进制代码，有 n 个；

输出： 2^n 个特定信息。

1. 译码器电路结构

以2线—4线译码器为例说明





■ 高电平输出有效的2线-4线译码器电路

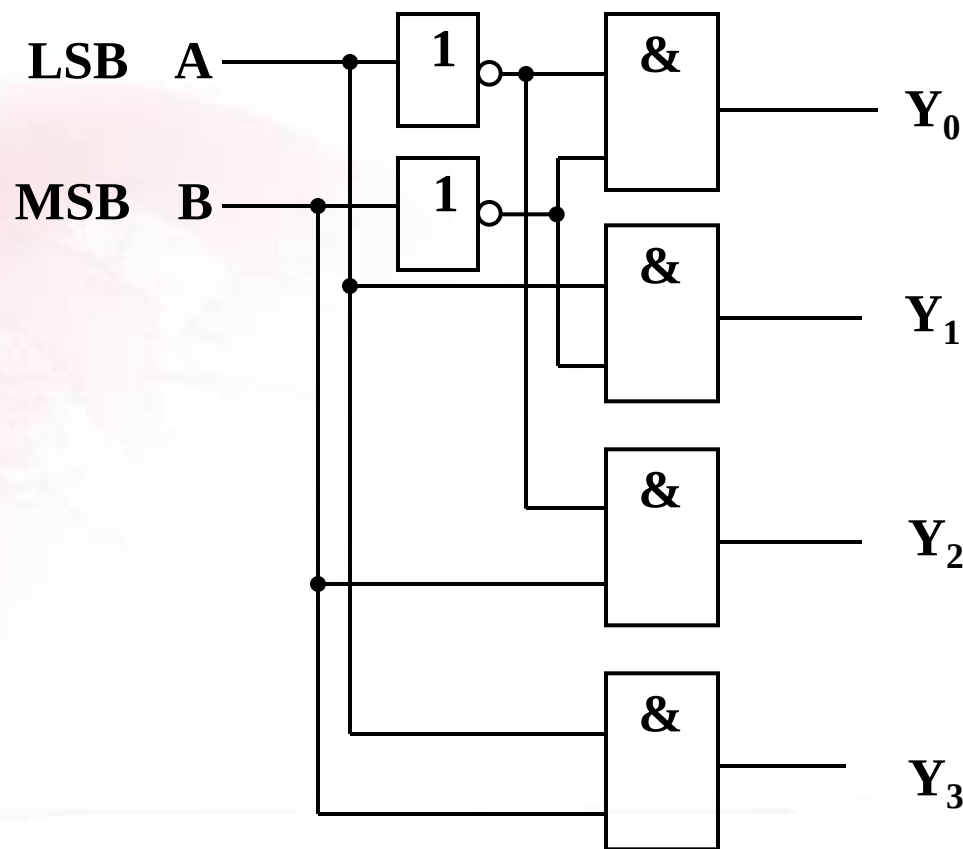
B	A	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

$$Y_0 = \overline{B}\overline{A} = m_0$$

$$Y_1 = \overline{B}A = m_1$$

$$Y_2 = B\overline{A} = m_2$$

$$Y_3 = BA = m_3$$





■ 低电平输出有效的2线-4线译码器电路

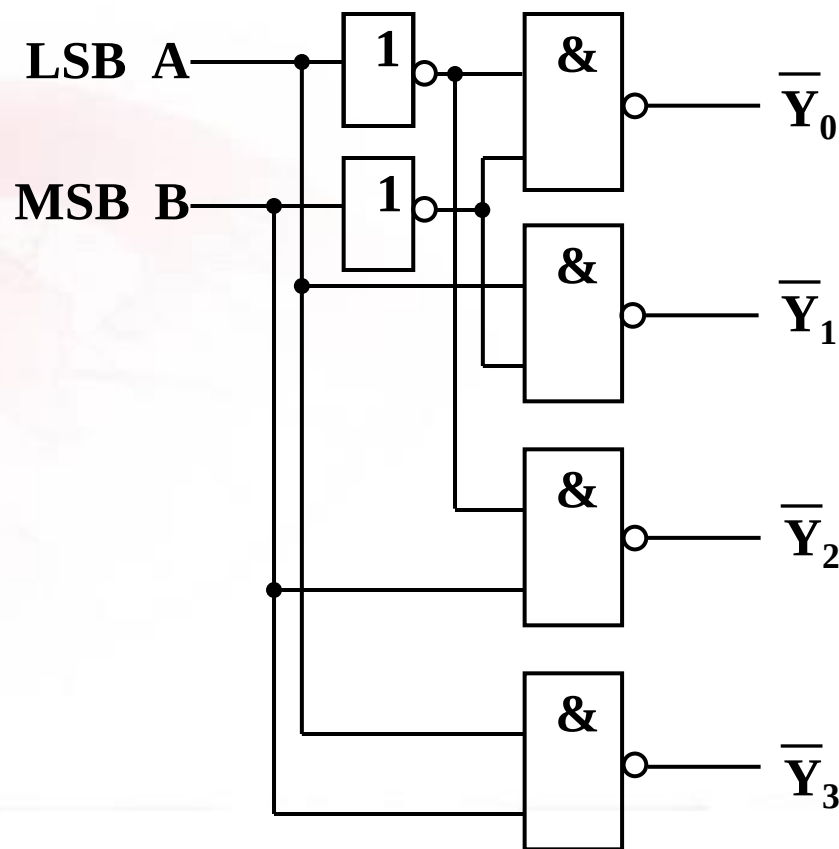
B	A	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

$$\overline{Y}_0 = \overline{\overline{B}\overline{A}} = \overline{m}_0$$

$$\overline{Y}_1 = \overline{\overline{B}A} = \overline{m}_1$$

$$\overline{Y}_2 = \overline{B\overline{A}} = \overline{m}_2$$

$$\overline{Y}_3 = \overline{BA} = \overline{m}_3$$





由前面分析容易得出:

① **高电平**输出有效二进制译码器, 输出逻辑表达式为:

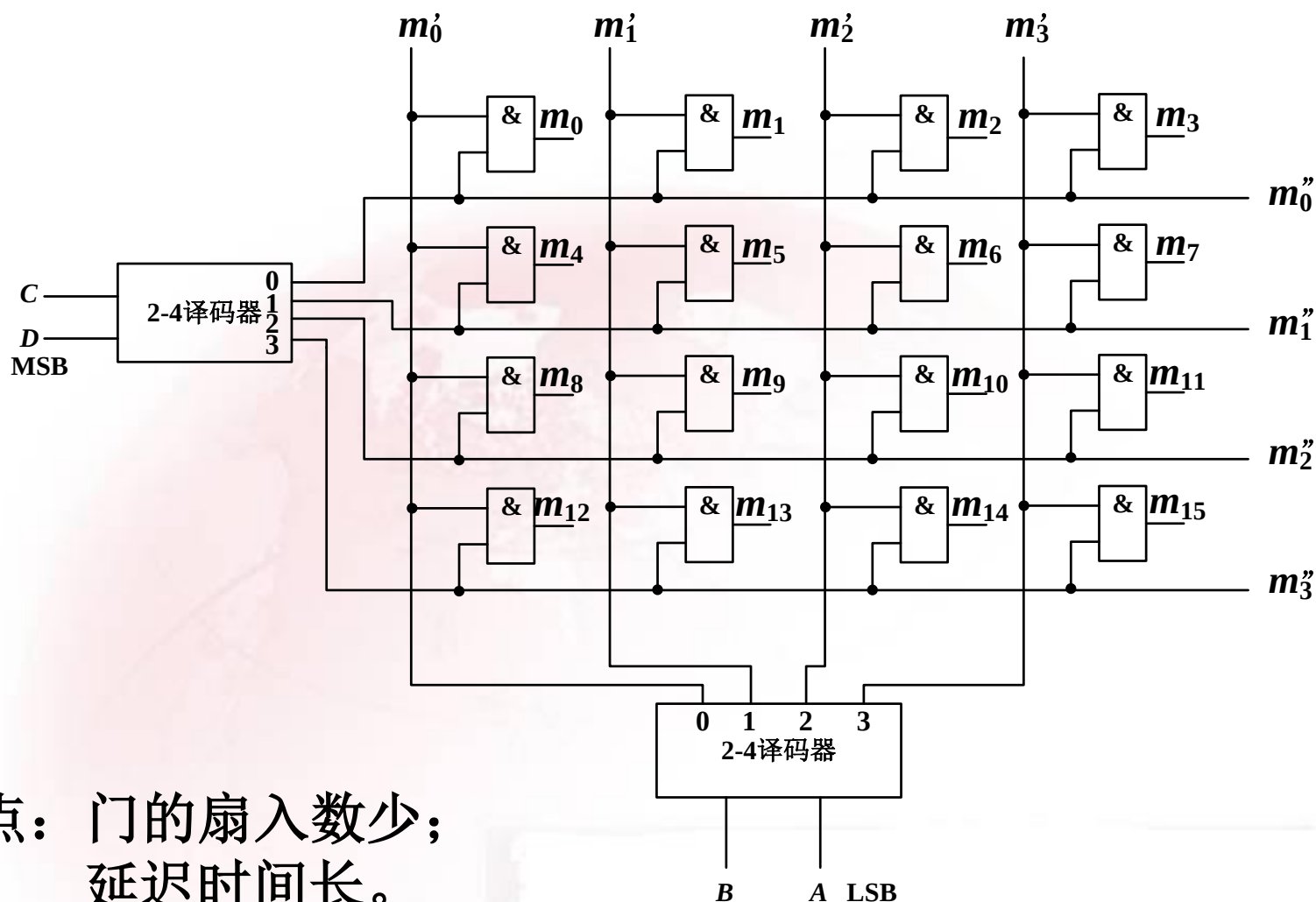
$$Y_i = m_i \quad (m_i \text{ 为输入变量所对应的最小项})$$

② **低电平**输出有效二进制译码器, 输出逻辑表达式为:

$$\bar{Y}_i = \bar{m}_i \quad (m_i \text{ 为输入变量所对应的最小项})$$



译码器的另一种结构：矩阵式结构

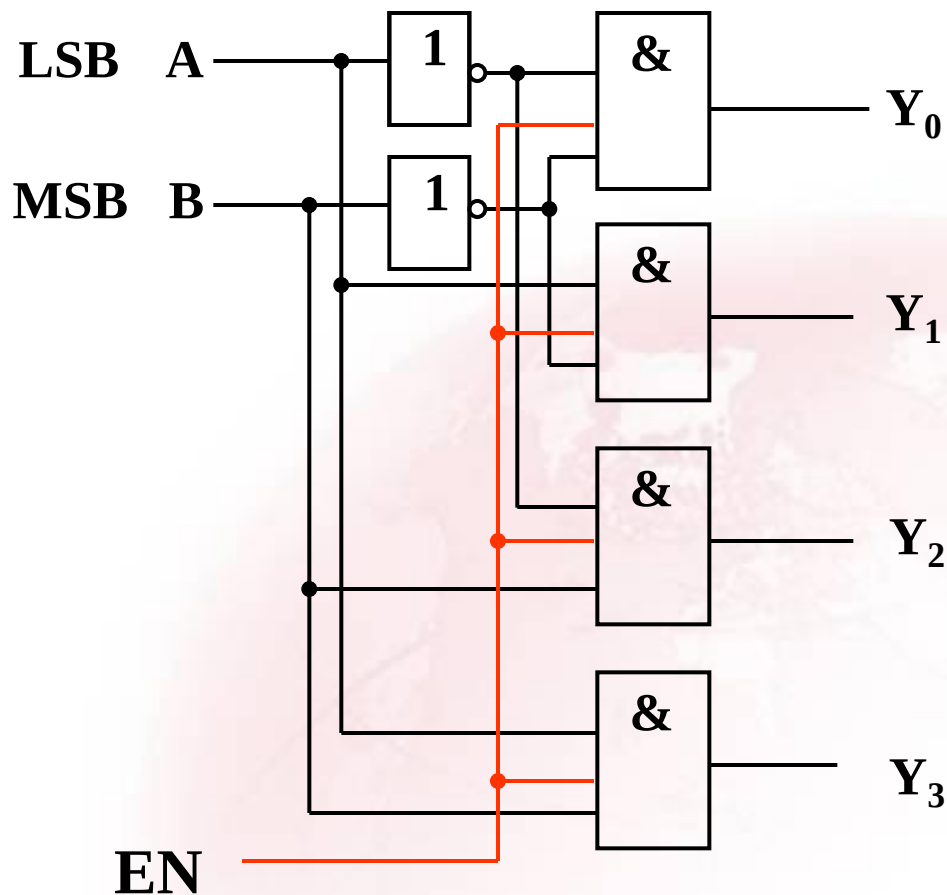


特点：门的扇入数少；
延迟时间长。



2. 译码器的使能控制输入端

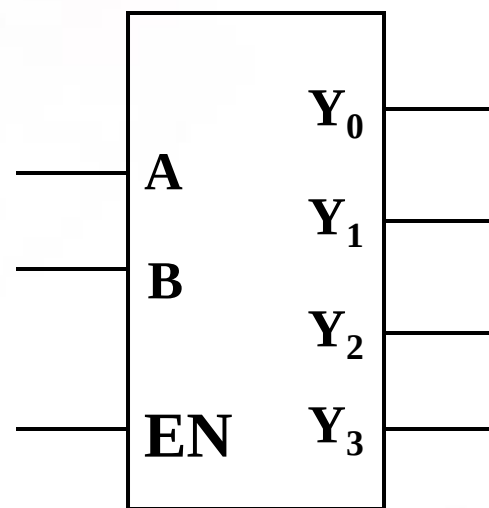
- 1) 利用使能输入控制端，既能使电路正常工作，也能使电路处于禁止工作状态；
- 2) 利用使能输入控制端，能实现译码器容量扩展。



逻辑图

EN为使能控制输入端，
EN=0，输出均为0；
EN=1，输出译码信号。

电路满足： $Y_i = m_i EN$



逻辑符号



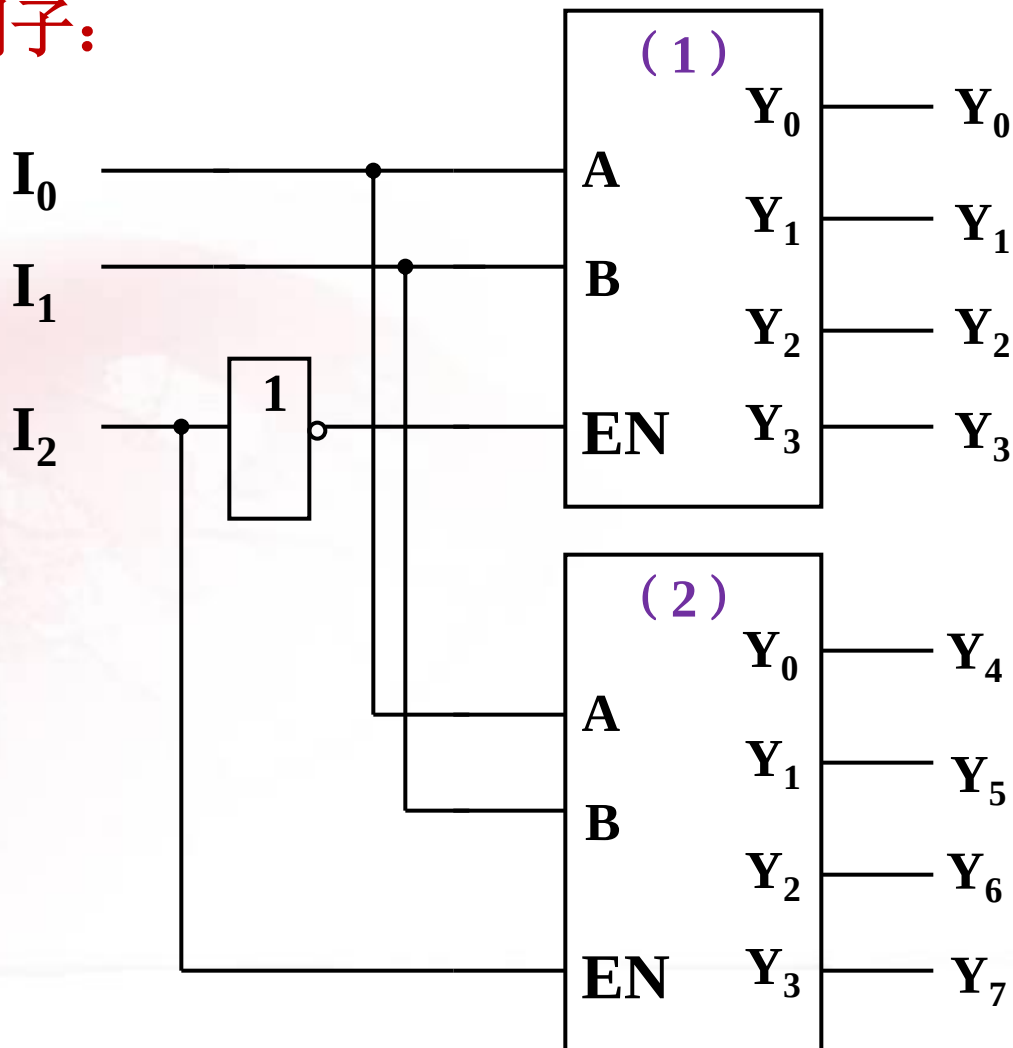
■ 利用使能端实现扩展的例子：

▲ 由两片2线—4线译码器组成3线—8线译码器

当 $I_2=0$ 时，(1)片工作，
(2)片禁止。

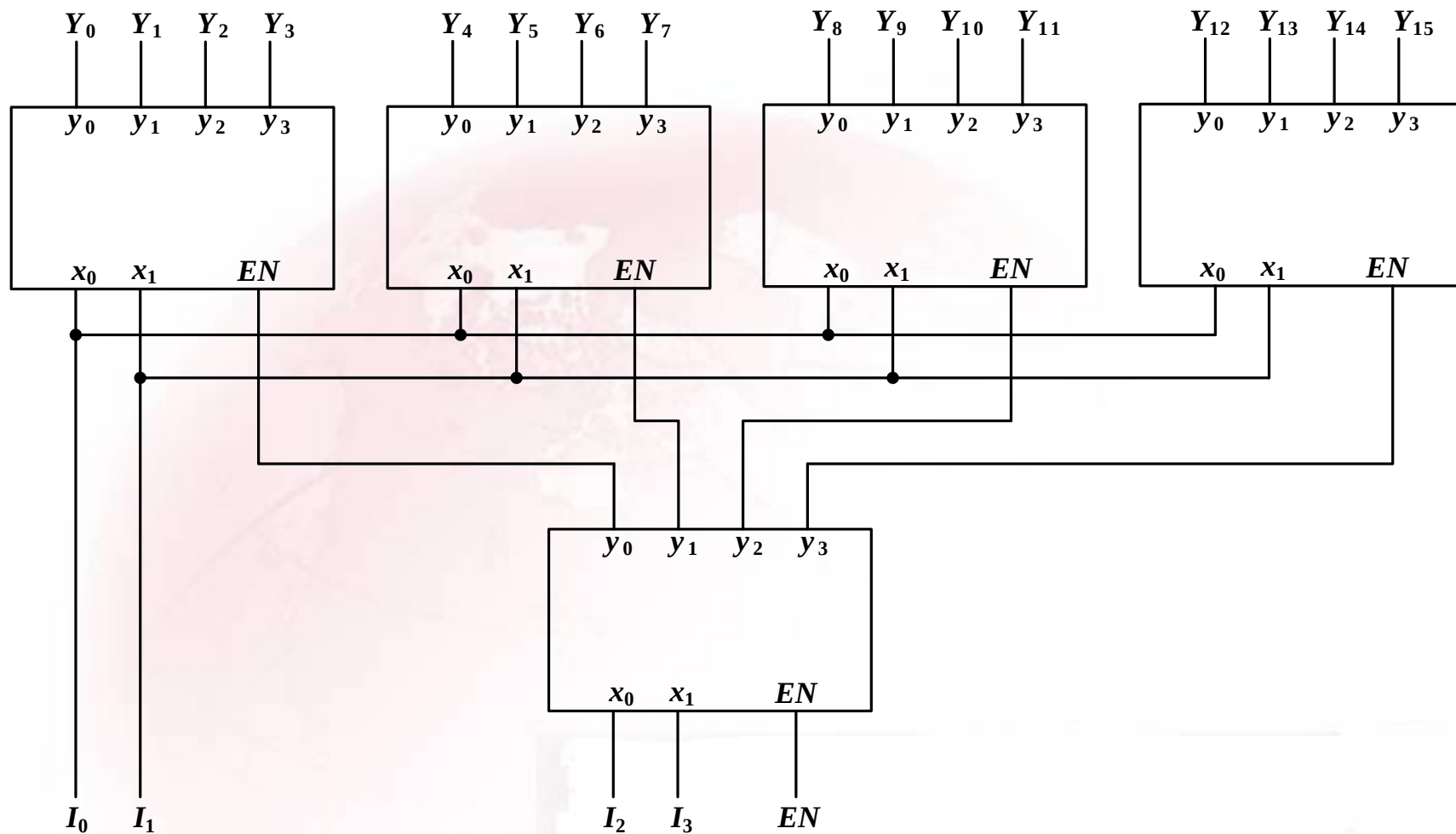
当 $I_2=1$ 时，(1)片禁止，
(2)片工作。

I_2	I_1	I_0	输出
0	0	0~1	$Y_0 \sim Y_3$
1	0	0~1	$Y_4 \sim Y_7$





▲ 由2线—4线译码器组成4线—16线译码器



4.3.2 二—十进制译码器 (常称4线—10线译码器)

真值表

输入BCD码

输出十个高、低电平

A ₃	A ₂	A ₁	A ₀	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$	$\overline{Y}_4$	$\overline{Y}_5$	$\overline{Y}_6$	$\overline{Y}_7$	$\overline{Y}_8$	$\overline{Y}_9$
0	0	0	0	0	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1	1	1
0	1	0	1	1	1	1	1	1	0	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1	1
0	1	1	1	1	1	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	1	1	0	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0
1	0	1	0	1	1	1	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

输出低电平有效

伪码



4线-10线译码器逻辑表达式:

$$\overline{Y}_0 = \overline{A_3} \overline{A_2} \overline{A_1} \overline{A_0}$$

$$\overline{Y}_1 = \overline{A_3} \overline{A_2} \overline{A_1} A_0$$

$$\overline{Y}_2 = \overline{A_3} \overline{A_2} A_1 \overline{A_0}$$

$$\overline{Y}_3 = \overline{A_3} \overline{A_2} A_1 A_0$$

$$\overline{Y}_4 = \overline{A_3} A_2 \overline{A_1} \overline{A_0}$$

$$\overline{Y}_5 = \overline{A_3} A_2 \overline{A_1} A_0$$

$$\overline{Y}_6 = \overline{A_3} A_2 A_1 \overline{A_0}$$

$$\overline{Y}_7 = \overline{A_3} A_2 A_1 A_0$$

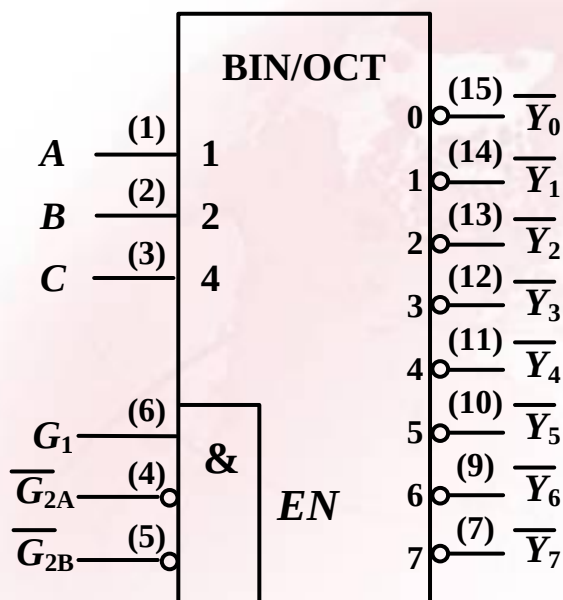
$$\overline{Y}_8 = A_3 \overline{A_2} \overline{A_1} \overline{A_0}$$

$$\overline{Y}_9 = A_3 \overline{A_2} \overline{A_1} A_0$$

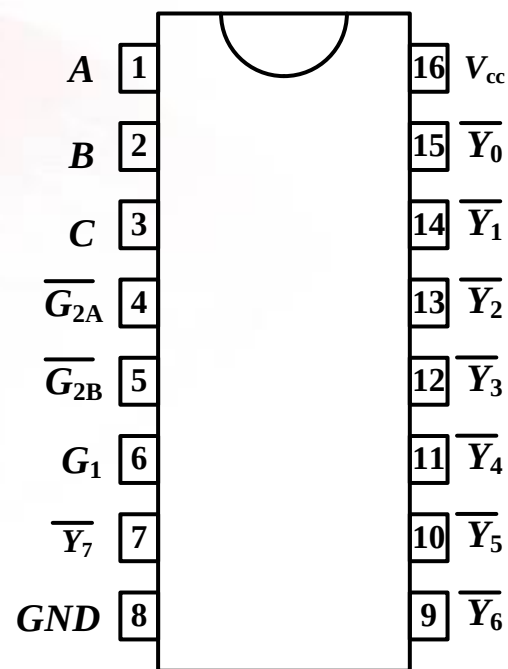


4.3.3 通用译码器集成电路

(1) 74138 带使能端3线—8线译码器



逻辑图



引脚图



74138功能表

G_1	$\overline{G_2}^*$	C	B	A	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$	$\overline{Y}_4$	$\overline{Y}_5$	$\overline{Y}_6$	$\overline{Y}_7$
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0
×	1	×	×	×	1	1	1	1	1	1	1	1
0	×	×	×	×	1	1	1	1	1	1	1	1

$$\overline{G_2}^* = \overline{G_{2A}} + \overline{G_{2B}}$$



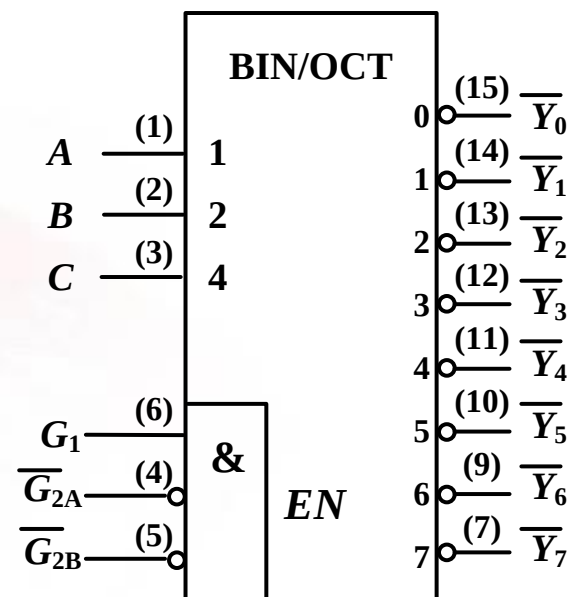
74138特性:

① 74138的**逻辑表达式**为:

$$\overline{Y_i} = m_i \overline{G_1} \overline{G_{2A}} \overline{G_{2B}}$$

② 电路**输出低电平有效**;

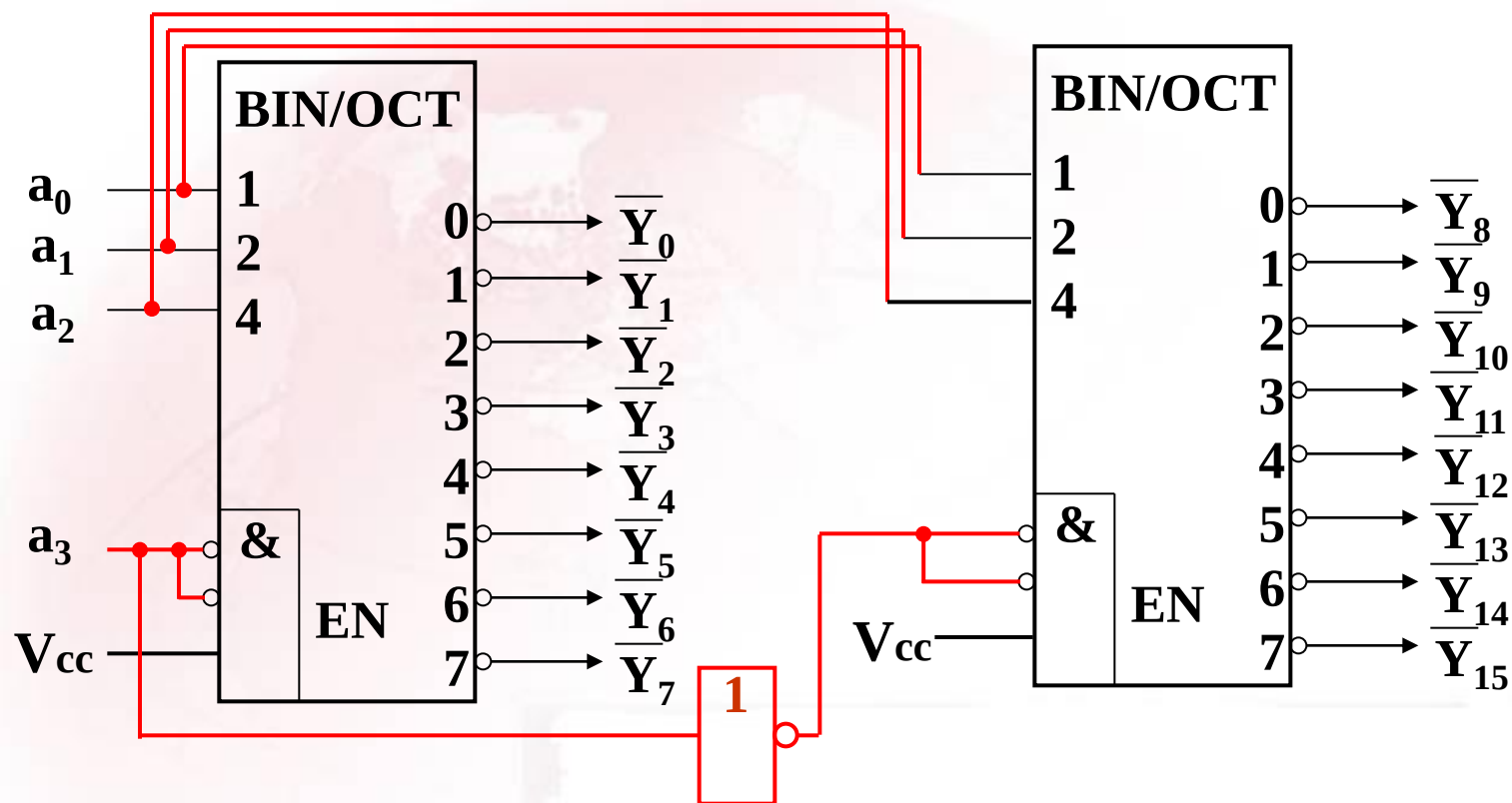
③ $G_1 \overline{G_{2A}} \overline{G_{2B}} = 100$, **电路工作**; 否则, 电路**禁止工作**, 电路输出均为1。





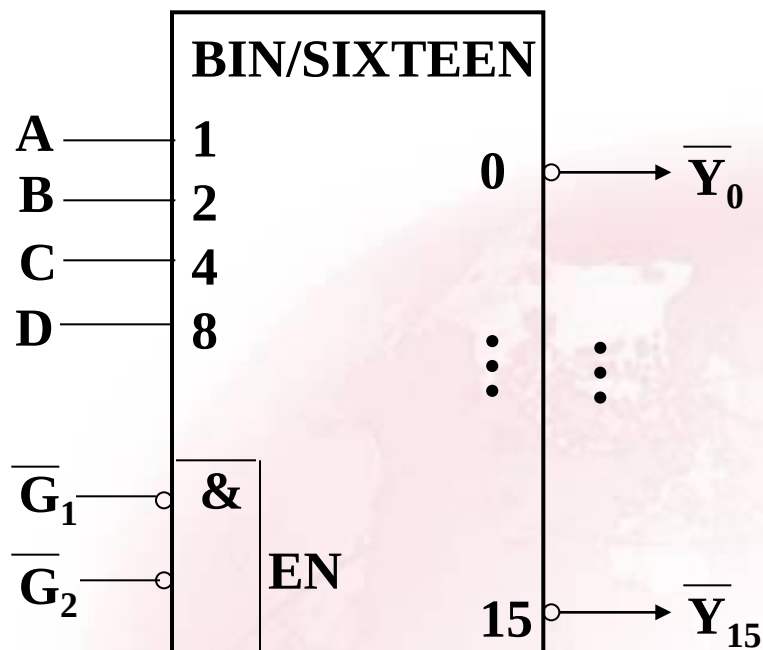
74138扩展举例:

试用两片74138构成4线—16线译码器



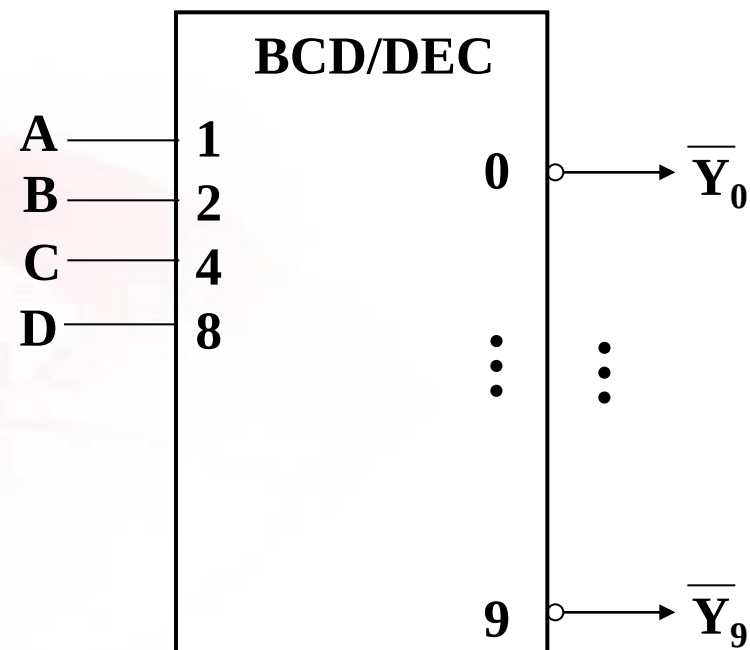


(2) 74154



4线—16线译码器

(3) 7442



4线—10线译码器



4.3.4 译码器应用举例

1. 译码器实现组合逻辑函数

原理： 二进制译码器能产生输入信号的全部最小项，而所有组合逻辑函数均可写成最小项之和的形式。

例1 试用3线– 8线译码器和逻辑门实现下列函数

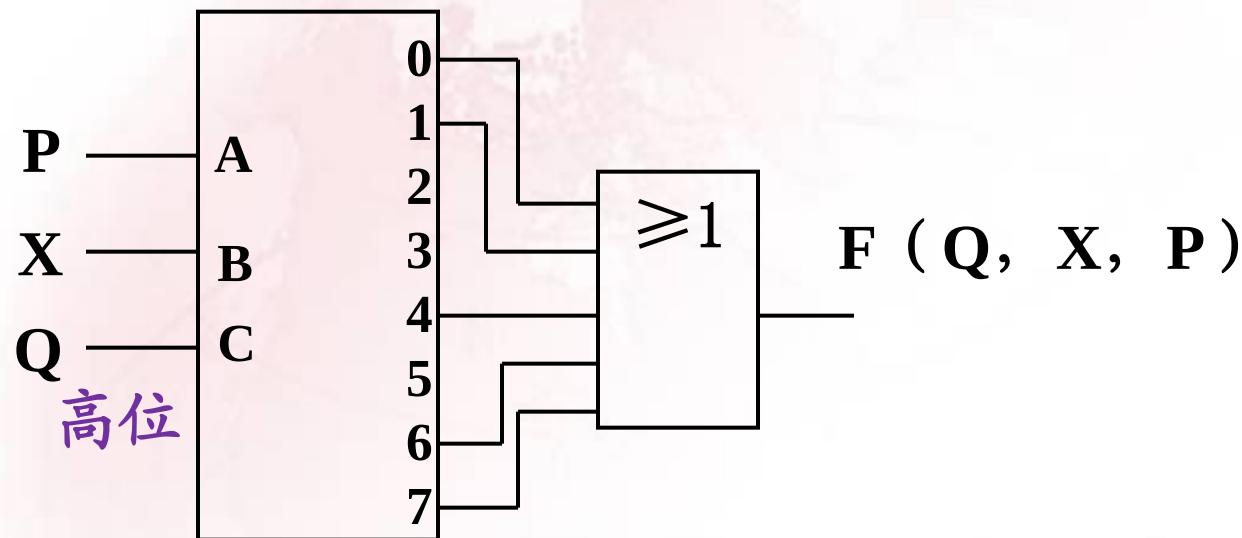
$$\begin{aligned} F(Q,X,P) &= \Sigma m(0, 1, 4, 6, 7) \\ &= \Pi M(2, 3, 5) \end{aligned}$$



解题的几种方法:

① 利用高电平输出有效的译码器和或门。

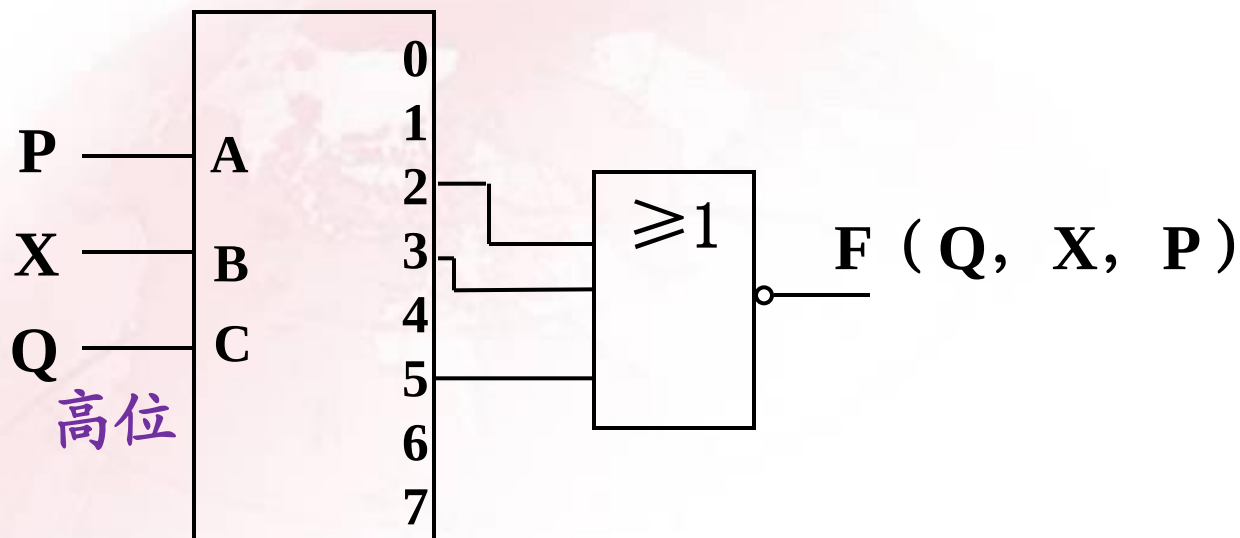
$$F(Q, X, P) = m_0 + m_1 + m_4 + m_6 + m_7$$





② 利用高电平输出有效的译码器和或非门。

$$F(Q,X,P) = \Sigma m(0, 1, 4, 6, 7) = \overline{m_2 + m_3 + m_5}$$

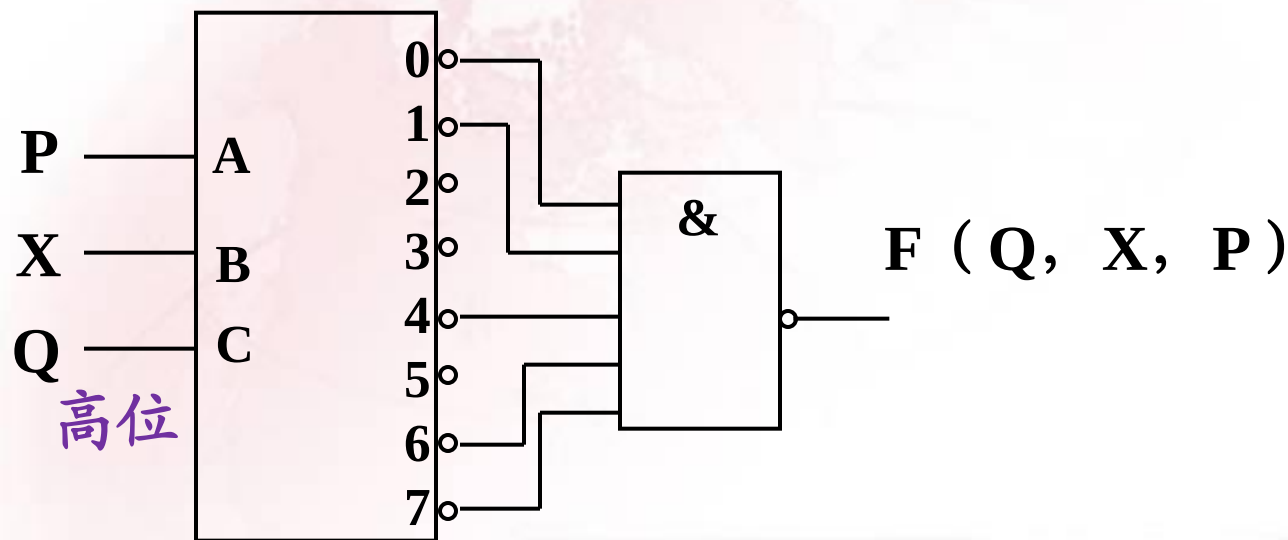




③ 利用低电平输出有效的译码器和与非门。

$$F(Q,X,P)=\Sigma m(0, 1, 4, 6, 7)$$

$$F(Q,X,P)=\overline{m_0}\overline{m_1}\overline{m_4}\overline{m_6}\overline{m_7}$$

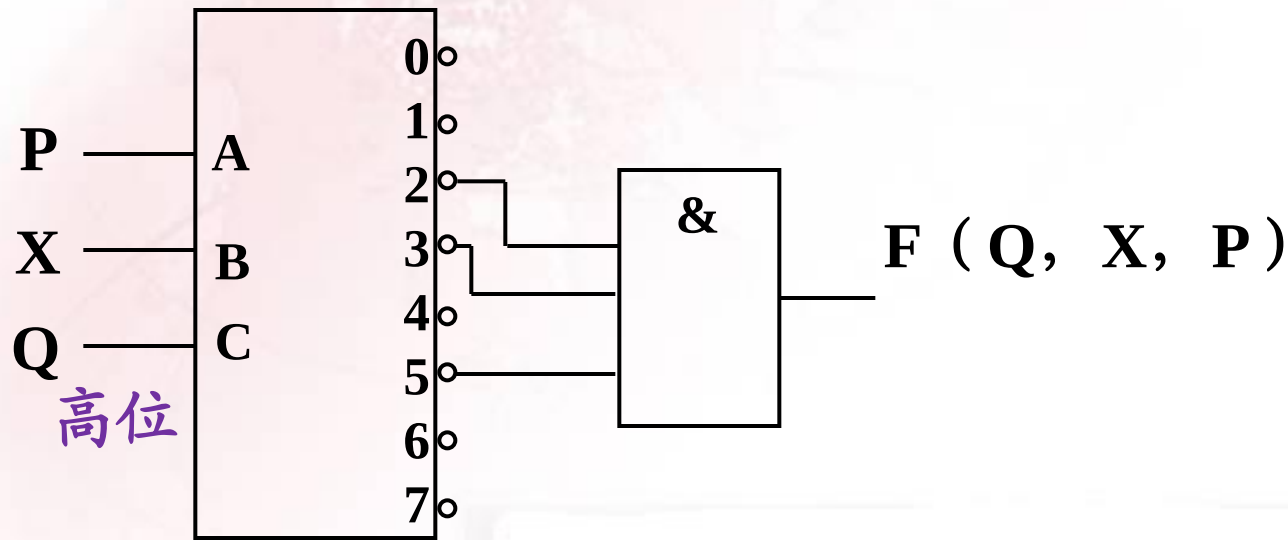




④利用低电平输出有效的译码器和与门。

$$F(Q,X,P) = \Sigma m(0, 1, 4, 6, 7) = \overline{m_2 + m_3 + m_5}$$

$$F(Q,X,P) = \overline{m_2} \overline{m_3} \overline{m_5}$$

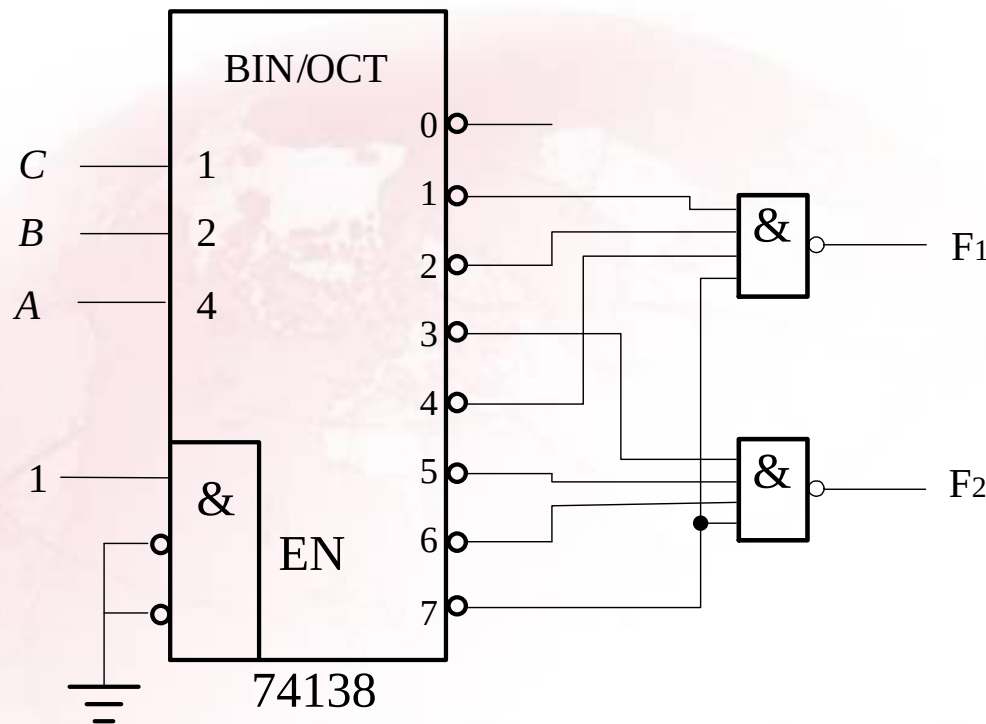




例2： 试用74138和少量门实现逻辑函数：

$$F_1(A,B,C) = \sum m(1,2,4,7) \quad F_2(A,B,C) = \sum m(3,5,6,7)$$

① 用74138和少量与非门实现该逻辑函数

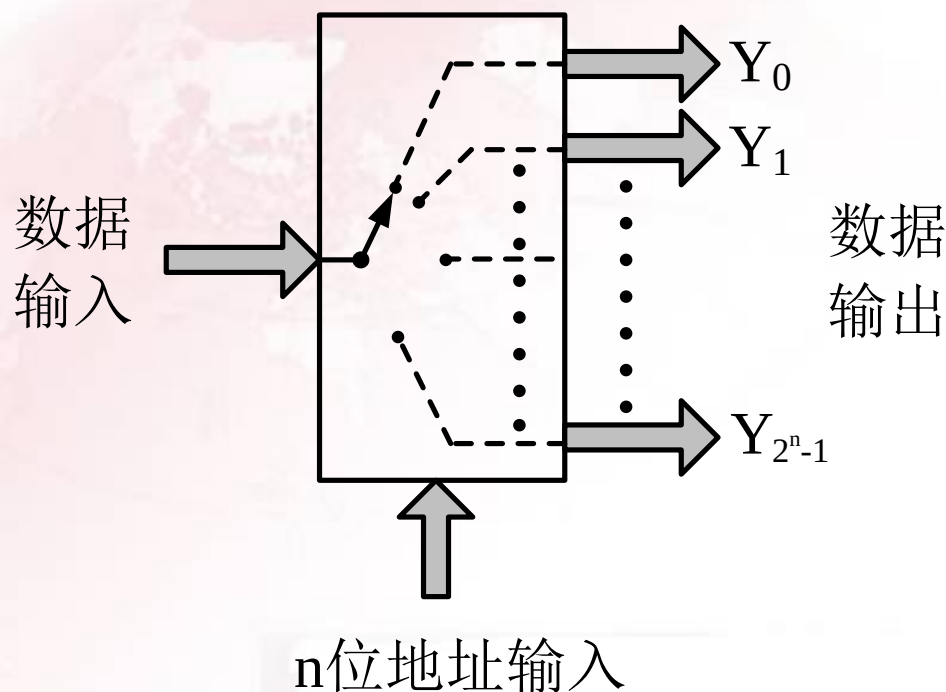


② 若用74138和少量与门如何实现该逻辑函数？



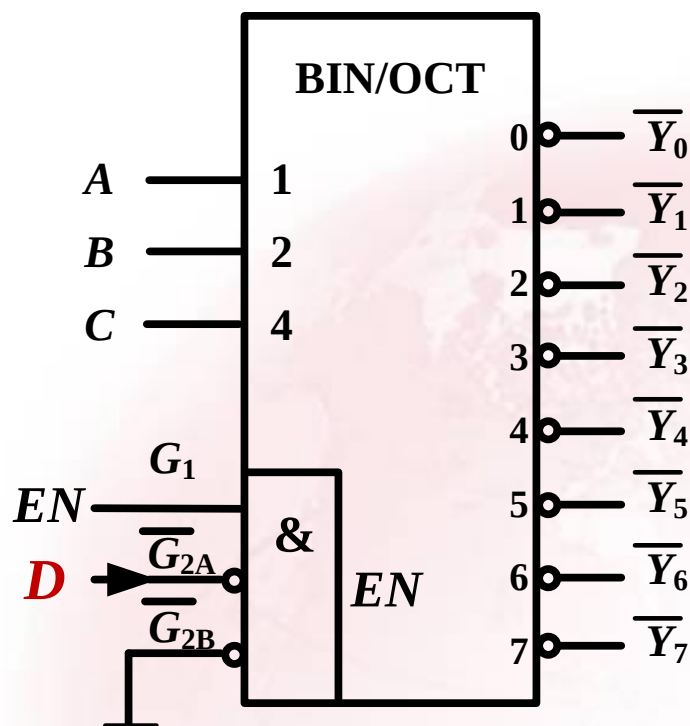
2. 数据分配器

数据分配是将一路数据源输入的数据根据需要送到不同的输出端上去，实现数据分配功能的逻辑电路称为数据分配器。





数据分配器一般用带使能控制端的二进制译码器实现。



74138输出表达式:

$$\overline{Y_i} = \overline{m_i G_1 G_{2A} G_{2B}}$$

分配器输出表达式:

$$\overline{Y_i} = \overline{m_i EN D}$$

用74138作为数据分配器



74138译码器作为数据分配器时的功能表

G_1	$\overline{G}_{2A}$	C	B	A	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$	$\overline{Y}_4$	$\overline{Y}_5$	$\overline{Y}_6$	$\overline{Y}_7$
1	D	0	0	0	D	1	1	1	1	1	1	1
1	D	0	0	1	1	D	1	1	1	1	1	1
1	D	0	1	0	1	1	D	1	1	1	1	1
1	D	0	1	1	1	1	1	D	1	1	1	1
1	D	1	0	0	1	1	1	1	D	1	1	1
1	D	1	0	1	1	1	1	1	1	D	1	1
1	D	1	1	0	1	1	1	1	1	1	D	1
1	D	1	1	1	1	1	1	1	1	1	1	D
0	×	×	×	×	1	1	1	1	1	1	1	1

$\overline{G}_{2B}=0$



4.3.5 显示译码器

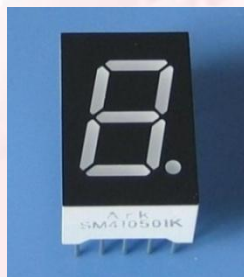
常用数字显示器:

- 半导体显示器，也称发光二极管显示器；
- 液体数字显示器，如液晶显示器、电泳显示器等。

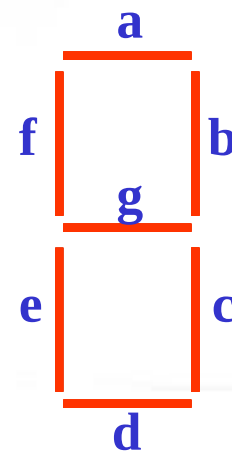


1. 半导体数码管

半导体数码管的7个发光段是7个条状的发光二极管（Light Emitting Diode，简称LED）。



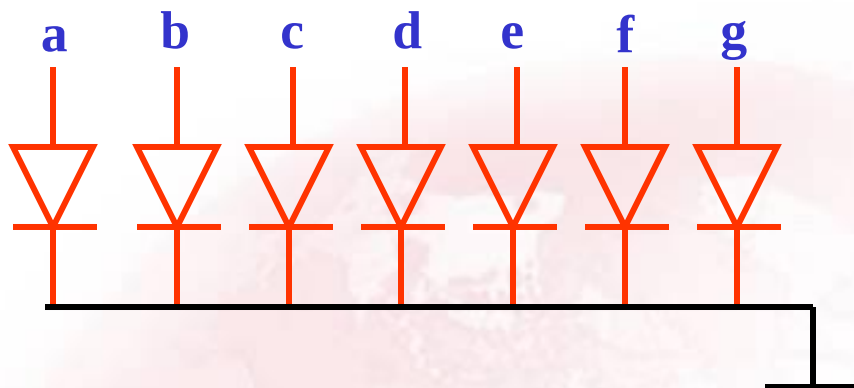
七段显示器
(LED)





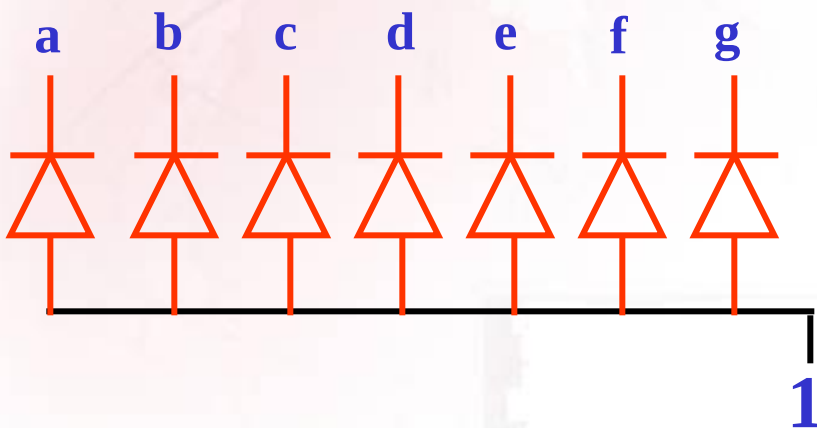
七段数码管的两种连接方法：

① 共阴

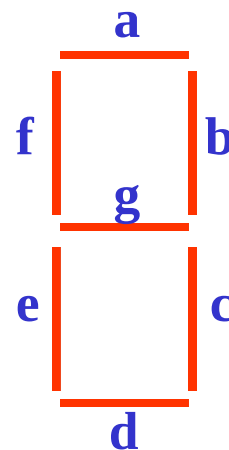


阳极加高
电平字段
亮。

② 共阳



阴极加低
电平字段
亮。





半导体数码管的工作电压比较低（ $1.5 \sim 3\text{V}$ ），能直接用TTL或CMOS集成电路驱动。除电压比较低外，半导体数码管还具有体积小、寿命长、可靠性高等优点，而且响应时间短（一般不超过 $0.1\mu\text{s}$ ），亮度也比较高。LED显示器的缺点是工作电流大，每一段的工作电流在 10mA 左右。



2. 液晶显示器（Liquid Crystal Display，简称LCD）

19世纪末，奥地利植物学家就发现了**液晶**，即液态的晶体，也就是说一种物质同时具备了**液体的流动性和类似晶体的某种排列特性**。

在电场的作用下，液晶分子的排列会产生变化，从而影响到它的光学性质（阻隔或使光束顺利通过），这种现象叫做**电光效应**。利用液晶的电光效应，英国科学家制造了第一块液晶显示器。

液晶显示器通过控制可见光的反射来达到显示目的。

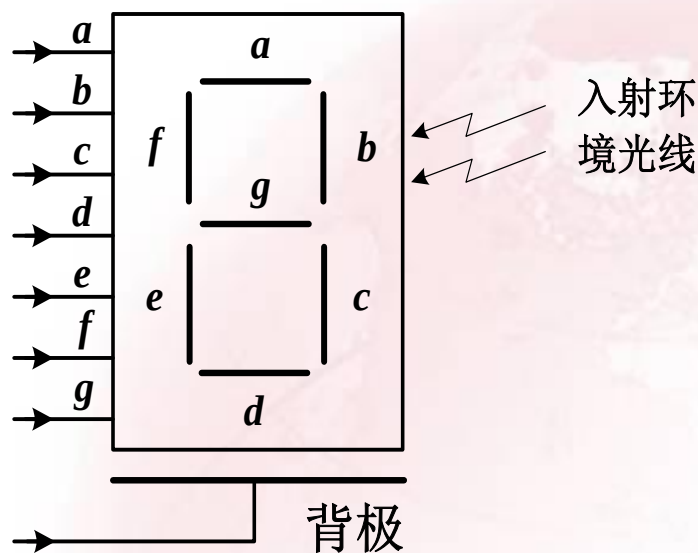
液晶显示器分两类：

反射式：液晶显示器使用的可见光是环境光线。

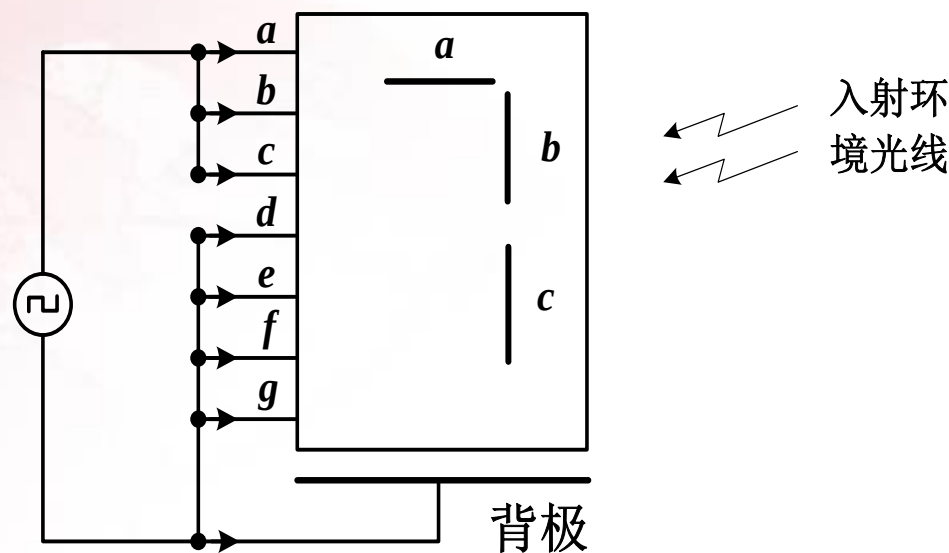
背光式：液晶显示器的可见光则由在显示器内特制小光源提供。



LCD须用低频交流信号驱动，一般使用方波信号，工作频率约为25~60Hz，信号幅值可以很低，在1V以下仍能工作。



七段液晶显示器示意图

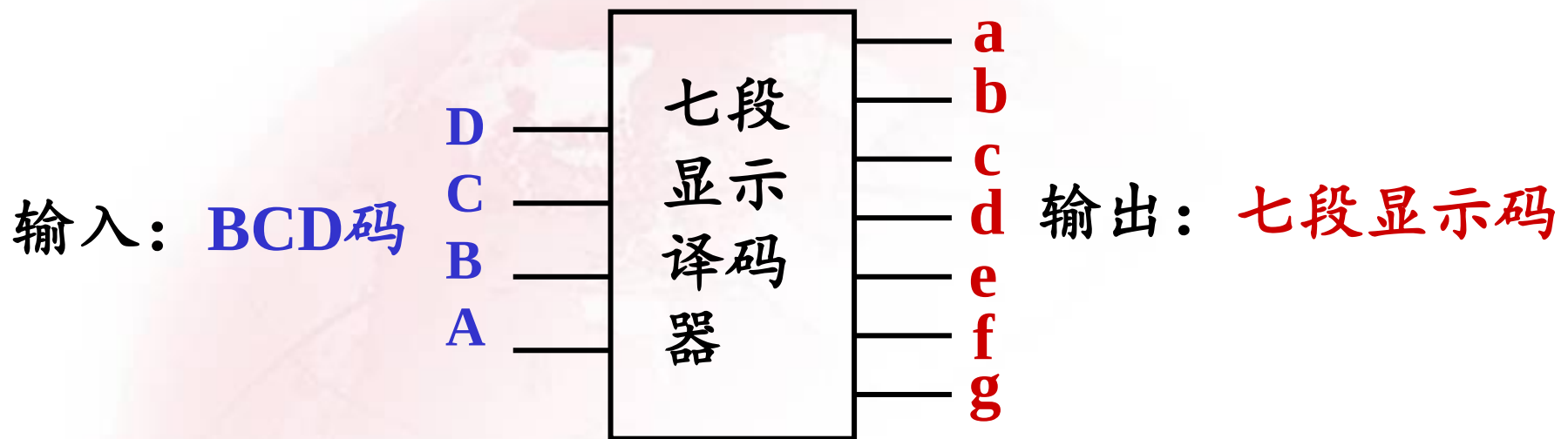


加驱动电压，显示‘7’字形



3. 显示译码器设计

功能：将表示数字的**BCD**码转换成**七段显示码**。

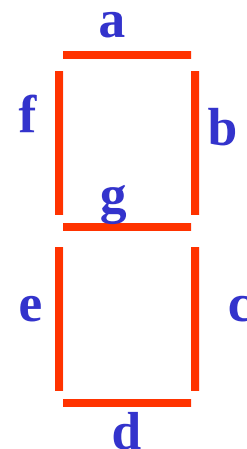




显示译码器设计步骤:

(以输入**8421BCD码**、输出驱动**共阳显示器**为例)

- ① 列真值表;
- ② 化简、写最简函数表达式;
- ③ 画电路图。





真 值 表

D	C	B	A	a	b	c	d	e	f	g	显示
0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	1	0	0	1	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0	2
0	0	1	1	0	0	0	0	1	1	0	3
0	1	0	0	1	0	0	1	1	0	0	4
0	1	0	1	0	1	0	0	1	0	0	5
0	1	1	0	0	1	0	0	0	0	0	6
0	1	1	1	0	0	0	1	1	1	1	7
1	0	0	0	0	0	0	0	0	0	0	8
1	0	0	1	0	0	0	0	1	0	0	9

化简后表达式:

$$a = A\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C$$

$$b = A\bar{B}C + \bar{A}BC$$

$$c = \bar{A}B\bar{C}$$

$$d = \bar{A}\bar{B}C + ABC + A\bar{B}\bar{C}\bar{D}$$

$$e = A + \bar{A}\bar{B}C$$

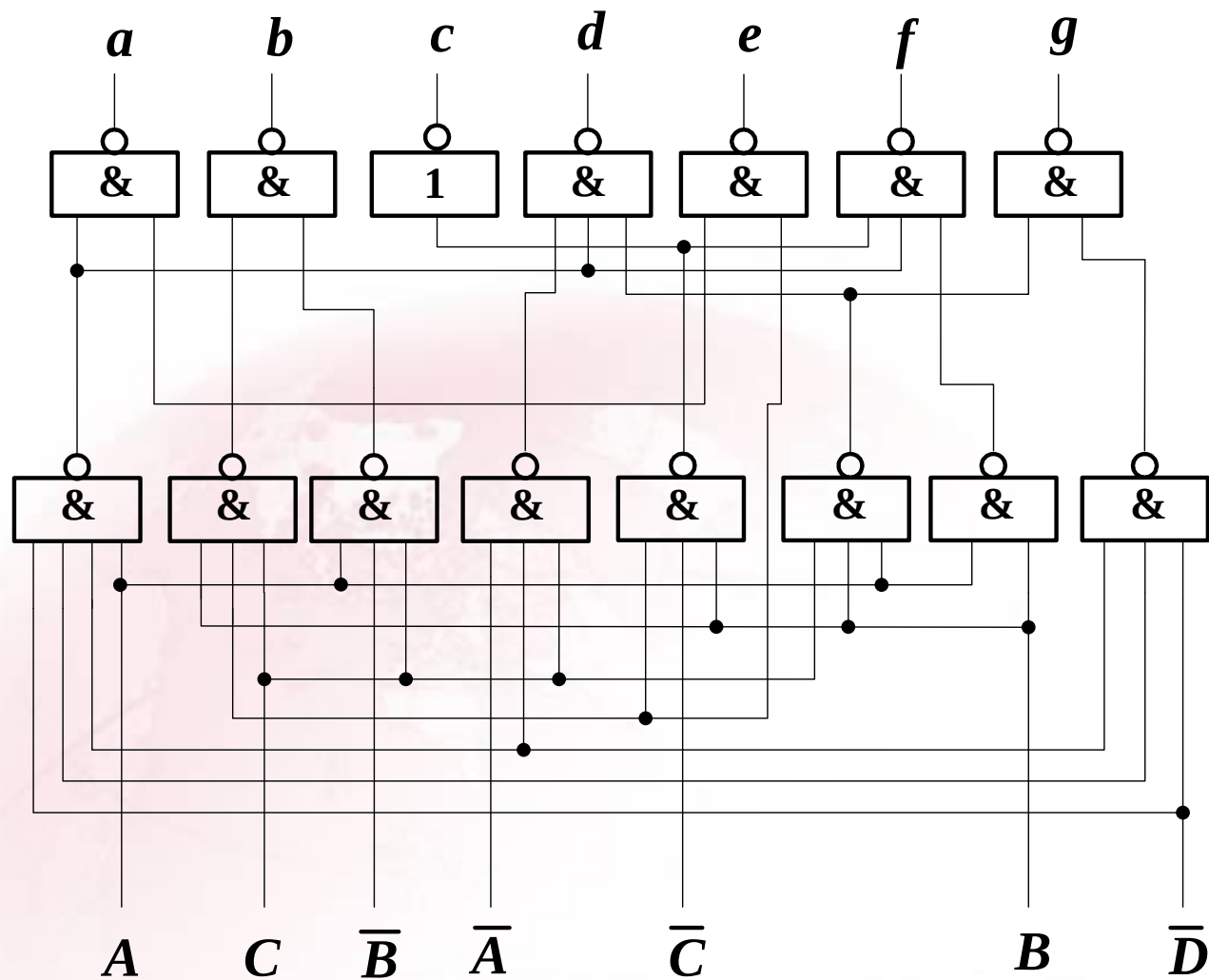
$$f = AB + A\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}$$

$$g = ABC + \bar{B}\bar{C}\bar{D}$$

化简说明:

① 利用了无关项;

② 考虑了多输出逻辑函数化简中的公共项。



七段显示译码器逻辑图



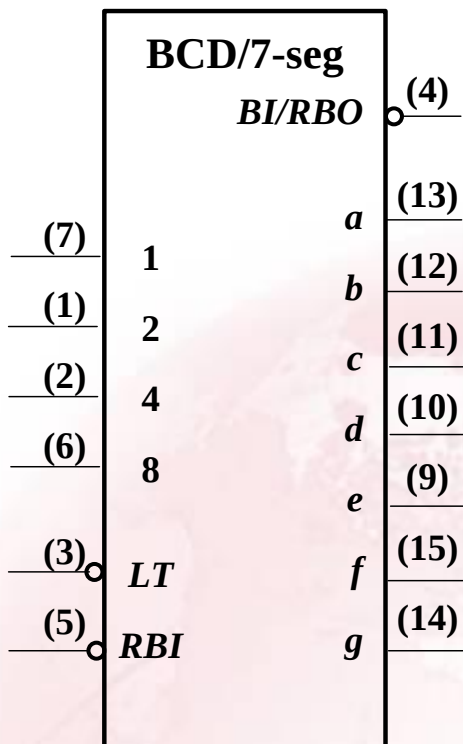
4. 通用七段显示译码器集成电路

常用的七段显示译码器集成电路有7446、7447、7448、7449和4511等。下面主要介绍七段显示译码器7448。

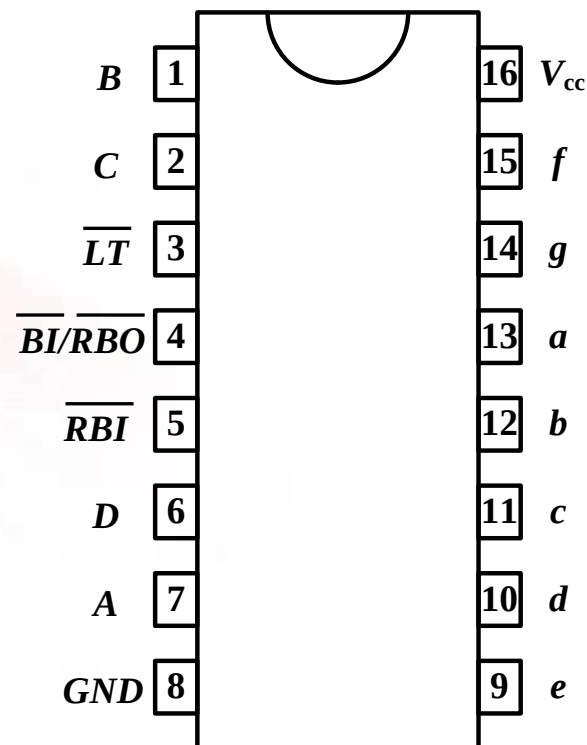
七段显示译码器7448输出高电平有效，用以驱动共阴极显示器。



BCD
码输入



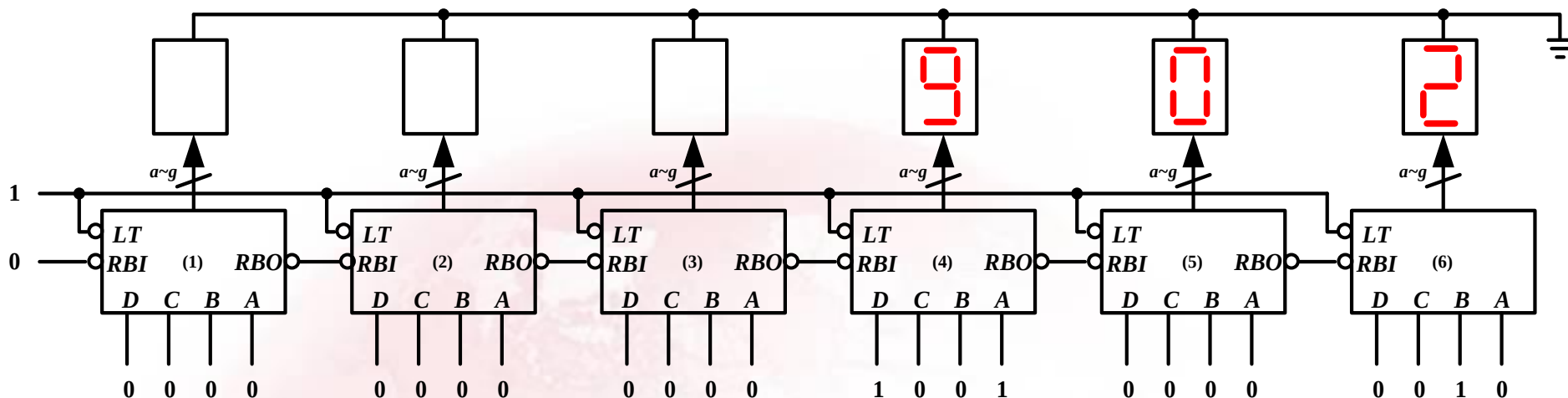
逻辑图



引脚图



7448实现多位显示



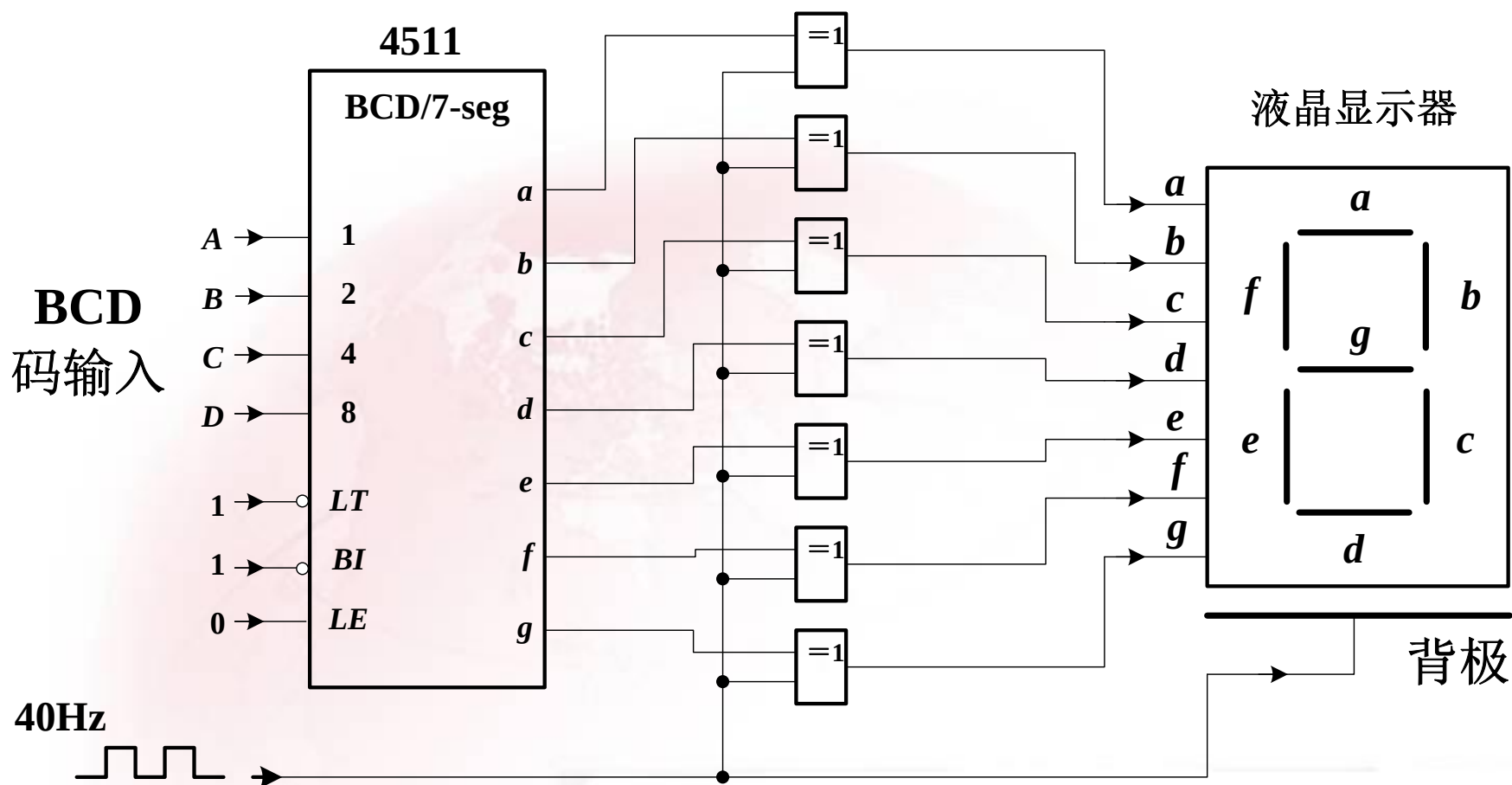
由于第1片的 $\overline{RBI}$ 为0，而 $DCBA=0000$ ，所以满足灭零条件， $\overline{RBO}=0$ 。第2、3片也满足灭零条件。

第4、5、6片驱动正常显示。

思考题：如第1片输入 $DCBA$ 不等于0000，2、3两片灭零条件吗？



74HC4511显示译码器驱动液晶数码管的一个例子





4.3.6 译码器的Verilog描述

例：3线-8线译码器74138的Verilog程序代码。

```
module Vr74138(G1,G2A_L,G2B_L,A,Y_L);  
    input G1,G2A_L,G2B_L;  
    input [2:0] A;  
    output [7:0] Y_L;  
  
    reg [7:0] Y_L;  
  
    always @ (G1 or G2A_L or G2B_L or A)  
    begin  
        if (G1 & ~G2A_L & ~G2B_L)  
            case(A)
```



```
0: Y_L=8'b01111111;  
1: Y_L=8'b10111111;  
2: Y_L=8'b11011111;  
3: Y_L=8'b11101111;  
4: Y_L=8'b11110111;  
5: Y_L=8'b11111011;  
6: Y_L=8'b11111101;  
7: Y_L=8'b11111110;  
default: Y_L=8'b11111111;  
endcase  
else Y_L=8'b11111111;  
end  
endmodule
```



例：七段译码器的Verilog程序代码。

```
module Vr7seg(A,B,C,D,EN, SEGA,SEGB,SEGC,
              SEGD,SEGE,SEGF,SEGG);
input A,B,C,D,EN;
output SEGA,SEGB,SEGC,SEGD,SEGE,SEGF,SEGG;

reg SEGA,SEGB,SEGC,SEGD,SEGE,SEGF,SEGG;
reg [1:7] SEGS;

always @ (A, B, C, D, EN)
begin
    if (EN)
        case({D,C,B,A})
```



```
0: SEGS=7'b1111110;  
1: SEGS=7'b0110000;  
2: SEGS=7'b1101101;  
3: SEGS=7'b1111001;  
4: SEGS=7'b0110011;  
5: SEGS=7'b1011011;  
6: SEGS=7'b0011111;  
7: SEGS=7'b1110000;  
8: SEGS=7'b1111111;  
9: SEGS=7'b1110011;  
  default: SEGS=7'bx;  
endcase  
else SEGS=7'b0;  
  {SEGA,SEGB,SEGC,SEGD,SEGE,SEGF,SEGG}=SEGS;  
end  
endmodule
```



4.4 数据选择器

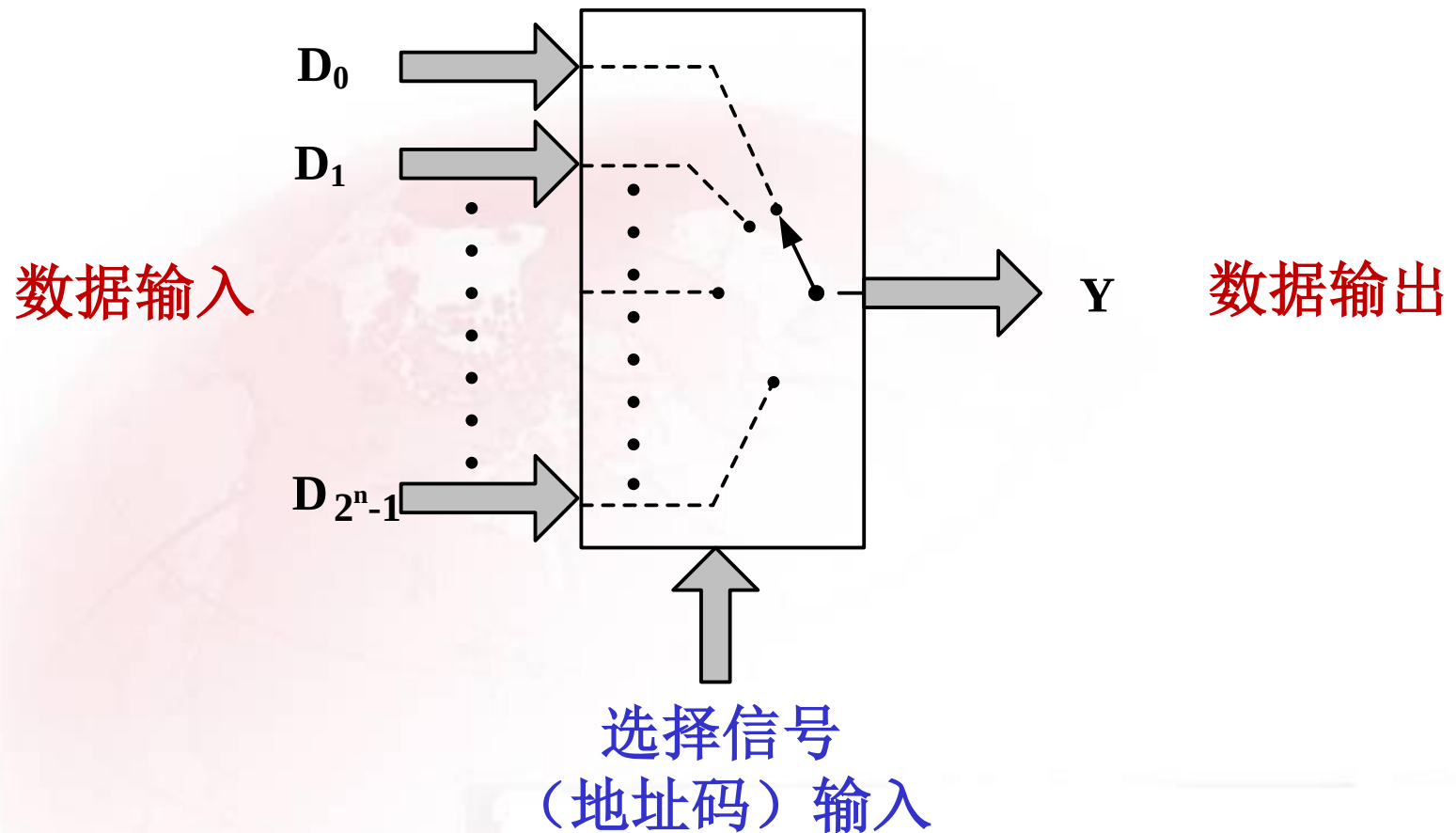
功能：从**多路输入数据**中选择其中的一路送至输出端。

数据选择器简称**MUX**，数据选择器的数据输入端数称为**通道数**。

常见的数据选择器有：二选一、四选一、八选一和十六选一等数据选择器。



数据选择器功能示意图:





4.4.1 数据选择器的电路结构

以四选一数据选择器为例讨论

功能表

A_1	A_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

输出函数表达式:

$$Y = (\bar{A}_1 \bar{A}_0) D_0 + (\bar{A}_1 A_0) D_1 \\ + (A_1 \bar{A}_0) D_2 + (A_1 A_0) D_3$$

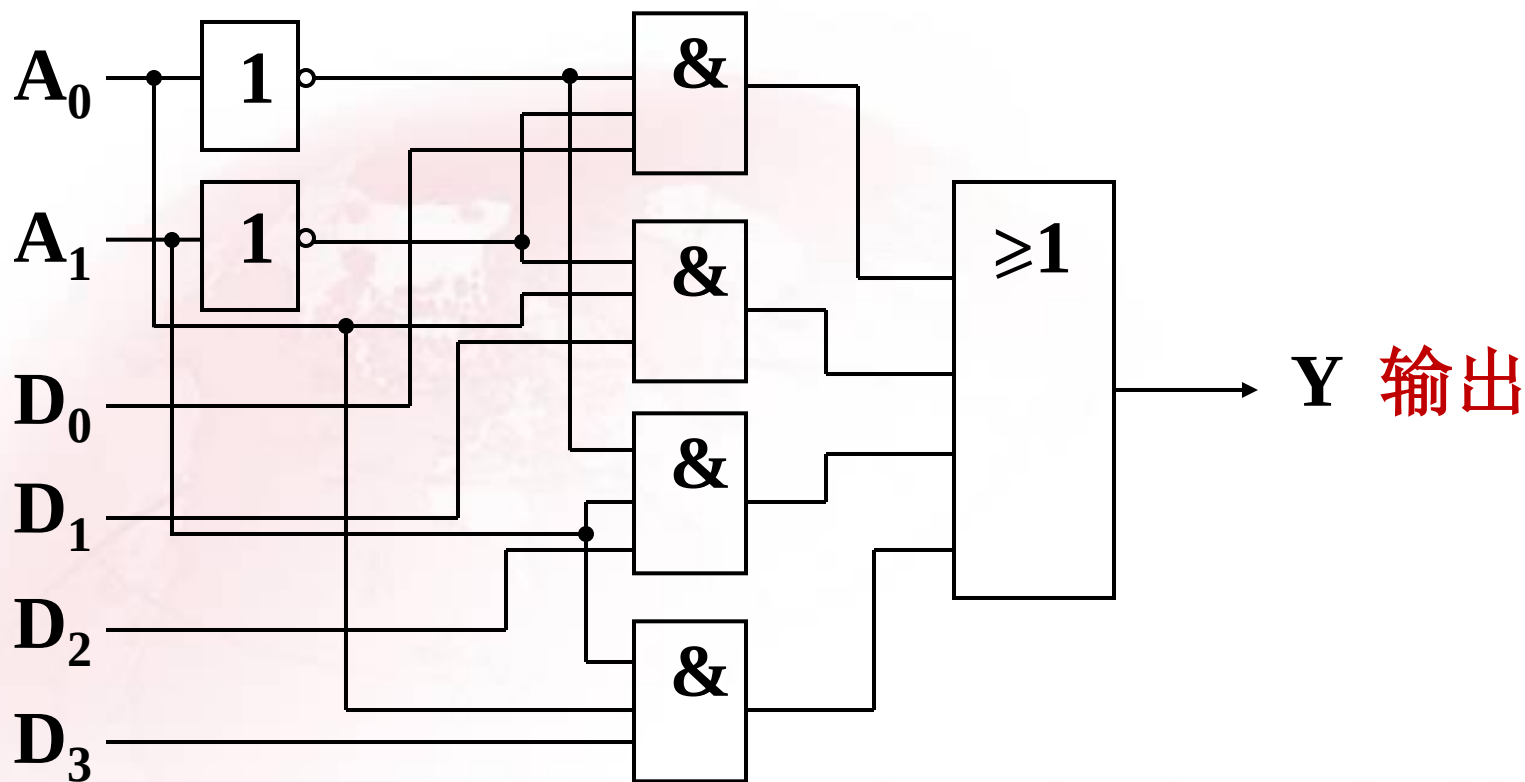
$$Y = \sum_{i=0}^3 m_i D_i$$



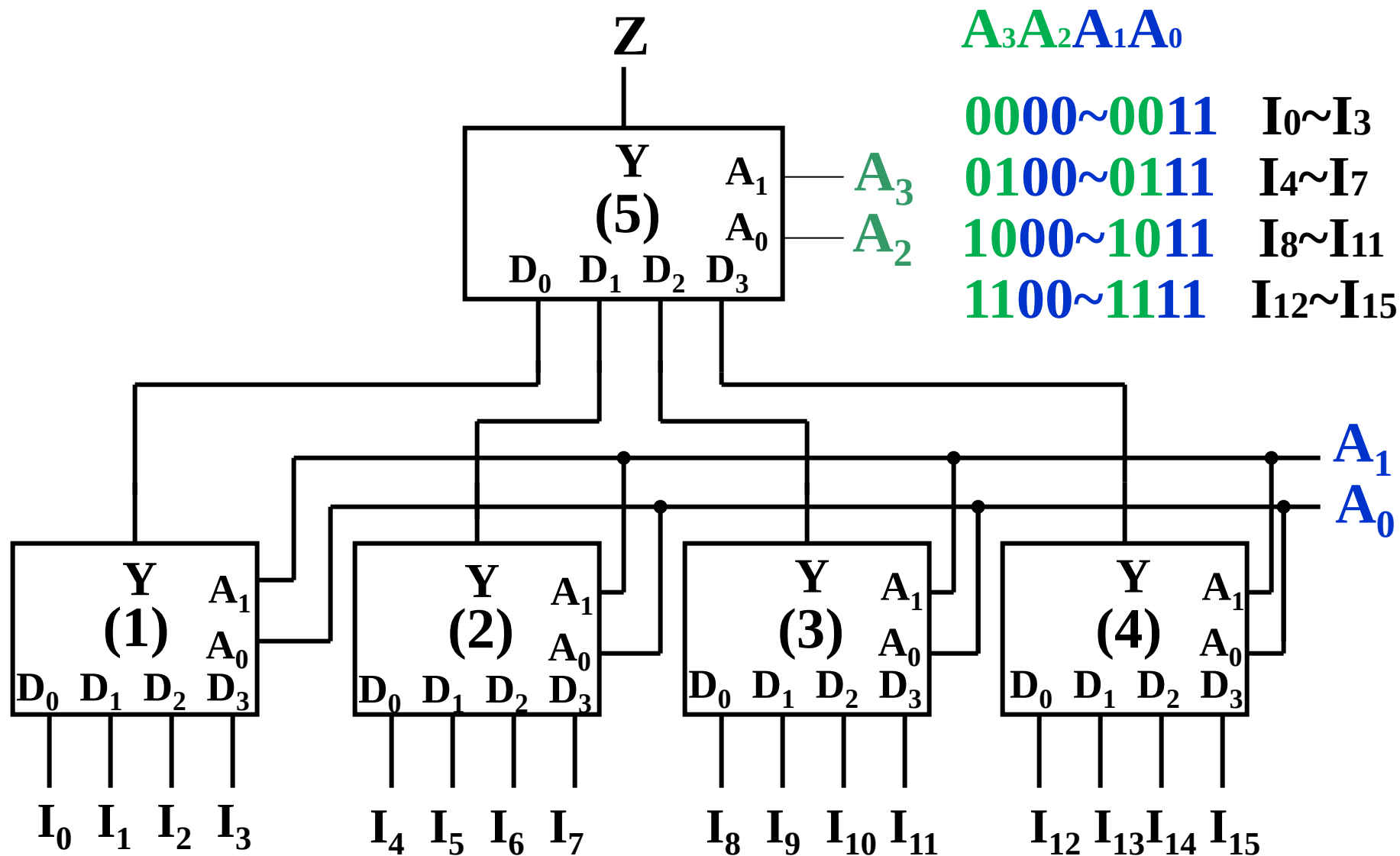
电路图:

地址

数据



数据选择器通道扩展：由四选一数据选择器组成十六选一数据选择器的例子





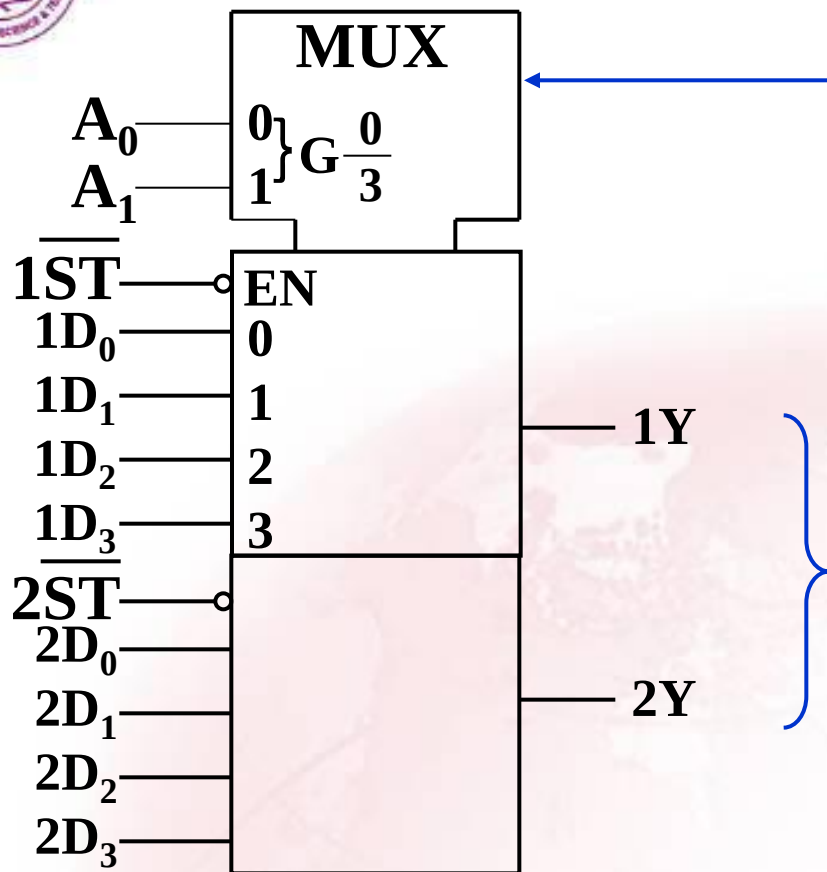
4.4.2 通用数据选择器集成电路

常用MUX集成电路

输入数	TTL	CMOS(数字)	CMOS(模拟)	ECL
16	74150	4515	4067	
2×8	74451		4096	
8	74151	4512	4051	10164
4×4	74453			
2×4	74153	4539	4052	10174
8×2	74604			
4×2	74157	4519		10159

数据选择器的逻辑符号及输入选通端：

以双四选一MUX**74153**、MUX**74HC4539**和八选一MUX**74151**说明。



74153

公共控制框

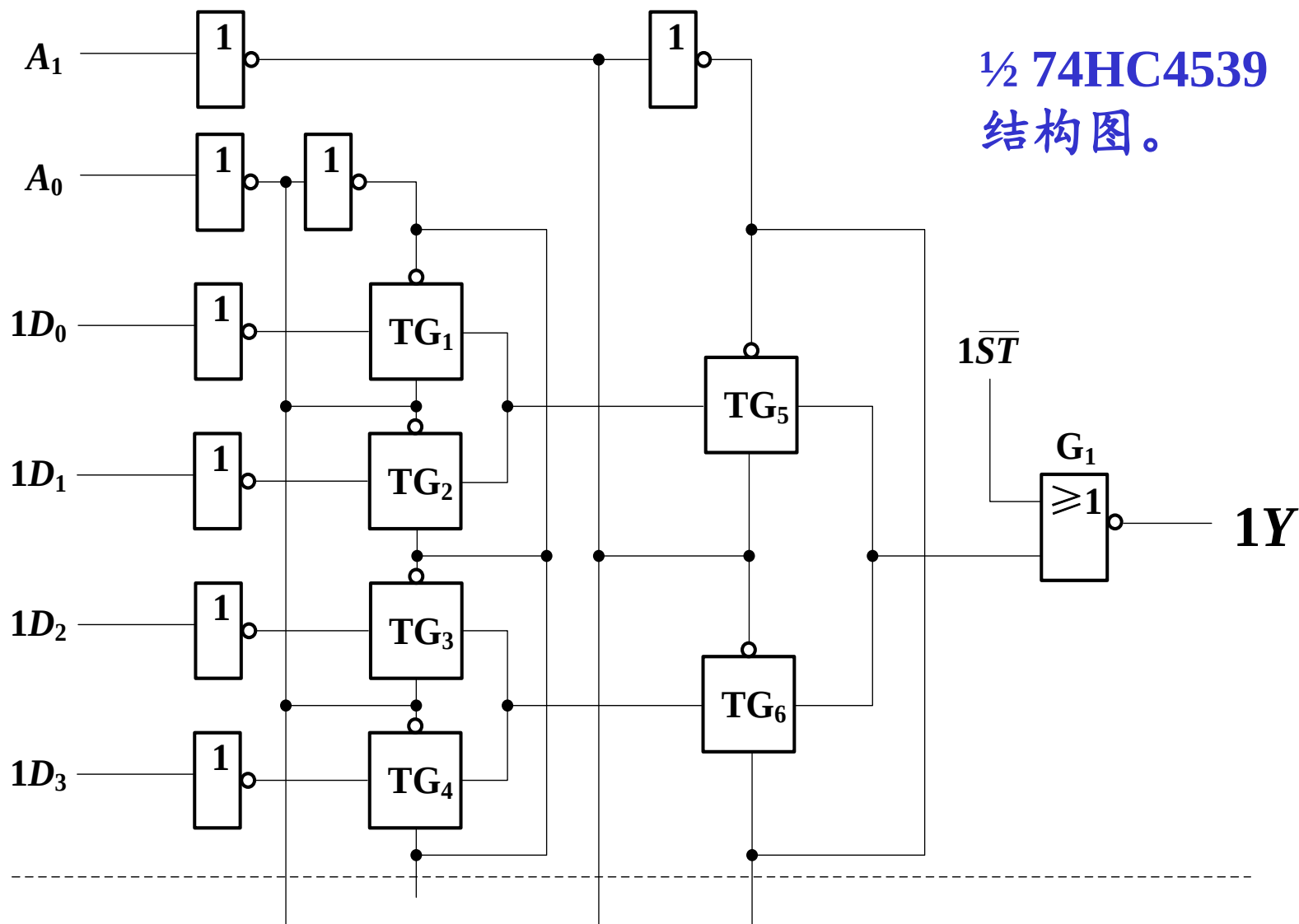
控制作用以“与”关联符号G表示，后面是0、1、2、3的简写。

两个相同的单元框

其中 $\overline{ST}$ 为低电平有效，用EN说明它的使能作用，由于这个EN后面无数字所以对本单元全部输入端0~3均起作用。

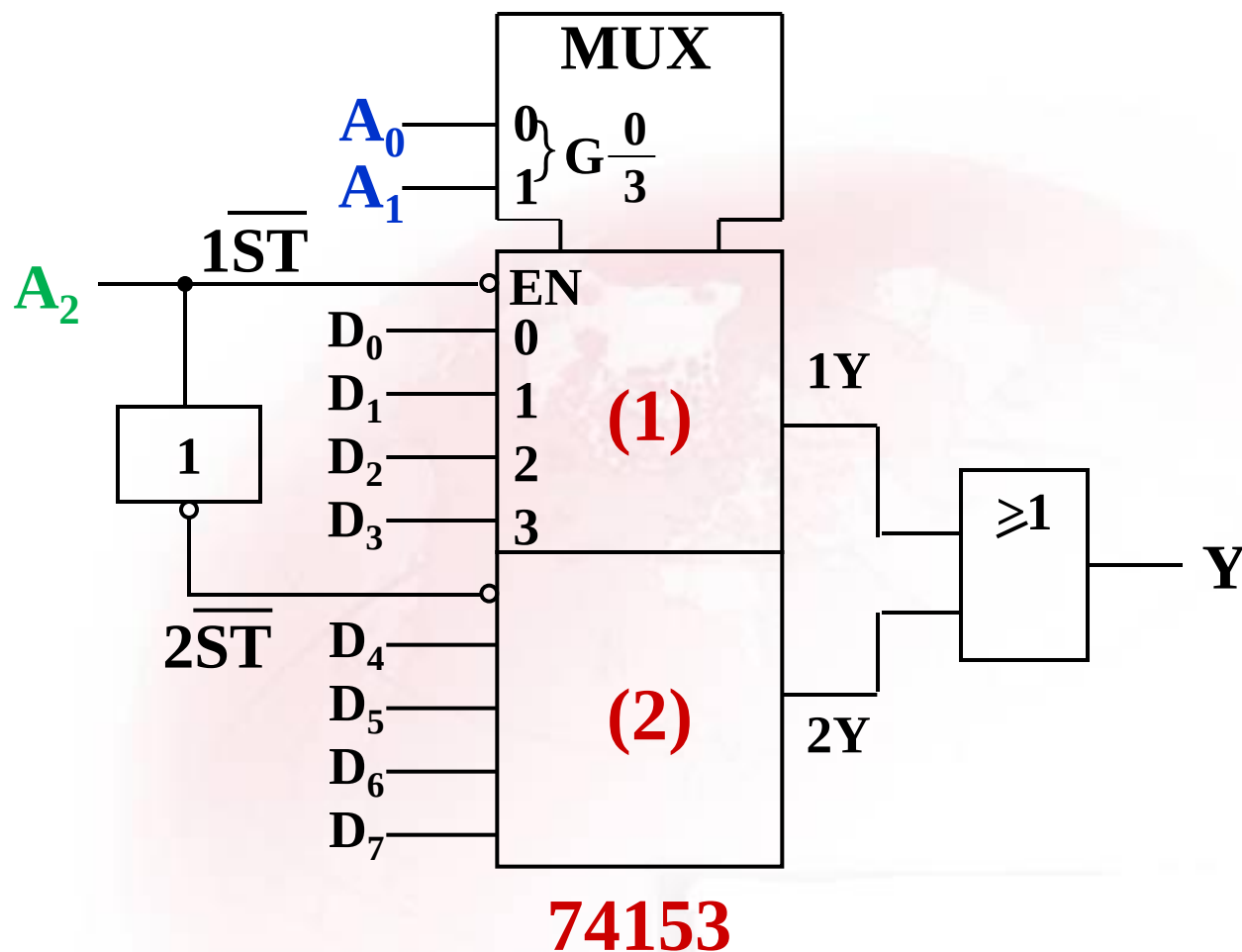
每个单元：
$$Y = (\overline{A_1}\overline{A_0}D_0 + \overline{A_1}A_0D_1 + A_1\overline{A_0}D_2 + A_1A_0D_3) \overline{\overline{ST}}$$

74HC4539的功能和逻辑符号和74153相同，但芯片内部由CMOS传输门组成。





利用选通控制端实现通道扩展的例子：



$A_2=0$ 时：

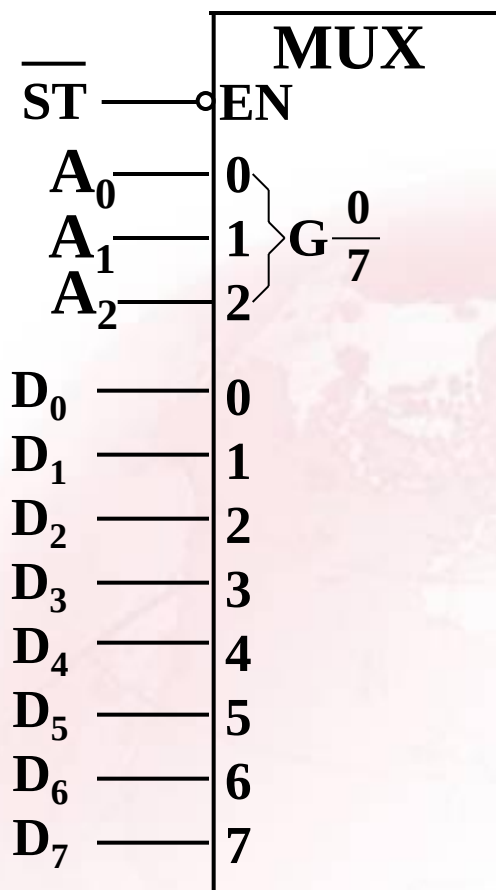
由 A_1A_0 选择 **(1)** D_i

$A_2=1$ 时：

由 A_1A_0 选择 **(2)** D_i



八选一MUX74151



74151

$$Y = \left(\sum_{i=0}^7 m_i D_i \right) \overline{\overline{ST}}$$

数据的反码 $\overline{Y}$ 的输出



4.4.3 数据选择器应用举例

1. 用数据选择器实现组合逻辑函数

基本思想：数据选择器的一般表达式 $Y = \sum_{i=0}^{N-1} m_i D_i$

可知，利用地址变量产生所有最小项，通过数据输入信号 D_i 的不同取值，来选取组成逻辑函数的所需最小项。

例：试用八选一数据选择器74151实现逻辑函数

$$F(A, B, C) = \sum m(0, 2, 3, 5)$$



解：待实现的函数为： $F(A, B, C) = \sum m(0, 2, 3, 5)$
 $= \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}C$

74151的输出表达式为：

$$Y = (\overline{A}_2\overline{A}_1\overline{A}_0D_0 + \overline{A}_2\overline{A}_1A_0D_1 + \overline{A}_2A_1\overline{A}_0D_2 + \overline{A}_2A_1A_0D_3 + A_2\overline{A}_1\overline{A}_0D_4 + A_2\overline{A}_1A_0D_5 + A_2A_1\overline{A}_0D_6 + A_2A_1A_0D_7)\overline{ST}$$

比较两式：

令： $\overline{ST}=0$ $A_2=A$; $A_1=B$; $A_0=C$

只要使： $D_0=D_2=D_3=D_5=1$ $D_1=D_4=D_6=D_7=0$

这样： $Y=F$



$$\overline{ST}=0$$

$$A_2=A ; A_1=B ; A_0=C$$

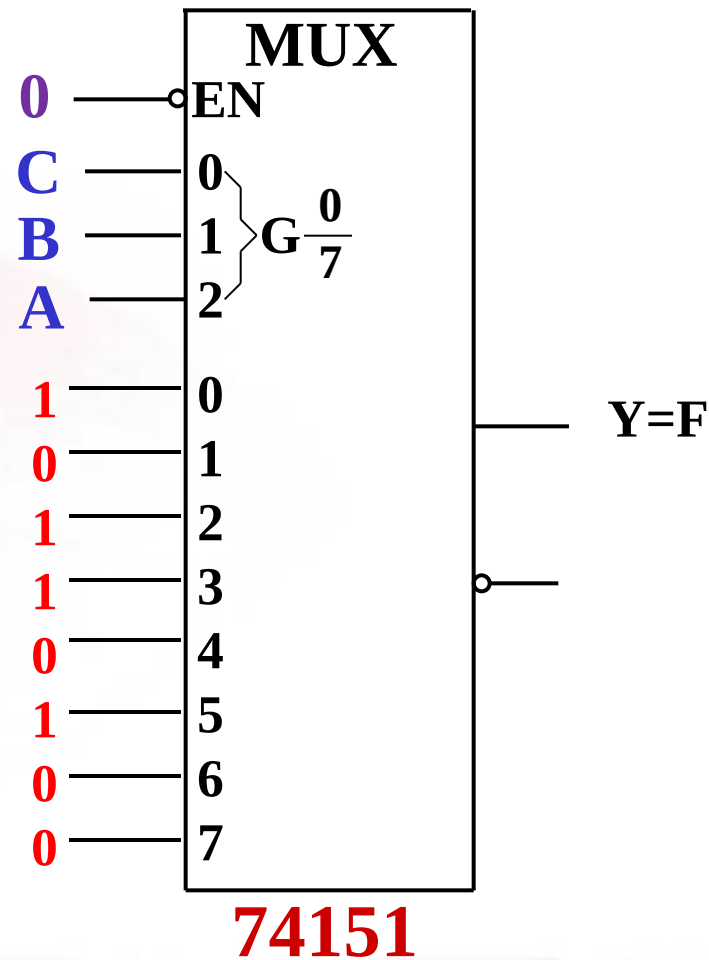
$$D_0=D_2=D_3=D_5=1$$

$$D_1=D_4=D_6=D_7=0$$

$$Y=F$$

注意：

- ① 实现逻辑函数时，MUX 必须被选通，即 $\overline{ST}=0$ 。
- ② 变量和地址端之间的连接必须正确。





例：试用四选一MUX实现逻辑函数

$$F = \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C + A\overline{B}\overline{C} + ABC$$

解：当MUX被选通时，其输出逻辑表达式为：

$$Y = (\overline{A}_1\overline{A}_0)D_0 + (\overline{A}_1A_0)D_1 + (A_1\overline{A}_0)D_2 + (A_1A_0)D_3$$

将函数F写成： $F = \overline{A}\overline{B} \cdot 1 + \overline{A}\overline{B} \cdot 0 + A\overline{B} \cdot \overline{C} + AB \cdot C$

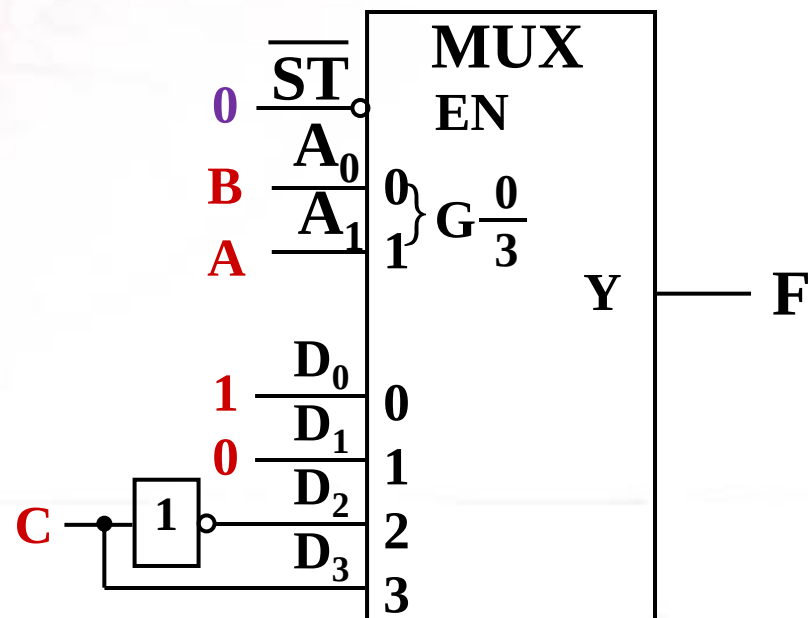
比较两式，令

$$A_1 = A; A_0 = B;$$

$$D_0 = 1, D_1 = 0, D_2 = \overline{C}, D_3 = C$$

则 $Y = F$

注：该题的解法不唯一。





例：用四选一数据选择器实现逻辑函数：

$$F(A,B,C,D)=\Sigma m(1,2,4,9,10,11,12,14,15)$$

解：令数据选择器的地址 $A_1A_0=AB$

AB \ CD				
	00	01	11	10
00		1		1
01	1			
11	1		1	1
10		1	1	1

$$\bar{A}\bar{B}(\bar{C}D+C\bar{D})=\bar{A}_1\bar{A}_0D_0$$

$$\bar{A}B(\bar{C}\bar{D})=\bar{A}_1A_0D_1$$

$$AB(C+\bar{D})=A_1A_0D_3$$

$$A\bar{B}(C+D)=A_1\bar{A}_0D_2$$

$$D_0=\bar{C}D+C\bar{D}=\overline{\bar{C}\bar{D}}\cdot\overline{CD}$$

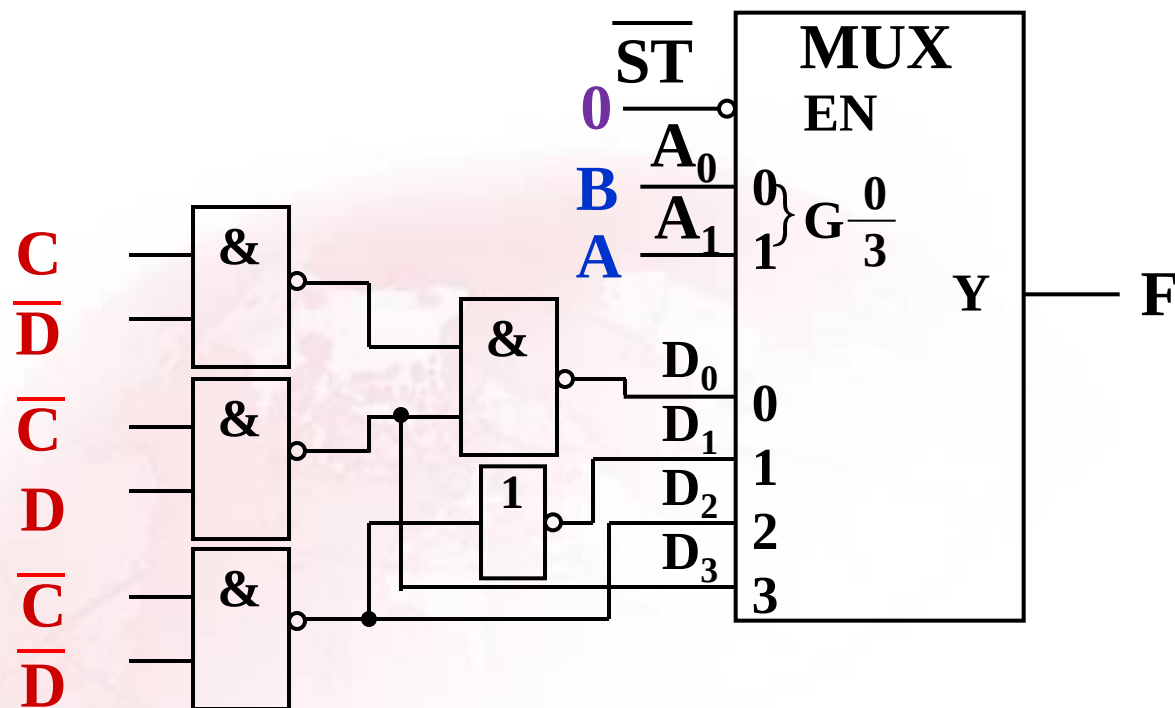
$$D_2=C+D=\overline{\bar{C}\bar{D}}$$

$$D_1=\bar{C}\bar{D}=\overline{CD}$$

$$D_3=C+\bar{D}=\overline{\bar{C}D}$$



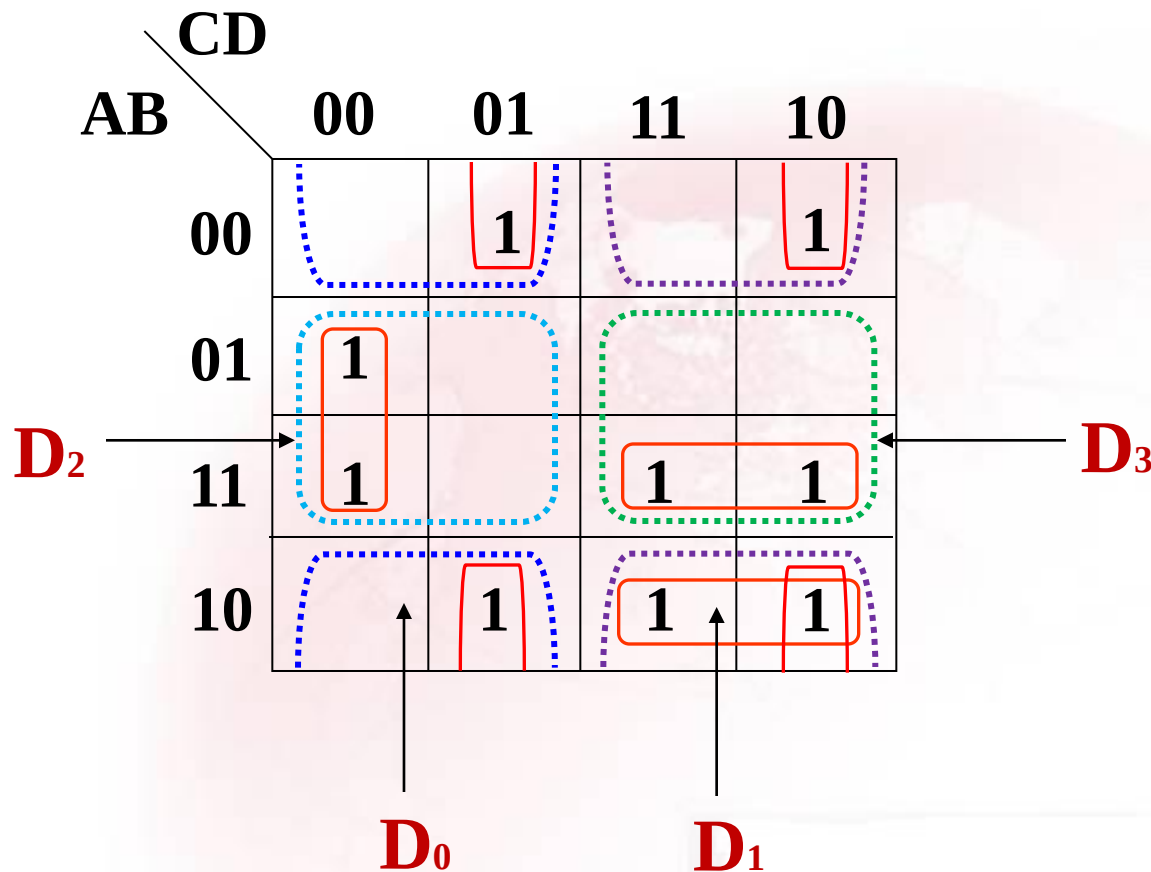
电路图：



注：上面采用A、B作为地址变量。实际上，地址变量的选取是任意的，选不同的变量为地址变量时，数据输入端的信号也要随之变化。



如果令数据选择器的地址 $A_1A_0=BC$





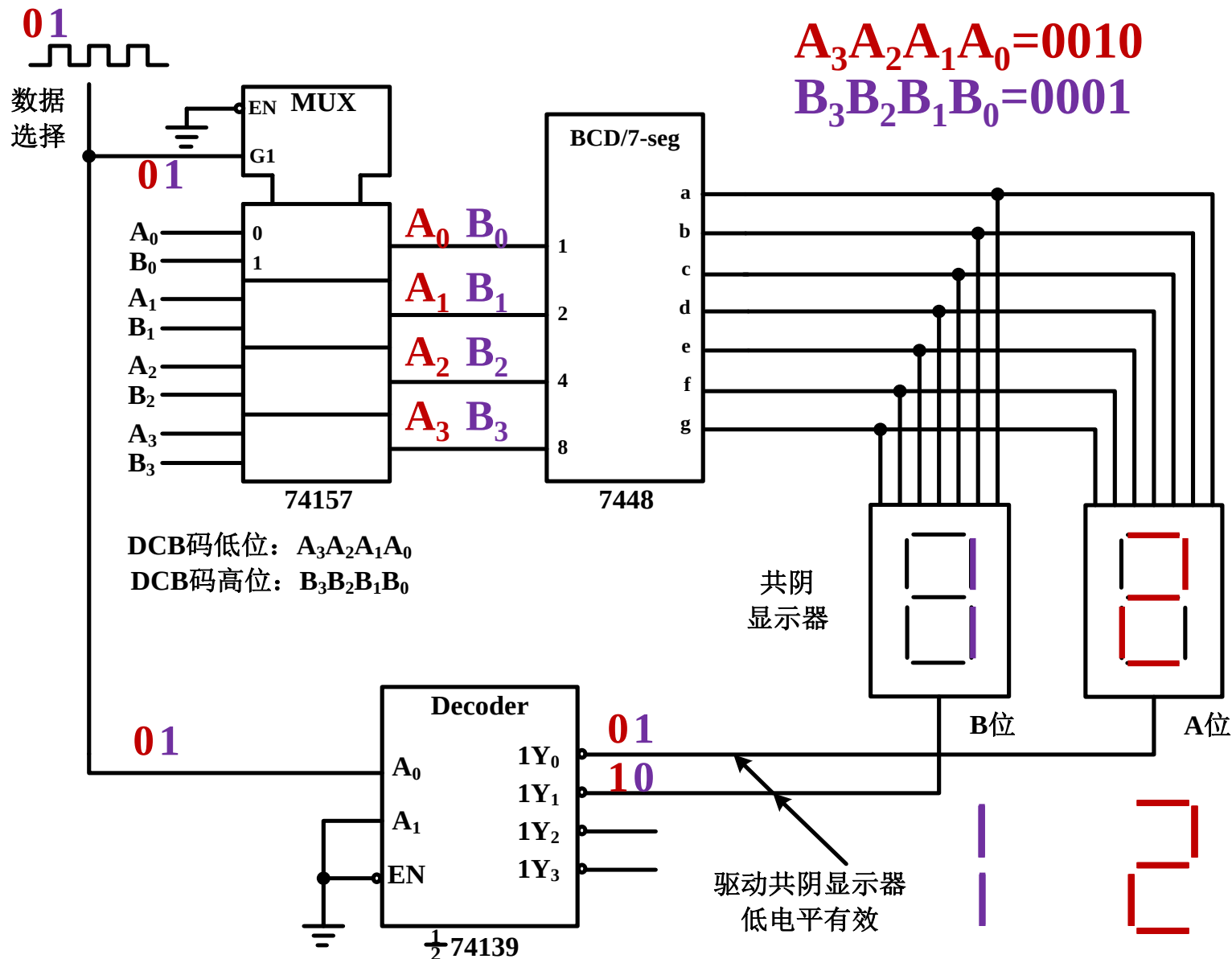
2. 动态显示电路

七段数码管驱动电路可分为两种，一种称为**静态显示**，另一种称为**动态显示**。

静态显示：每一个数码管由单独的七段显示译码器驱动。

动态显示：使用数据选择器的分时复用功能，将任意多个数码管的显示驱动，由一个七段显示译码器来完成。

■ 利用数据选择器实现数码管动态显示





4.4.4 数据选择器的Verilog描述

例：4选1数据选择器的Verilog程序代码。

```
module Vrmux41(A,B,C,D,S1,S0,Y);
```

```
    input A,B,C,D;
```

```
    input S1,S0;
```

```
    output Y;
```

```
    reg Y;
```

```
    always @ (A or B or C or D or S1 or S0) begin  
        case({S1,S0})
```



```
2'b00: Y<=A;  
2'b01: Y<=B;  
2'b10: Y<=C;  
2'b11: Y<=D;  
default: Y<=A;  
endcase  
end  
endmodule
```



例：总线数据选择器的Verilog程序代码

```
module Vrbus_mux41(A,B,C,D,S1,S0,Y);  
  input [3:0] A,B,C,D;  
  input S1,S0;  
  output [3:0] Y;  
  reg [3:0] Y;  
  
  always @ (A or B or C or D or S1 or S0) begin  
    case({S1,S0})
```



```
2'b00: Y<=A;  
2'b01: Y<=B;  
2'b10: Y<=C;  
2'b11: Y<=D;  
default: Y<=A;  
endcase  
end  
endmodule
```



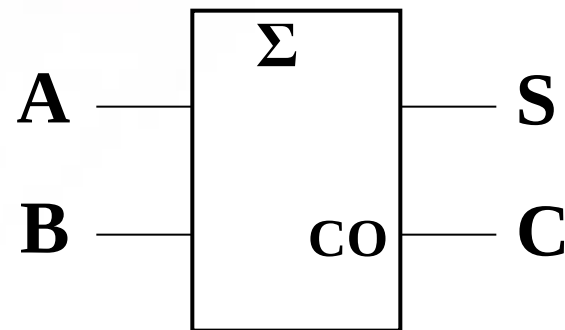
4.5 算术运算电路

算术运算电路的核心为**加法器**。

4.5.1 基本加法器

1. 半加器(**HA**)

仅考虑两个一位二进制数相加,而不考虑低位的进位,称为**半加**。



半加器逻辑符号



设：A、B为两个加数，S 为本位的和，C 为本位向高位的进位。则半加器的真值表、逻辑表达式、逻辑图如下所示。

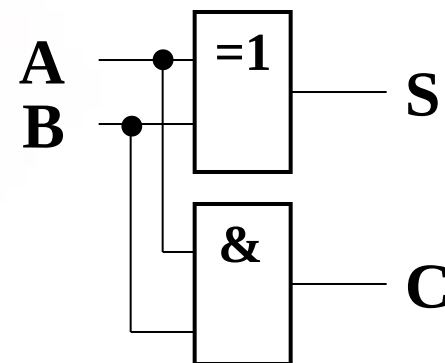
真值表

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

逻辑方程

$$S = \bar{A}B + A\bar{B} = A \oplus B$$

$$C = AB$$



逻辑图



2. 全加器 (FA)

在多位数相加时，除考虑本位的两个加数外，既要考虑低位向本位的进位，又要考虑本位向高位的进位。

例：

	1	1	0	1	被加数
	1	1	1	1	加数
+) 1	1	1	1	1	0 进位
	1	1	1	0	0 和

实际参加一位数相加，必须有三个输入变量，它们是：

本位加数 A_i 、 B_i ； 低位向本位的进位 C_{i-1}

一位全加器的输出结果为：

本位和 S_i ； 本位向高位的进位 C_i



一位全加器电路设计:

(1) 一位全加器真值表

A_i	B_i	C_{i-1}	C_i	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

(2) 输出逻辑表达式

$$S_i(A_i, B_i, C_{i-1}) = \sum m(1, 2, 4, 7)$$

$$\begin{aligned} &= (\bar{A}_i \bar{B}_i + A_i B_i) C_{i-1} \\ &\quad + (A_i \bar{B}_i + \bar{A}_i B_i) \bar{C}_{i-1} \\ &= A_i \oplus B_i \oplus C_{i-1} \end{aligned}$$

而半加器的和为: $S = A_i \oplus B_i$

因此: $S_i = S \oplus C_{i-1}$

$$C_i(A_i, B_i, C_{i-1}) = \sum m(3, 5, 6, 7)$$

$$\begin{aligned} &= (\bar{A}_i B_i + A_i \bar{B}_i) C_{i-1} + A_i B_i \\ &= (A_i \oplus B_i) C_{i-1} + A_i B_i \\ &= S C_{i-1} + A_i B_i \end{aligned}$$

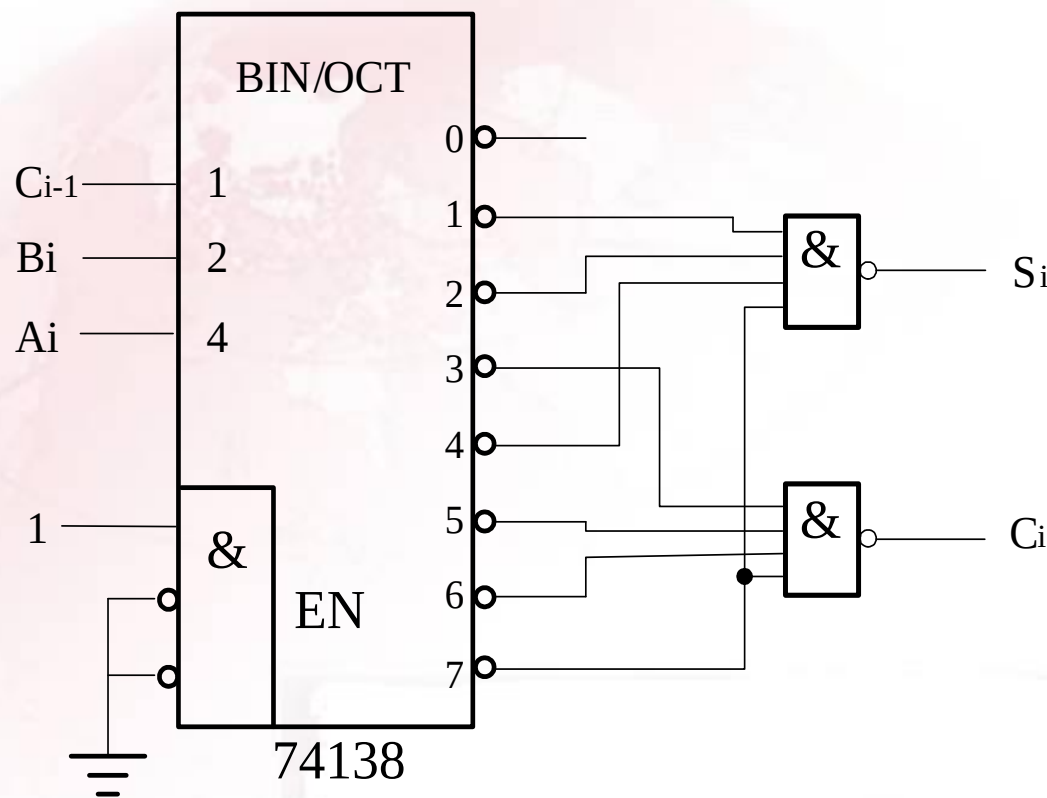


(3) 全加器电路图

■ 用一片3线-8线译码器74138和两个与非门实现一位全加器。

$$S_i(A_i, B_i, C_{i-1}) = \sum m(1, 2, 4, 7)$$

$$C_i(A_i, B_i, C_{i-1}) = \sum m(3, 5, 6, 7)$$

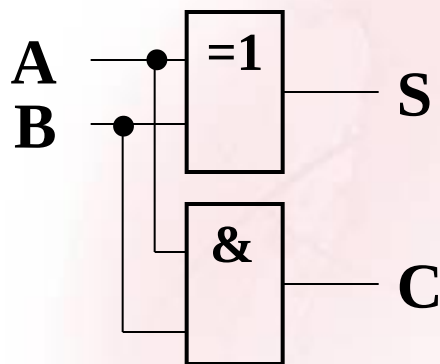




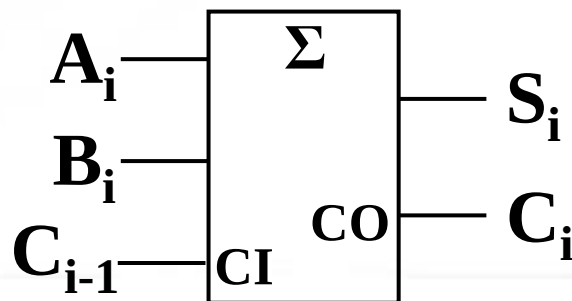
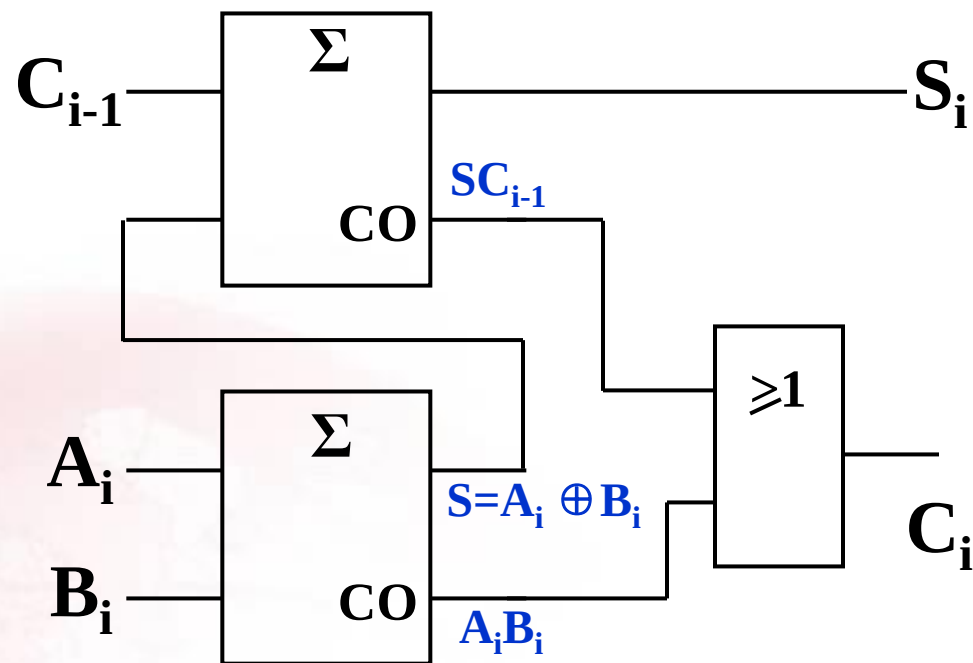
■ 由两个半加器实现一位全加器

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$



半加器逻辑图

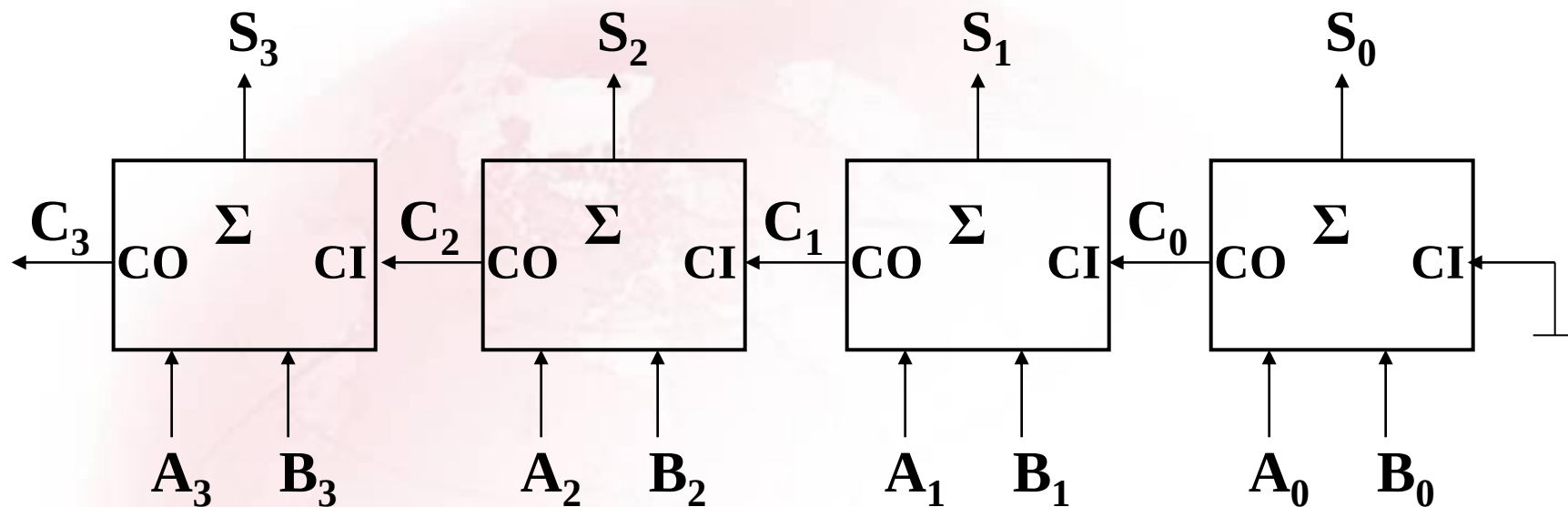


全加器逻辑符号



3. 串行进位加法器

当有多位数相加时，可模仿**笔算**，用**全加器**构成串行进位加法器。



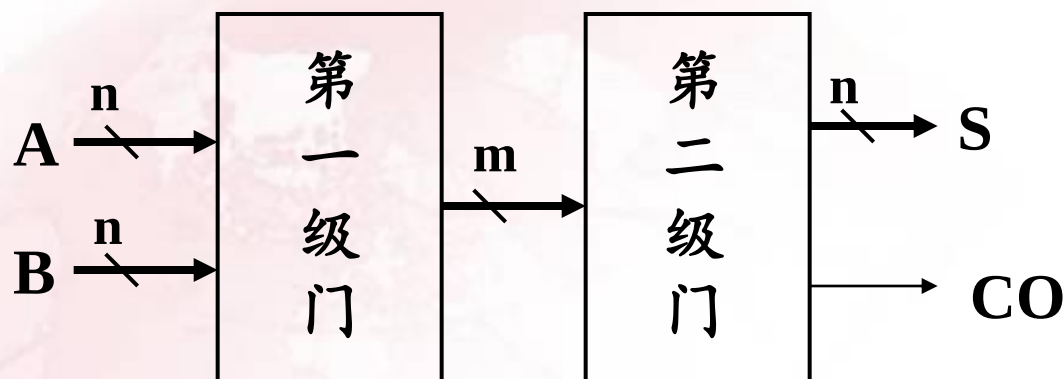
四位串行进位加法器

串行进位加法器特点：结构简单；运算速度慢。



4.5.2 高速加法器

1. 全并行加法器



特点：速度最快，电路复杂。



2. 超前进位加法器

设计思想：通过逻辑电路提前得出加到每一位全加器上的进位输入信号，而无需从最低位开始逐位传递进位信号。

全加器的进位表达式：

$$\begin{aligned}C_i &= (\bar{A}_i B_i + A_i \bar{B}_i) C_{i-1} + A_i B_i \\&= \bar{A}_i B_i C_{i-1} + A_i \bar{B}_i C_{i-1} + A_i B_i \bar{C}_{i-1} + A_i B_i C_{i-1} \\&= \mathbf{A_i B_i} + \mathbf{(A_i + B_i) C_{i-1}}\end{aligned}$$

令： $\mathbf{G_i = A_i B_i}$ ---进位产生项 $\mathbf{P_i = (A_i + B_i)}$ ---进位传送项

则： C_i 的一般表达式为 $C_i = \mathbf{G_i} + \mathbf{P_i C_{i-1}}$



若两个三位二进制数相加

$$A=A_2A_1A_0 \quad B=B_2B_1B_0$$

因为： C_i 的一般表达式为 $C_i = G_i + P_i C_{i-1}$

则有： $C_0 = G_0$; $C_1 = G_1 + P_1 C_0 = G_1 + P_1 G_0$;

$$C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 G_0$$

由 P_i 、 G_i 并经过两级门电路就可求得进位信号。

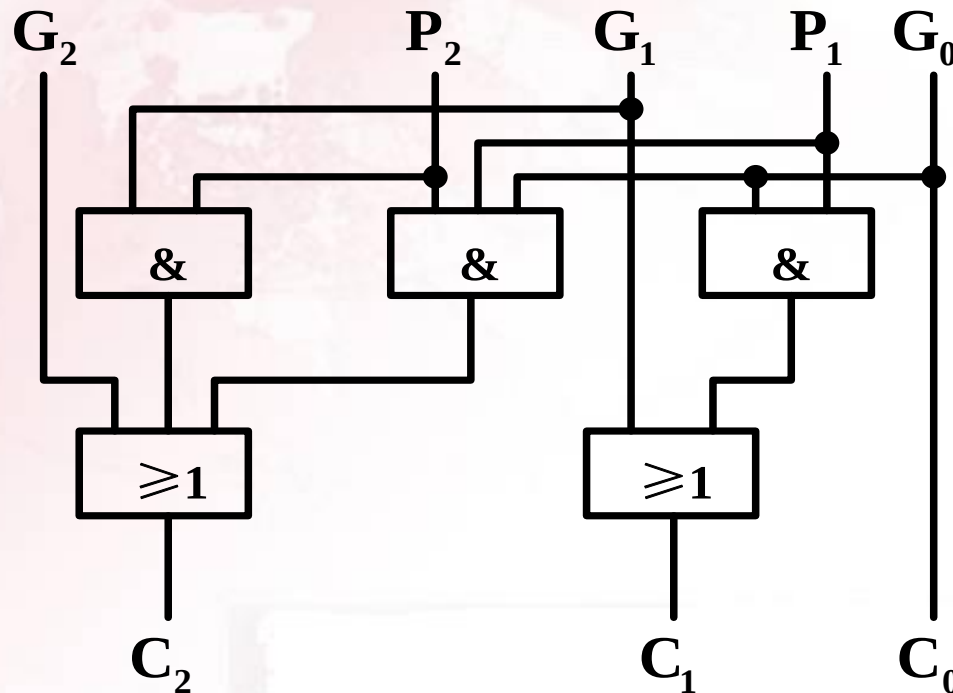
根据 G_i 、 P_i 来求进位信号的电路称为超前进位电路 (CLA)。



CLA逻辑图:

$$C_0 = G_0 ; \quad C_1 = G_1 + P_1 C_0 = G_1 + P_1 G_0 ;$$

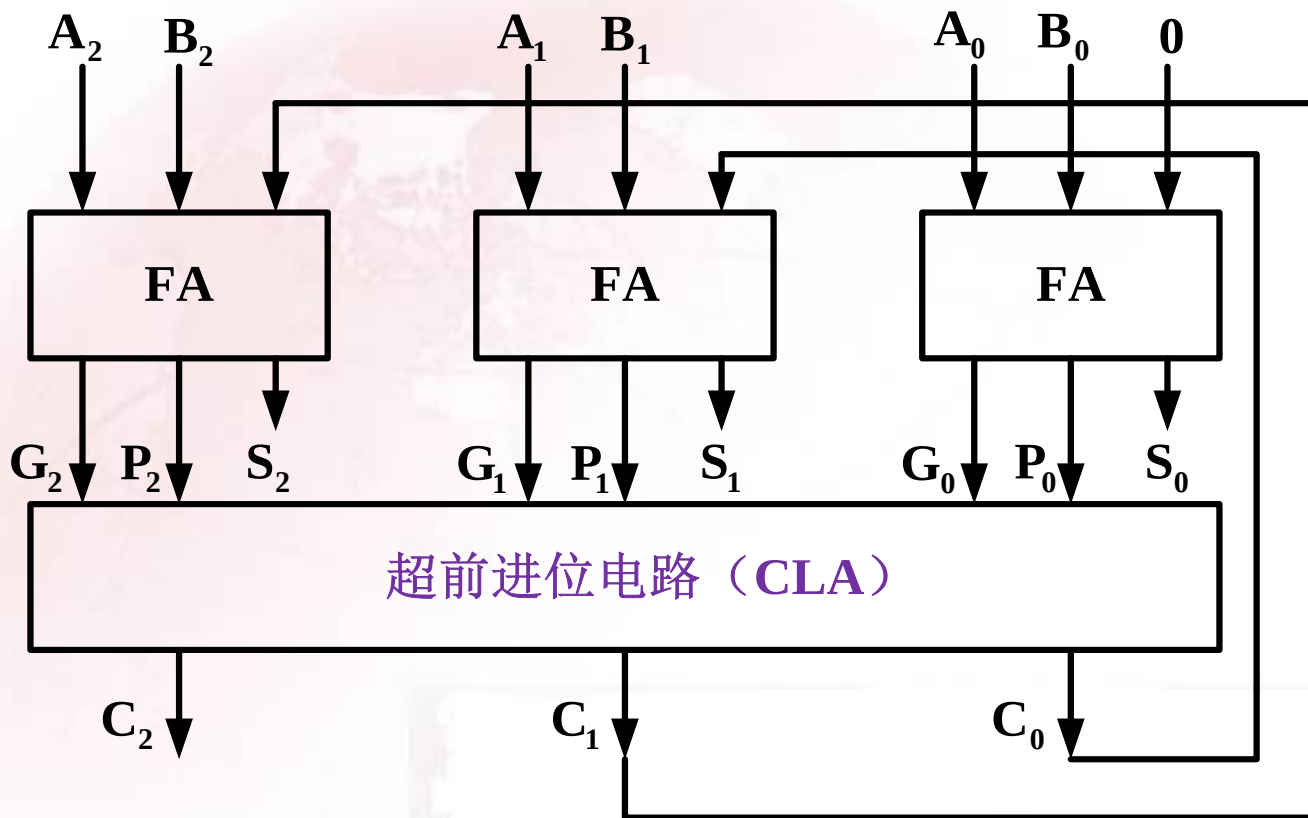
$$C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 G_0$$





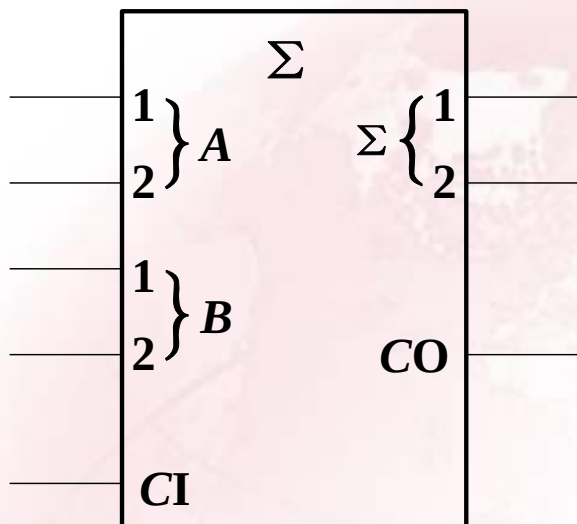
实际实现中，是将求 G_i 和 P_i 的电路CLA放进全加器中，原全加器内的进位产生电路因不再需要而被删除。

3位超前进位加法器

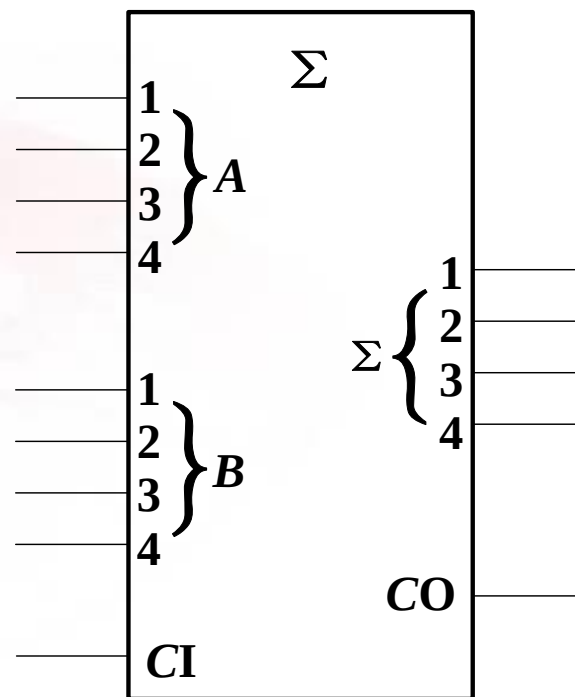




4.5.3 通用加法器集成电路



2位并行加法器7482



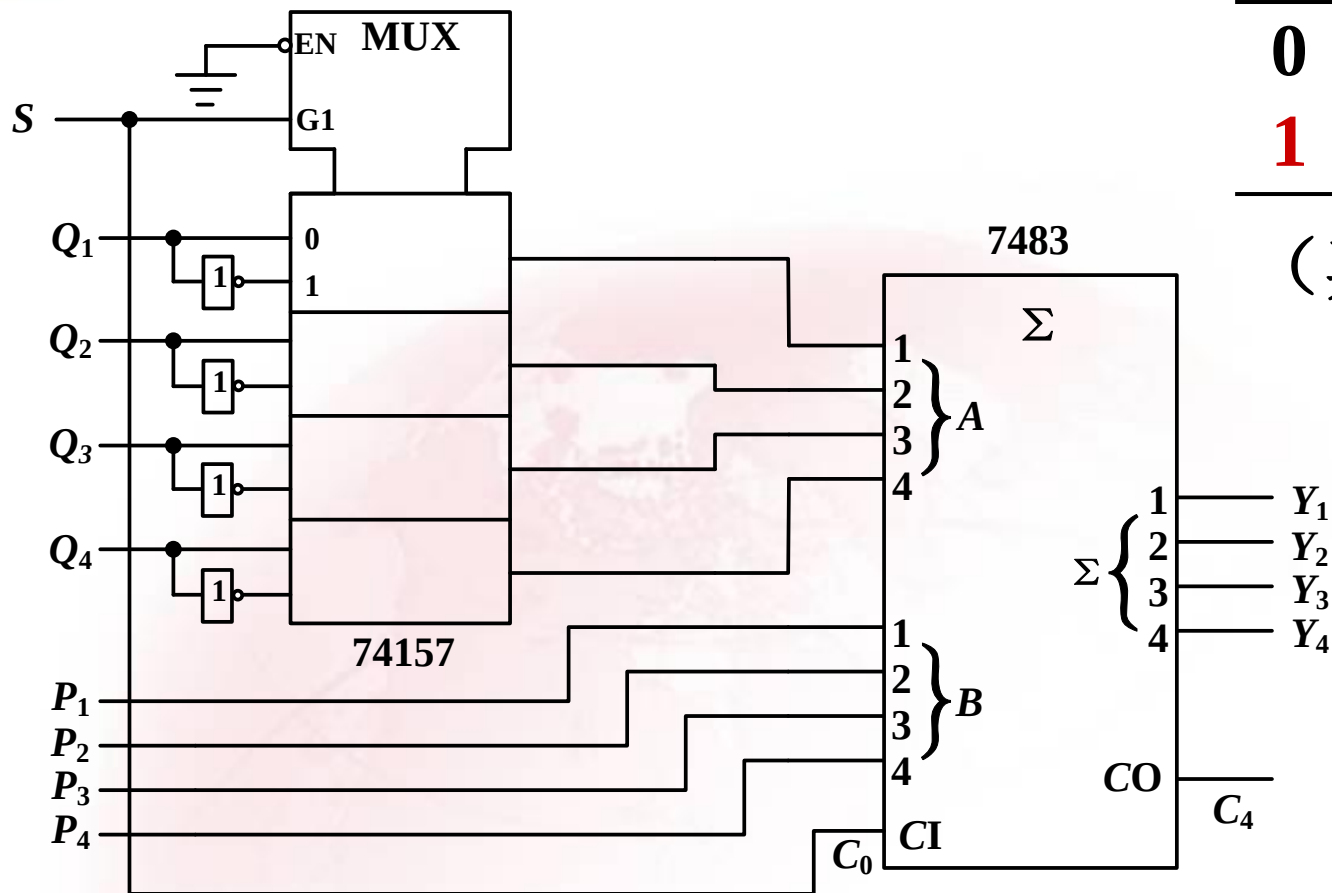
4位并行加法器7483



4.5.4 加法器应用举例

1. 用 4×2 选1数据选择器74157和4位全加器7483，构成4位二进制加/减器。

在二进制补码系统中，减法功能由加“减数”的补码实现。



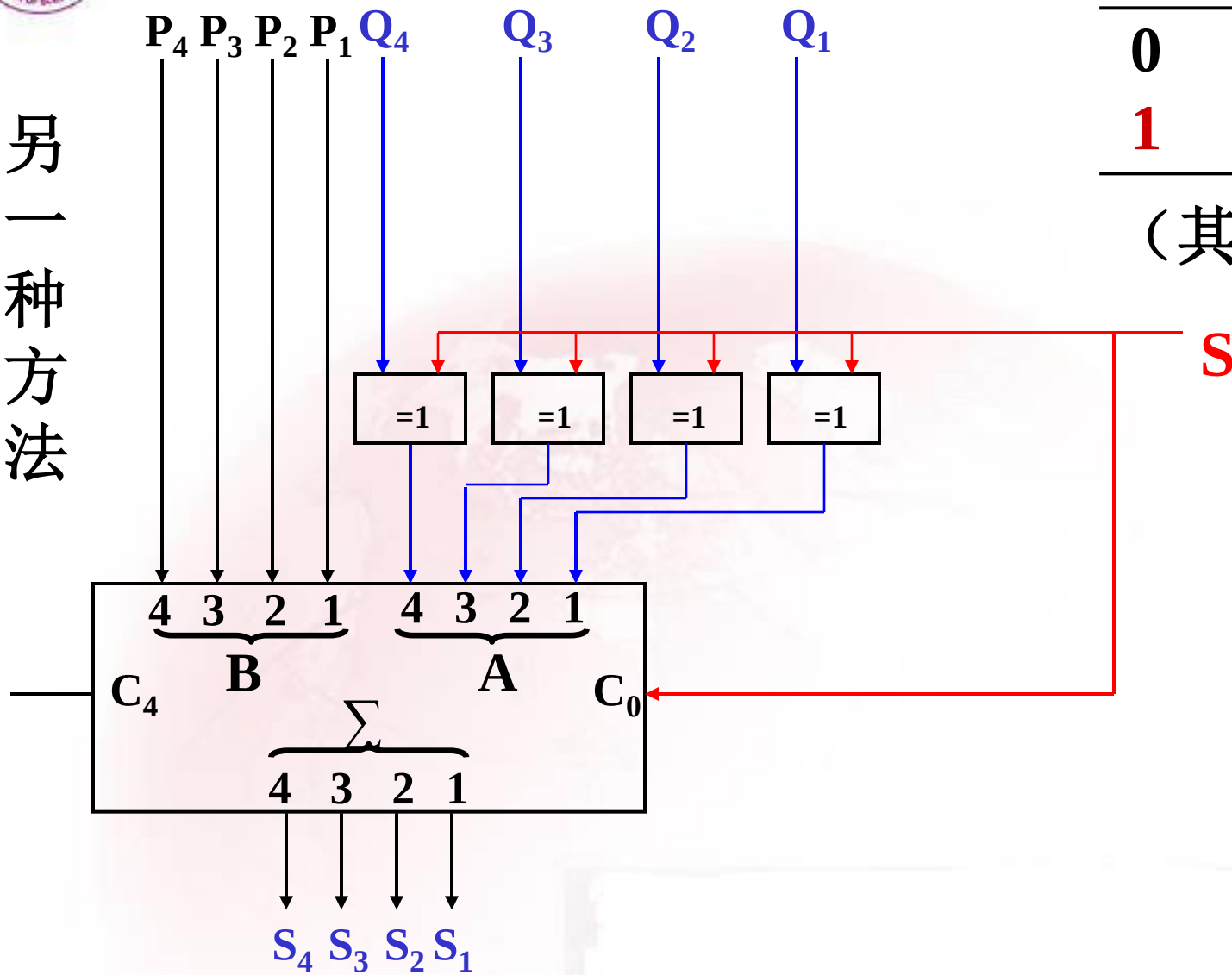
S	功能
0	$(P)_2 + (Q)_2$
1	$(P)_2 - (Q)_2$

(其中 $P \geq Q$)

二进制加/减器



另一种方法



S	功能
0	$(P)_2 + (Q)_2$
1	$(P)_2 - (Q)_2$

(其中 $P \geq Q$)



※关于减法电路探讨

(1)式的实现方法: (以4位数相减为例)

▲ 二进制减法运算

$$N_{\text{补}} = 2^n - N_{\text{原}} \quad (N_{\text{原}} \text{ 为 } n \text{ 位})$$

$$N_{\text{原}} = 2^n - N_{\text{补}}$$

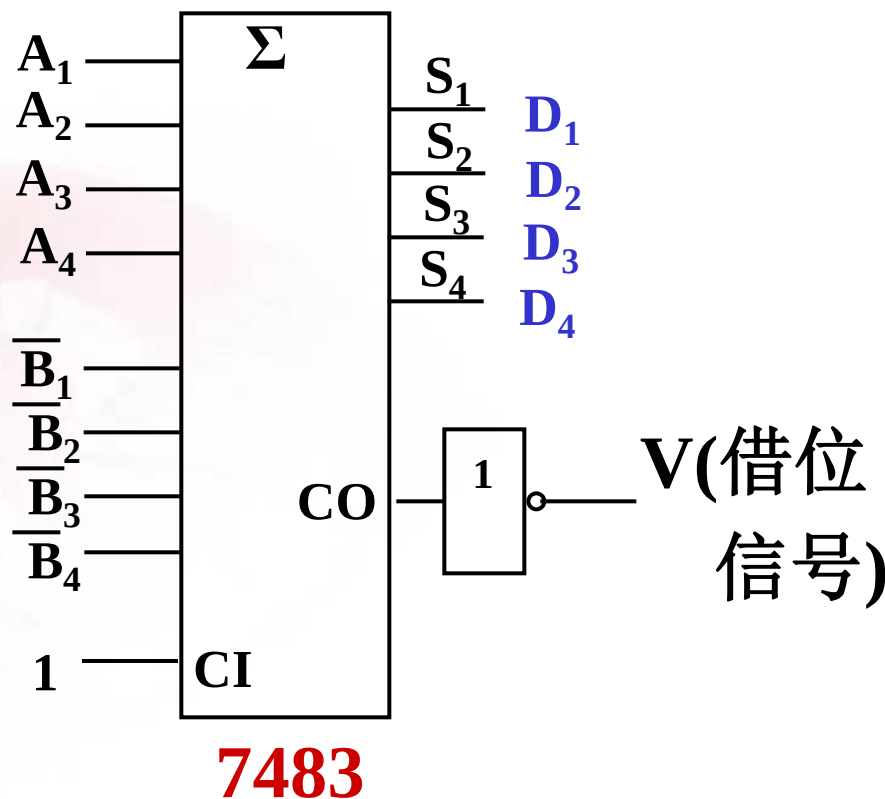
$$N_{\text{补}} = N_{\text{反}} + 1$$

$$A - B = A - B_{\text{原}}$$

$$= A - (2^n - B_{\text{补}})$$

$$= A + B_{\text{反}} + 1 - 2^n \quad (1)$$

借位信号实现减 2^n 的功能: 当 $A + B_{\text{反}} + 1$ 的高位有进位时, 该进位信号和 2^n 相减使最高位为0, 反之为1。





▲ 分两种情况讨论:

(1) $A - B \geq 0$

设 $A=0101$, $B=0001$

求补码相加演算过程如下:

$$\begin{array}{r} 0101 \text{ (A)} \\ + 1110 \text{ (B}_{\text{反}}) \\ \hline 1 \text{ (加1)} \\ \hline 1 \ 0100 \\ \downarrow \\ 0 \ 0100 \text{ (进位反相)} \\ \uparrow \text{借位} \end{array}$$

运算结果为4和实际相同。

(2) $A - B < 0$

设 $A=0001$, $B=0101$

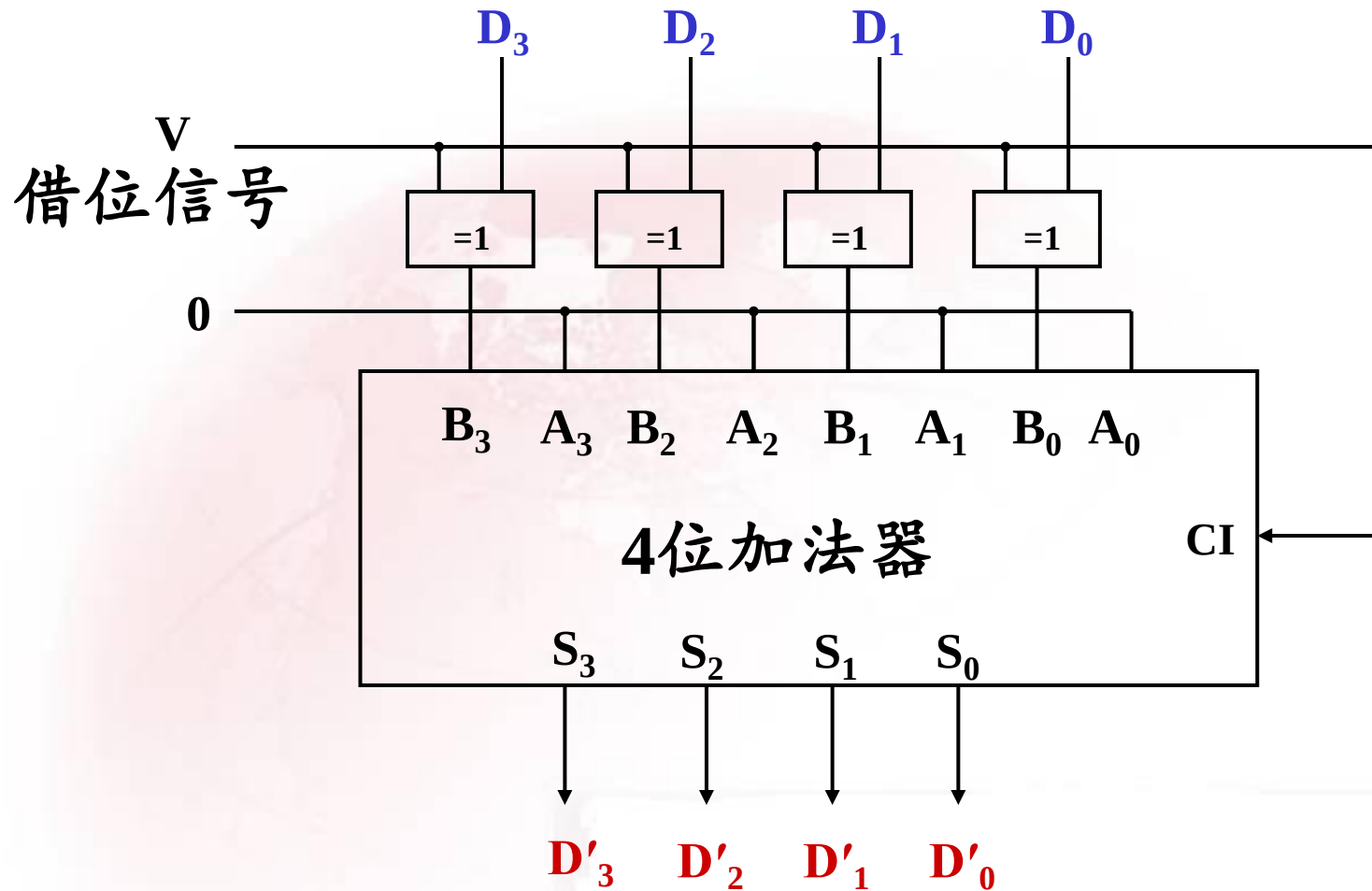
求补码相加演算过程如下:

$$\begin{array}{r} 0001 \text{ (A)} \\ + 1010 \text{ (B}_{\text{反}}) \\ \hline 1 \text{ (加1)} \\ \hline 0 \ 1100 \\ \downarrow \\ 1 \ 1100 \text{ (进位反相)} \\ \uparrow \text{借位} \end{array}$$

运算结果为-4的补码，最高位的1为符号位。

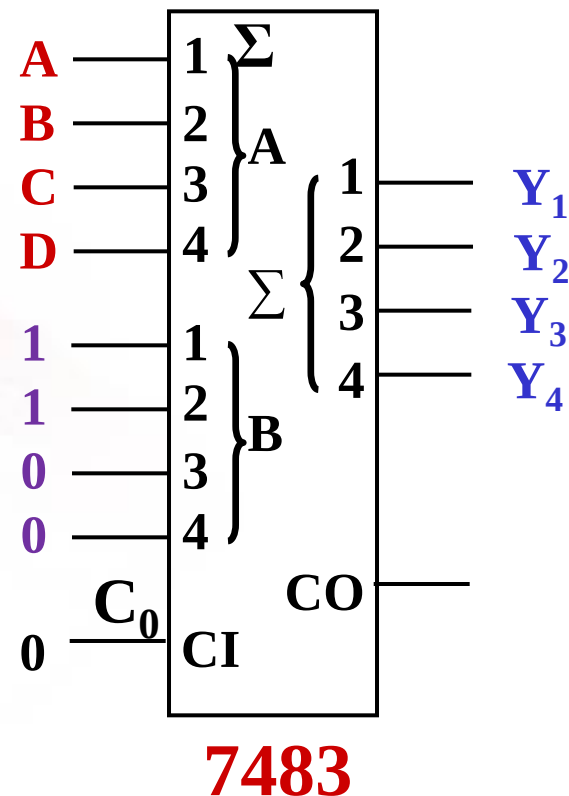


▲ 由符号决定求补的逻辑图





2. 将8421BCD码转换为余3BCD码的代码转换电路。



问题：如何将余3BCD码转换为8421BCD码？



思考1: 试将图中4位二进制加法器7483连接成一个能将**余3BCD码**转换成**2421BCD码**的代码转换电路，可加少量门。

余3码 $A_4A_3A_2A_1$	2421码 $Y_4Y_3Y_2Y_1$
0 0 1 1	0 0 0 0
0 1 0 0	0 0 0 1
0 1 0 1	0 0 1 0
0 1 1 0	0 0 1 1
0 1 1 1	0 1 0 0
1 0 0 0	1 0 1 1
1 0 0 1	1 1 0 0
1 0 1 0	1 1 0 1
1 0 1 1	1 1 1 0
1 1 0 0	1 1 1 1

$$Y_4Y_3Y_2Y_1 = A_4A_3A_2A_1 + 1100 + 1$$

$$Y_4Y_3Y_2Y_1 = A_4A_3A_2A_1 + \bar{F}\bar{F}\bar{F}\bar{F} + \bar{F}$$

其中： $F = A_4$

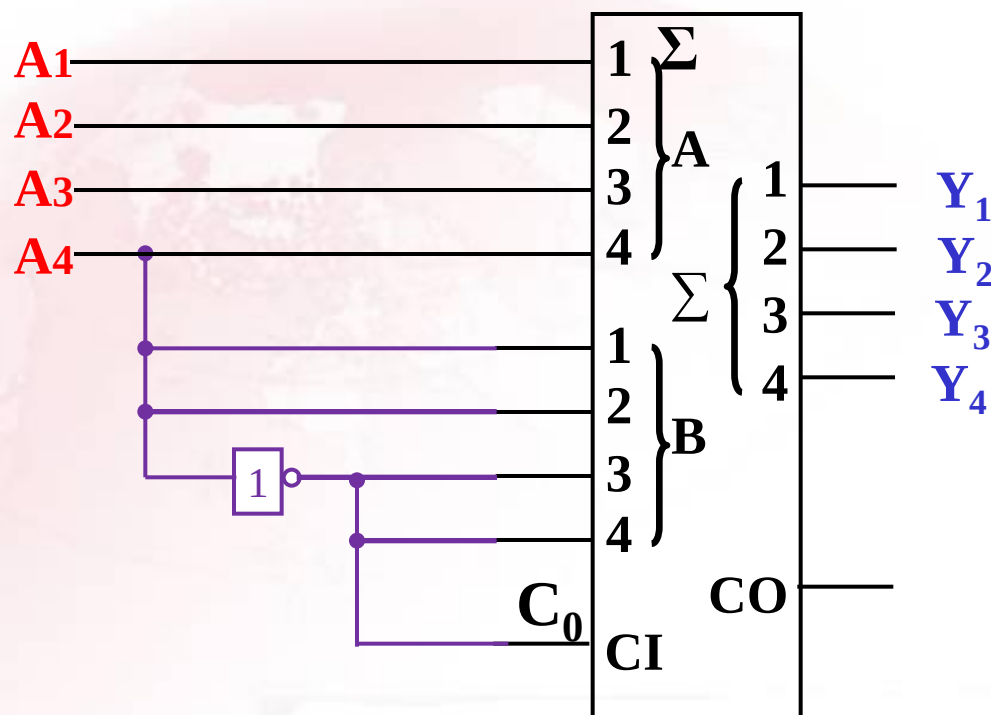
$$Y_4Y_3Y_2Y_1 = A_4A_3A_2A_1 + 0011 + 0$$



$$Y_4 Y_3 Y_2 Y_1 = A_4 A_3 A_2 A_1 + \bar{A}_4 \bar{A}_4 A_4 A_4 + \bar{A}_4$$

电路图：

余3BCD码



7483

2421
BCD码



思考2: 试用4位全加器7483设计一个能将**8421BCD**码转换为**5421BCD**码的代码转换电路。

	8421码				5421码			
	D	C	B	A	Y_4	Y_3	Y_2	Y_1
①	0	0	0	0	0	0	0	0
	0	0	0	1	0	0	0	1
	0	0	1	0	0	0	1	0
	0	0	1	1	0	0	1	1
	0	1	0	0	0	1	0	0
②	0	1	0	1	1	0	0	0
	0	1	1	0	1	0	0	1
	0	1	1	1	1	0	1	0
	1	0	0	0	1	0	1	1
	1	0	0	1	1	1	0	0

解: ① $Y_4Y_3Y_2Y_1 = DCBA + 0000 + 0$

② $Y_4Y_3Y_2Y_1 = DCBA + 0011 + 0$

则有: $Y_4Y_3Y_2Y_1 = DCBA + 00FF + 0$

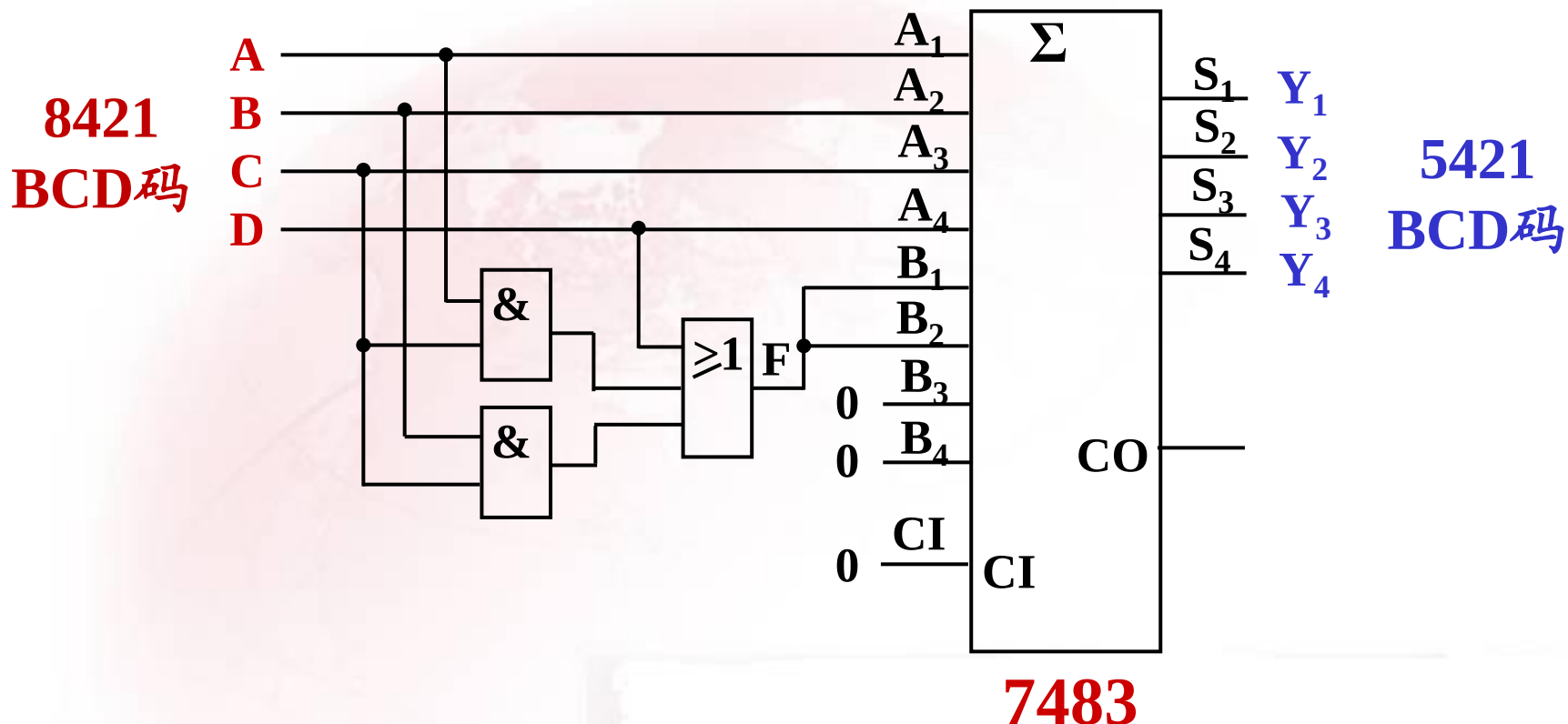
DC \ BA				
	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	×	×	×	×
10	1	1	×	×

$$F = D + CA + CB$$



$Y_4Y_3Y_2Y_1 = DCBA + 00FF + 0$, 其中 $F = D + CA + CB$

电路图:



3. 利用7483(四位二进制加法器)构成8421BCD码加法器。

二进制数和8421BCD码对照表

十进制数	二进制数(和)					8421BCD码(和)				
	C ₄	S ₄	S ₃	S ₂	S ₁	K ₄	Y ₄	Y ₃	Y ₂	Y ₁
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1

$K_4 = C_4 = 0$
 $Y_4 Y_3 Y_2 Y_1 =$
 $S_4 S_3 S_2 S_1 + 0000$

$K_4 = \overline{C_4} = 1$
 $Y_4 Y_3 Y_2 Y_1 =$
 $S_4 S_3 S_2 S_1 + 0110$
有溢出



十进制数	二进制数(和)					8421BCD码(和)				
	C_4	S_4	S_3	S_2	S_1	K_4	Y_4	Y_3	Y_2	Y_1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

$$K_4 = C_4 = 1$$

$$Y_4 Y_3 Y_2 Y_1 =$$

$$S_4 S_3 S_2 S_1 + 0110$$

无溢出

总结上表, 可得: $Y_4 Y_3 Y_2 Y_1 = S_4 S_3 S_2 S_1 + 0FF0$

对于F: ① $C_4 = 1$ 时, $F = C_4$

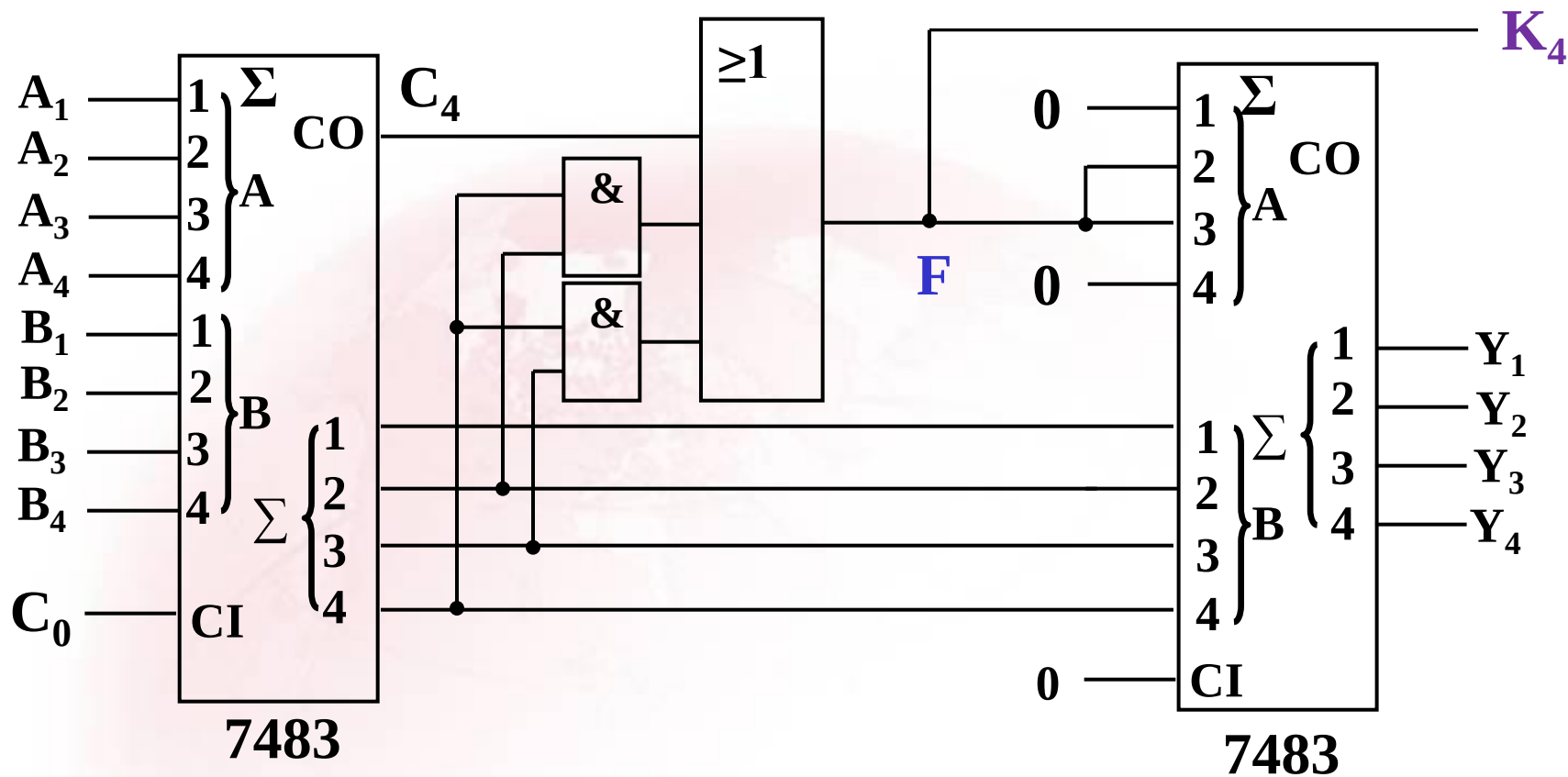
② $C_4 = 0$ 时, $F = S_4 S_3 + S_4 S_2$

所以: $F = C_4 + \bar{C}_4 (S_4 S_3 + S_4 S_2) = C_4 + S_4 S_3 + S_4 S_2$

由于 K_4 与F取值一样, 因此得: $K_4 = C_4 + S_4 S_3 + S_4 S_2$



电路图:

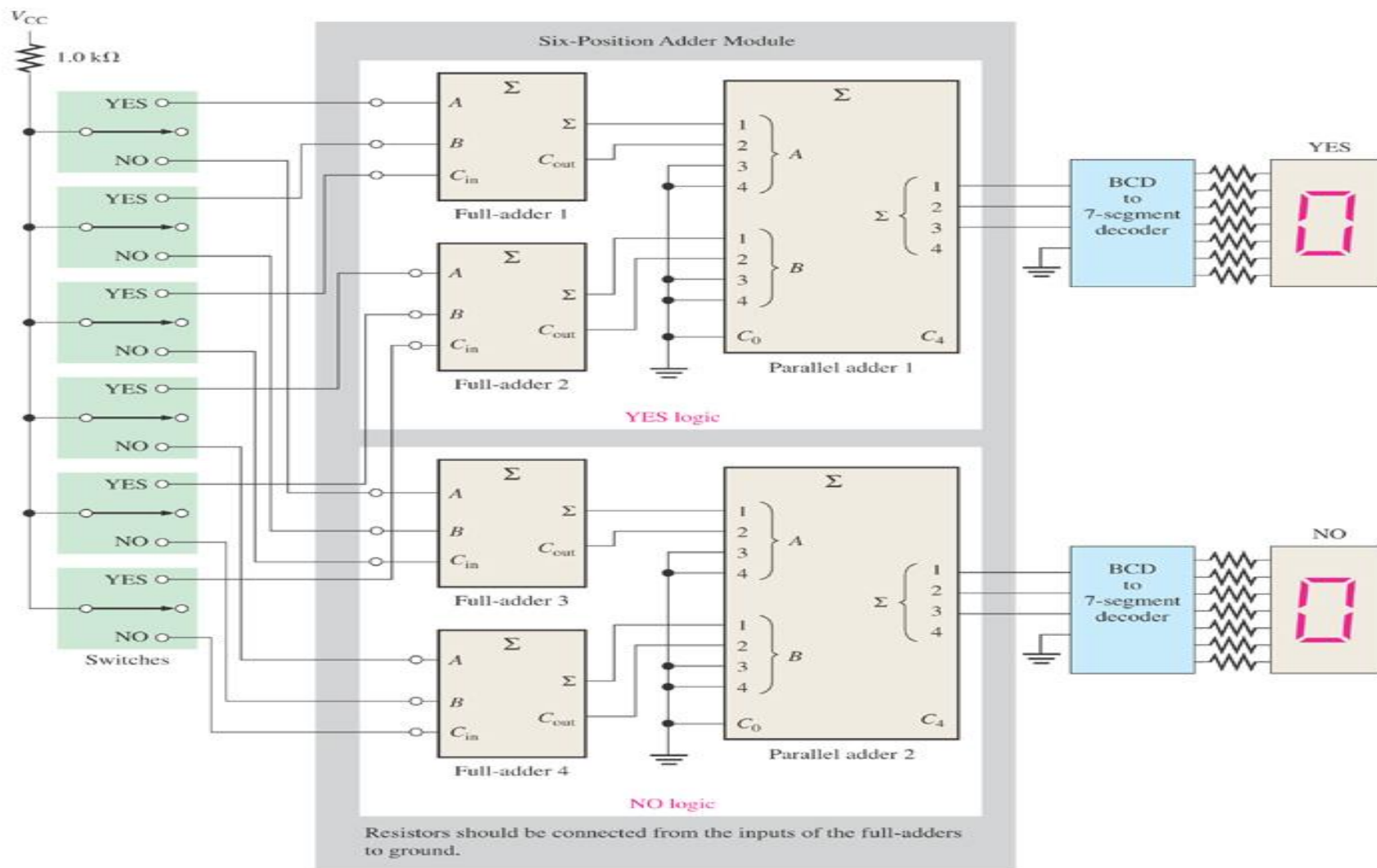


(按二进制加法运算)

(转换为8421BCD码的结果)



4. An Application Example--- Voting System





全减器

在多位数相减时，除考虑本位的两个减数外，既要考虑低位向本位的借位，又要考虑本位向高位的借位。

实际参加一位数相减，必须有三个输入变量，它们是：

本位减数 A_i 、 B_i ；

低位向本位的借位 J_{i-1}

一位全减器的输出结果为：

本位差 D_i ；

本位向高位的借位 J_i

一位全减器真值表

A_i	B_i	J_{i-1}	J_i	D_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



4.5.5 加法器电路的Verilog描述

例：8位无符号数加法器的Verilog程序代码。

```
module Vradder8(A,B,S,COUT);  
  input [7:0] A,B;  
  output [7:0] S;  
  output COUT;  
  
  assign {COUT,S}=A+B;  
endmodule
```



例：8位带符号数加法器的Verilog程序代码。

```
module Vradders8(A,B,S,OVFL);  
    input [7:0] A,B;  
    output [7:0] S;  
    output OVFL;  
  
    assign S=A+B;  
    assign OVFL=(A[7]==B[7]) && (S[7]!=A[7]);  
endmodule
```



例：一位8421BCD码加法器的Verilog程序代码。

```
module Vrbcdadder(A,B,CIN,S,COUT);  
  input [3:0] A,B;  
  input CIN;  
  output [3:0] S;  
  output COUT;  
  
  reg [3:0] S;  
  reg COUT;  
  reg [4:0] SBIN;
```



```
always @ (A,B,CIN)
begin SBIN<=A+B+CIN;
  if(SBIN>5'b01001)
  begin
    S<=SBIN[3:0]+4'b0110;
    COUT<=1'b1;
  end
else
  begin S<=SBIN[3:0];
    COUT<=1'b0;
  end
end
endmodule
```




4.6 数值比较器

数值比较器用来判断两个二进制数的**大小**或**相等**。

4.6.1 一位数值比较器

真值表

A	B	$Y_{(A>B)}$	$Y_{(A<B)}$	$Y_{(A=B)}$
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

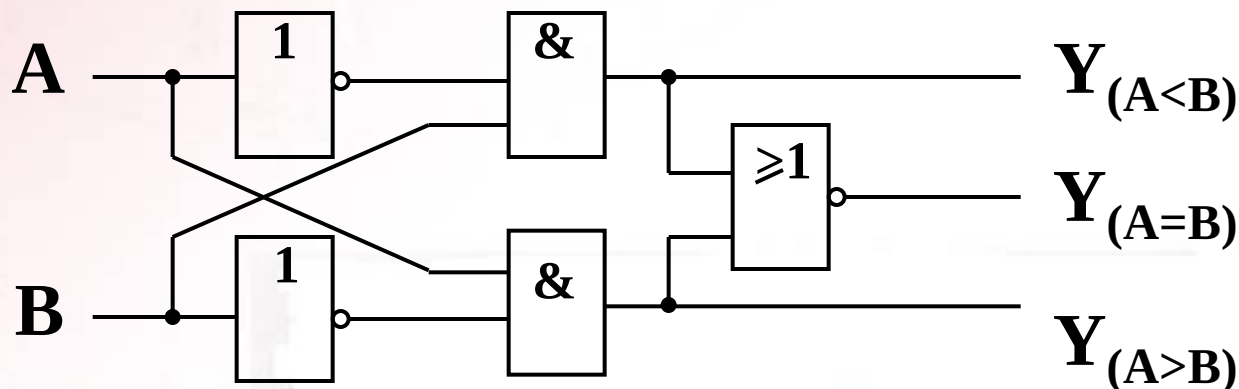
表达式

$$Y_{(A>B)} = A\bar{B}$$

$$Y_{(A<B)} = \bar{A}B$$

$$Y_{(A=B)} = A \odot B$$

逻辑图





4.6.2 多位数值比较器

比较两个多位数，应首先从高位开始，逐位比较。

例如： $A=A_3A_2A_1A_0$ $B=B_3B_2B_1B_0$

比较方法为：

- ① 首先比较 A_3 和 B_3 ，如 $A_3B_3=10$ ，则 $A>B$ ，如 $A_3B_3=01$ ，则 $A<B$ ；如 $A_3B_3=00$ 或 11 (相等)，则比较 A_2 和 B_2 ；
- ② 比较 A_2 和 B_2 ，如 $A_2B_2=10$ ，则 $A>B$ ，如 $A_2B_2=01$ ，则 $A<B$ ；如 $A_2B_2=00$ 或 11 (相等)，则比较 A_1 和 B_1 ；
- ③ 比较 A_1 和 B_1 ，如 $A_1B_1=10$ ，则 $A>B$ ，如 $A_1B_1=01$ ，则 $A<B$ ；如 $A_1B_1=00$ 或 11 (相等)，则比较 A_0 和 B_0 ；
- ④ 比较 A_0 和 B_0 ，如 $A_0B_0=10$ ，则 $A>B$ ，如 $A_0B_0=01$ ，则 $A<B$ ；如 $A_0B_0=00$ 或 11 (相等)，则比较 $A=B$ 。



四位数值比较器逻辑表达式:

$$Y_{(A>B)} = \overline{A_3}B_3 + (A_3 \odot B_3) \overline{A_2}B_2 + (A_3 \odot B_3) (A_2 \odot B_2) \overline{A_1}B_1 + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) \overline{A_0}B_0$$

$$Y_{(A<B)} = \overline{A_3}B_3 + (A_3 \odot B_3) \overline{A_2}B_2 + (A_3 \odot B_3) (A_2 \odot B_2) \overline{A_1}B_1 + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) \overline{A_0}B_0$$

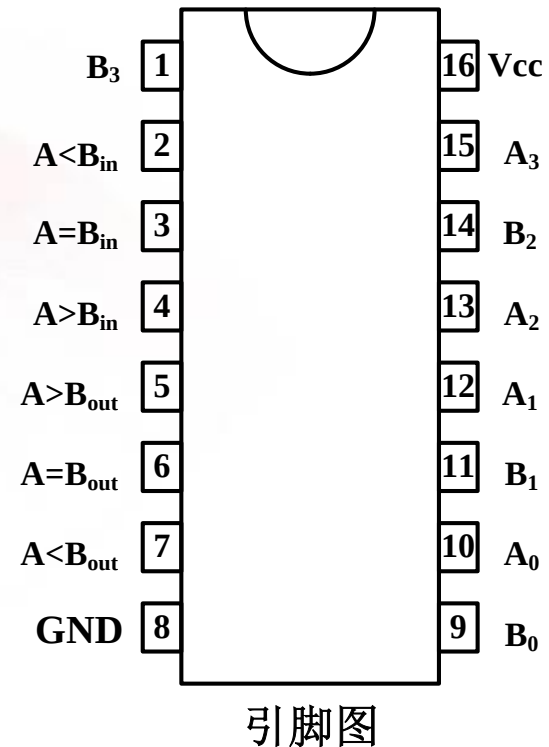
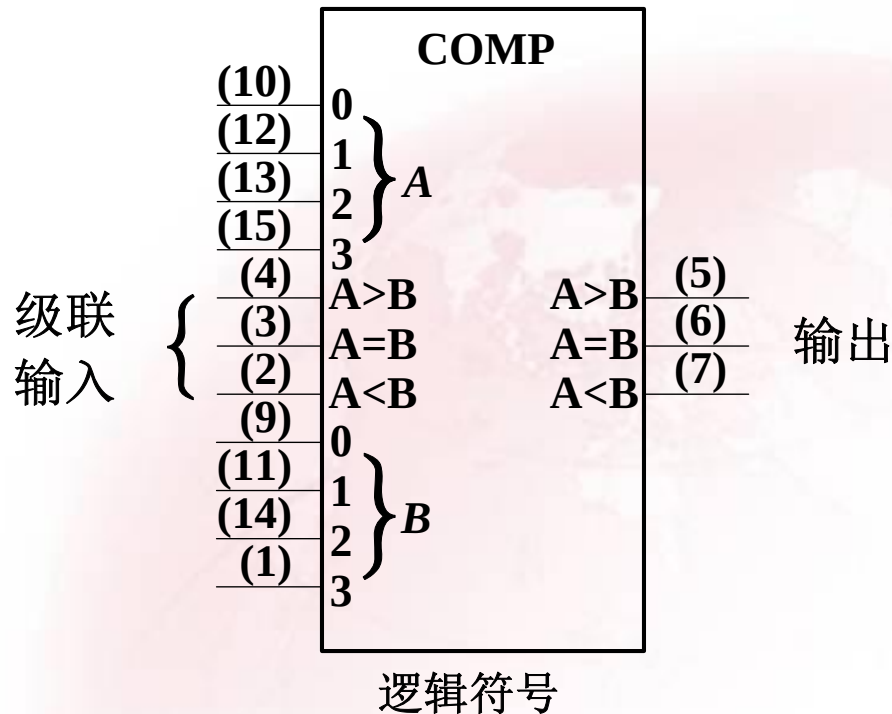
$$Y_{(A=B)} = (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) (A_0 \odot B_0)$$

4.6.3 通用数值比较器集成电路

通用数值比较器集成电路有多个品种，属CMOS电路的4位数值比较器的有74HC85（对应的TTL电路为74LS85）、CC14585等。



74HC85为带级联输入的4位数值比较器





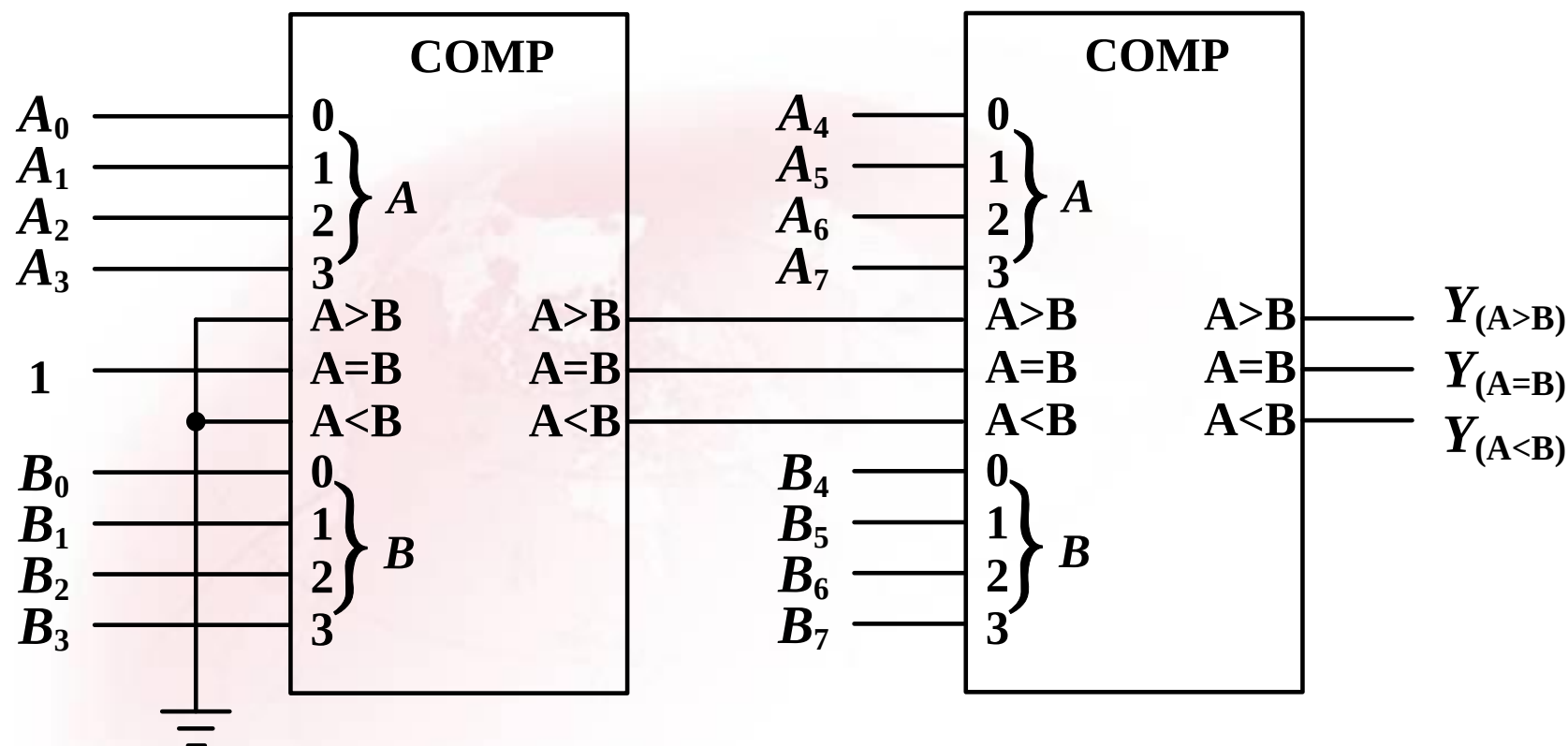
四位数值比较器真值表(74HC85)

比较输入				级联输入			输出		
$A_3 B_3$	$A_2 B_2$	$A_1 B_1$	$A_0 B_0$	$I_{(A>B)}$	$I_{(A<B)}$	$I_{(A=B)}$	$Y_{(A>B)}$	$Y_{(A<B)}$	$Y_{(A=B)}$
$A_3>B_3$	×	×	×	×	×	×	1	0	0
$A_3<B_3$	×	×	×	×	×	×	0	1	0
$A_3=B_3$	$A_2>B_2$	×	×	×	×	×	1	0	0
$A_3=B_3$	$A_2<B_2$	×	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1>B_1$	×	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1<B_1$	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0>B_0$	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0<B_0$	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	1	0	0	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	1	0	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	×	×	1	0	0	1
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	1	1	0	0	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	0	0	1	1	0

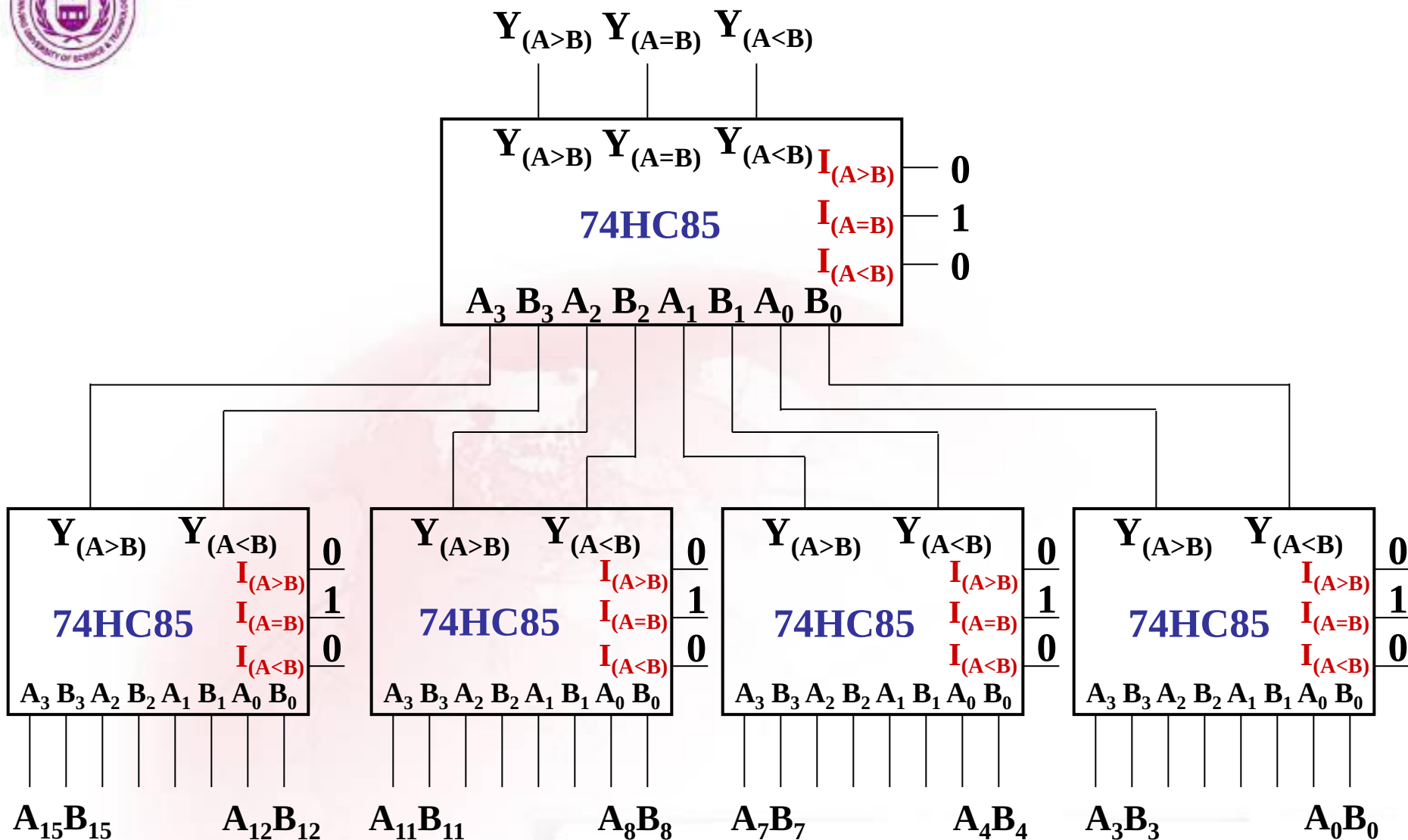
$I_{(A>B)}$ 、 $I_{(A<B)}$ 、 $I_{(A=B)}$ 是另外两个低位数比较结果。设置低位数比较结果输入端是为了与其它数值比较器连接，以便扩展更多位数值比较器。



比较器的扩展:



8位数值比较器（串行接法）



16位数值比较器(并行接法)



串行接法和并行接法性能比较:

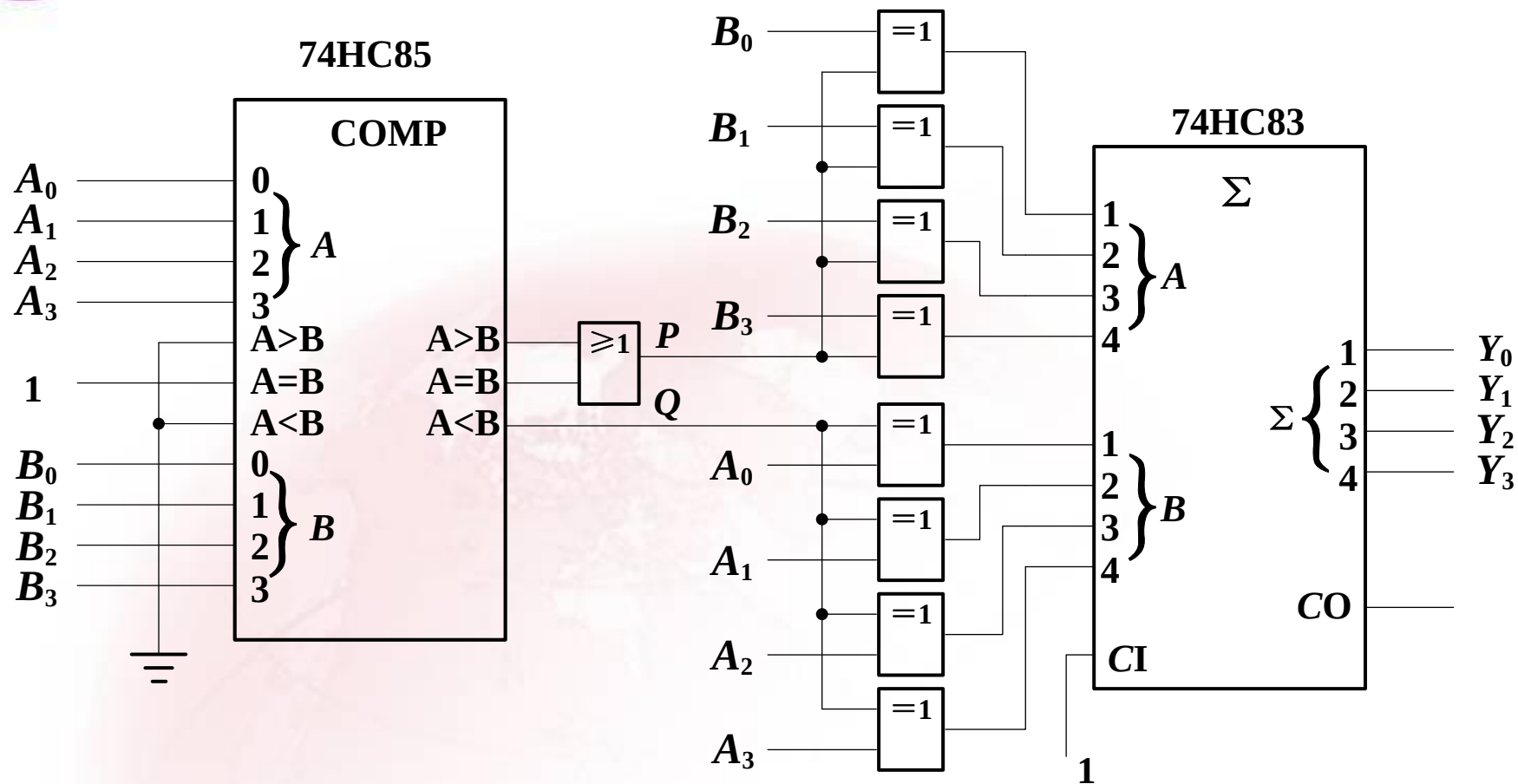
串行接法**电路简单**, 但速度**慢**;

并行接法**电路复杂**, 速度**快**。

4.6.4 数值比较器应用举例

例: 设计一个求两数之差绝对值电路。

设计思路: 先将两数比较, 对小的数求补, 将得到的补码与另一数相加, 得到结果。



求两数之差绝对值电路



4.6.5 数值比较器的Verilog描述

带级联输入的4位数值比较器Verilog程序代码。

```
module Vr7485(A,B,AGTBIN,ALTBIN,AEQBIN,AGTBOUT,ALTBOUT,AEQBOUT);  
  input [3:0] A,B;  
  input AGTBIN,ALTBIN,AEQBIN;  
  output AGTBOUT,ALTBOUT,AEQBOUT;  
  
  reg AGTBOUT,ALTBOUT,AEQBOUT;  
  
  always @ (A,B,AGTBIN,ALTBIN,AEQBIN)  
    if(A==B)  
      begin AGTBOUT=AGTBIN;ALTBOUT=ALTBIN;AEQBOUT=AEQBIN; end  
    else if (A>B)  
      begin AGTBOUT=1'b1;ALTBOUT=1'b0;AEQBOUT=1'b0; end  
    else if (A<B)  
      begin AGTBOUT=1'b0;ALTBOUT=1'b1;AEQBOUT=1'b0; end  
endmodule
```



※ 4.7 代码转换器

重点介绍能实现**BCD**码和自然二进制码之间转换的代码转换器的设计方法，并介绍通用代码转换器集成电路的使用方法。

4.7.1 BCD—二进制码转换器

转换过程：

- (1) 将**BCD**码中的每一位的权值用二进制数表示；
- (2) 将所给**BCD**码中 ‘1’所代表的二进制数相加；
- (3) 相加的结果即为所给**BCD**码的等效二进制数。



如两位十进制数的8421BCD码为:

$B_3B_2B_1B_0$ $A_3A_2A_1A_0$

(十位) (个位)

BCD位权与二进制数对照表

BCD 位	BCD 权值	二进制数表示						
		64	32	16	8	4	2	1
A_0	1	0	0	0	0	0	0	1
A_1	2	0	0	0	0	0	1	0
A_2	4	0	0	0	0	1	0	0
A_3	8	0	0	0	1	0	0	0
B_0	10	0	0	0	1	0	1	0
B_1	20	0	0	1	0	1	0	0
B_2	40	0	1	0	1	0	0	0
B_3	80	1	0	1	0	0	0	0

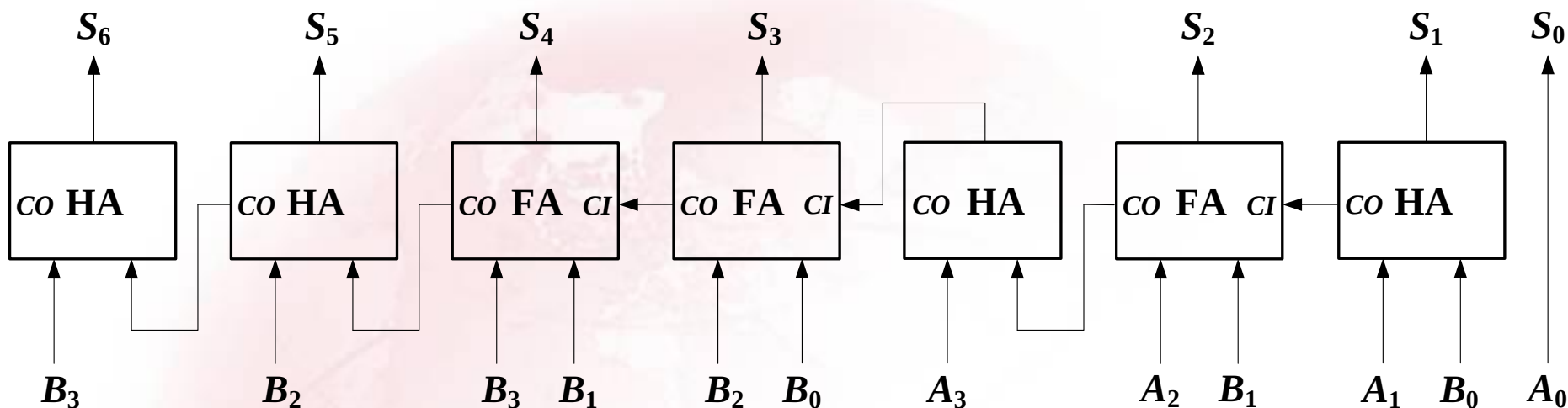


例如，要将BCD码1000 0111（十进制数87）转换为二进制，其算式如下：

80	40	20	10	8	4	2	1	
1	0	0	0	0	1	1	1	
								0 0 0 0 0 0 1
								0 0 0 0 0 1 0
								0 0 0 0 1 0 0
								+ 1 0 1 0 0 0 0
								1 0 1 0 1 1 1



根据对照表，借助半加器和全加器，可设计出转换电路。

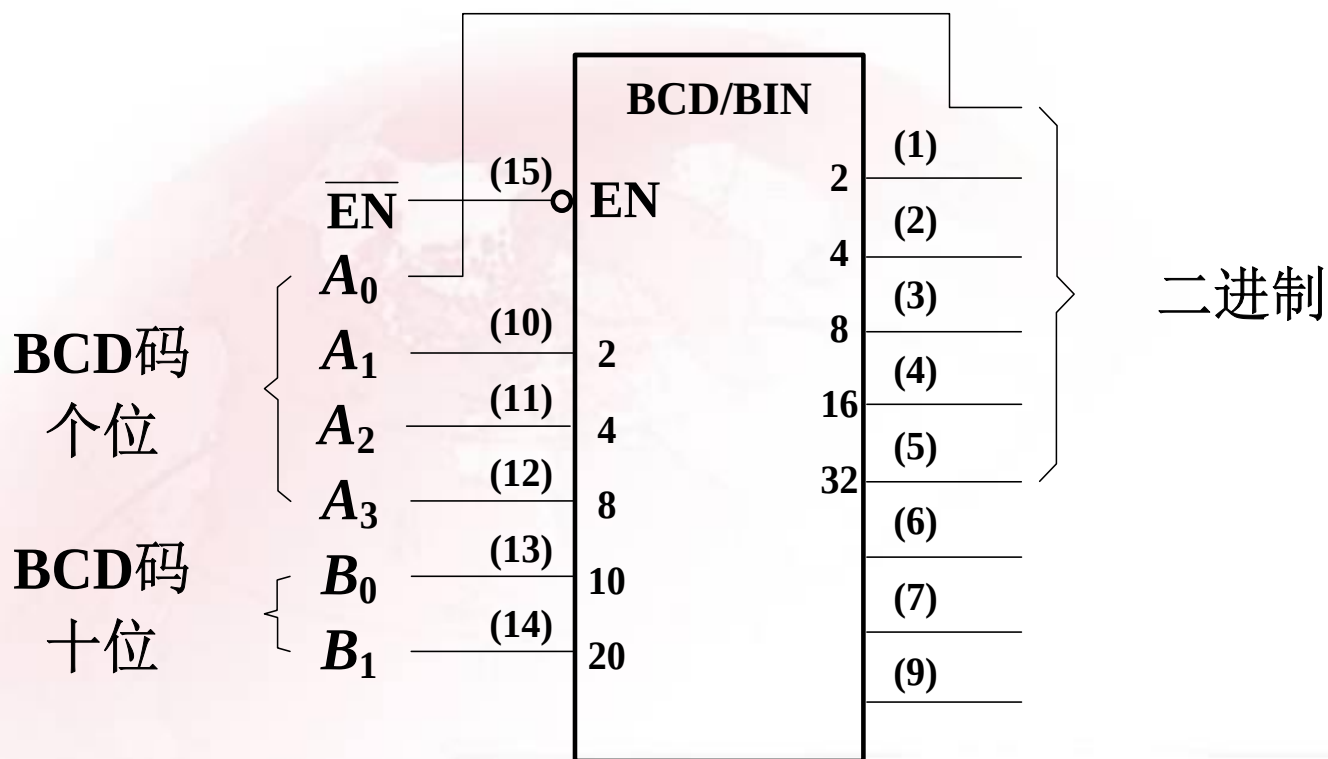


由半加器和全加器组成的两位BCD—二进制代码转换器



4.7.2 通用BCD—二进制和二进制—BCD码转换器集成电路

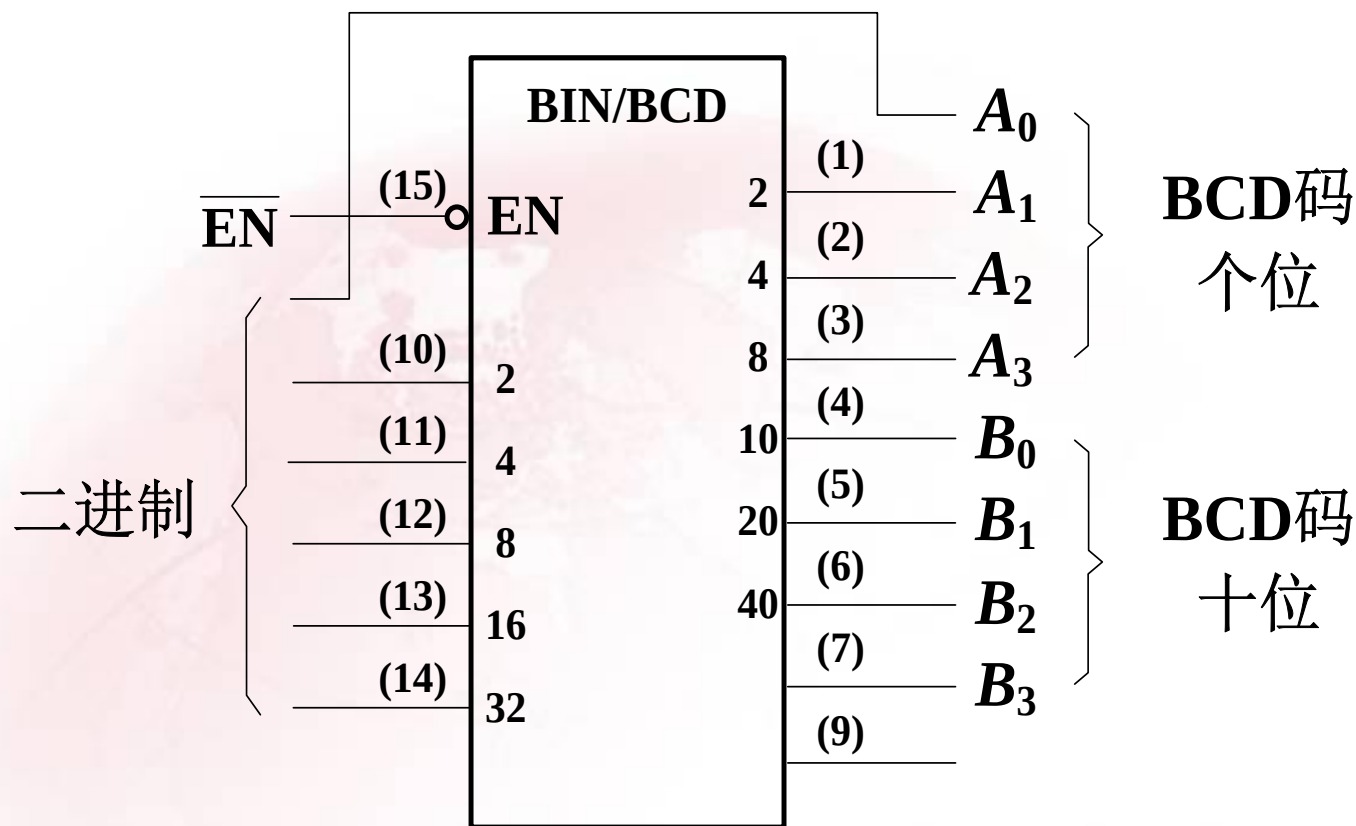
1. BCD—二进制代码转换器74184



74184 6位BCD—BIN代码转换器



2. 二进制—BCD代码转换器74185



74185 6位BIN—BCD代码转换器



4.7.3 代码转换电路的Verilog描述

例：两位BCD码（个位和十位）转换为二进制数的代码转换器的Verilog程序代码

```
module Vrbcd_to_bin(ONES,TENS,BINARY);  
  input [3:0] ONES,TENS;  
  output [6:0] BINARY;  
  reg [6:0] BINARY;  
  reg [6:0] TIMES;  
  
  always @ (ONES,TENS)  
  begin  
    TIMES<=TENS*10;  
    BINARY<=TIMES+ONES;  
  end  
endmodule
```




例：设计一个能将S位二进制码转换为格雷码的码转换电路

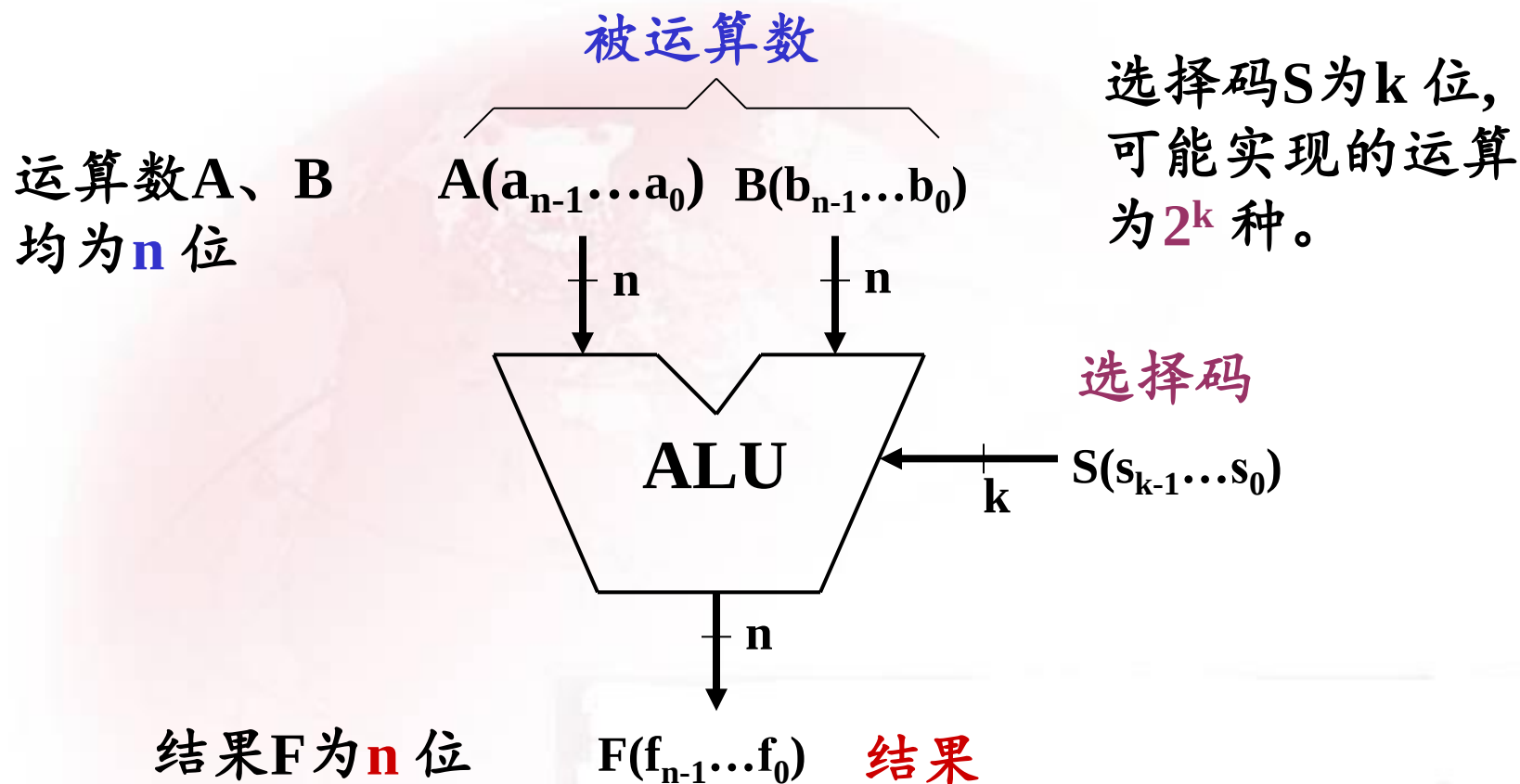
```
module Vrbn_to_gray(BIN,GOUT);  
    parameter S=4;  
    input [S:1] BIN;  
    output [S:1] GOUT;  
    reg [S:1] GOUT;  
    integer j;  
  
    always @ (BIN)  
    begin  
        GOUT[S]=BIN[S];  
        begin  
            for (j=1;j<S;j=j+1)  
            begin  
                GOUT[j]=BIN[j]^BIN[j+1];  
            end  
        end  
    end  
endmodule
```



※4.8 数字系统设计举例：算术逻辑单元(ALU)

算术逻辑单元(ALU)是计算机等数字系统的主要运算部件。

ALU的逻辑符号：





设计具有八种功能的ALU，S有3位，假设功能表如下

选择码			ALU	
S_2	S_1	S_0	功 能	说 明
0	0	0	$F=A+B$	加
0	0	1	$F=A-B$	减
0	1	0	$F=A+1$	加 1
0	1	1	$F=A-1$	减 1
1	0	0	$F=A \cdot B$	与
1	0	1	$F=A+B$	或
1	1	0	$F=\bar{A}$	非
1	1	1	$F=A \oplus B$	异或

$S_2=0$: 算术运算

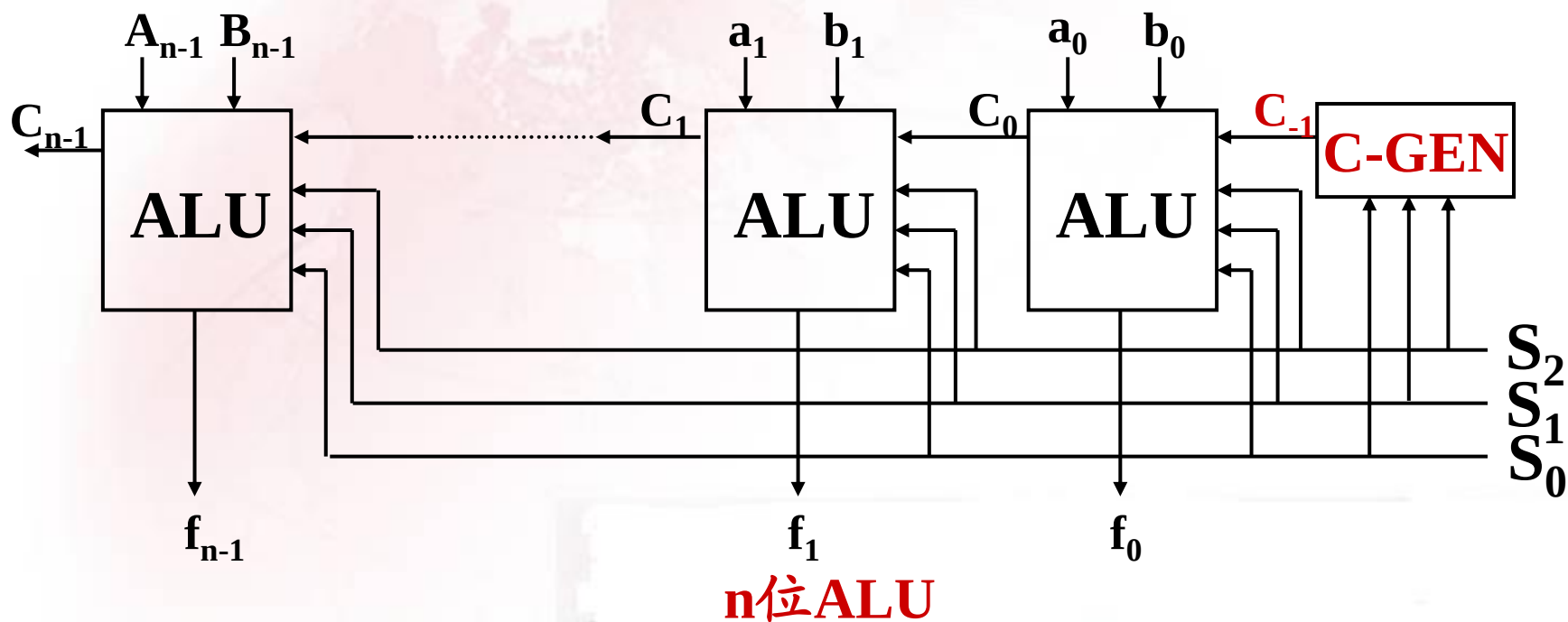
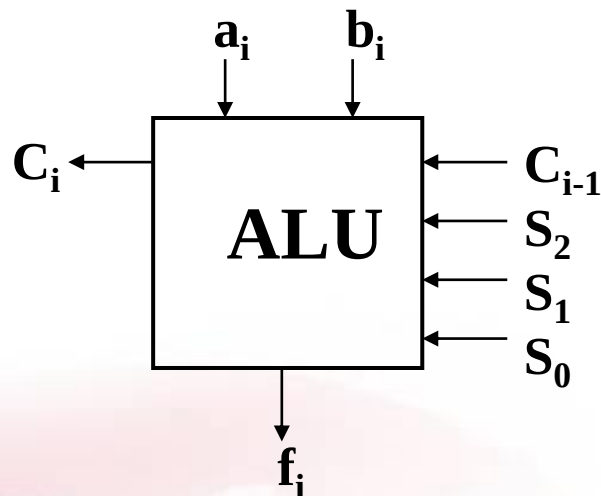
$S_2=1$: 逻辑运算

设计思想:

设计采用自顶向下的方法，将能进行n位运算的ALU分解为n个能进行一位运算的ALU，最后将n个一位ALU连接成n位ALU。

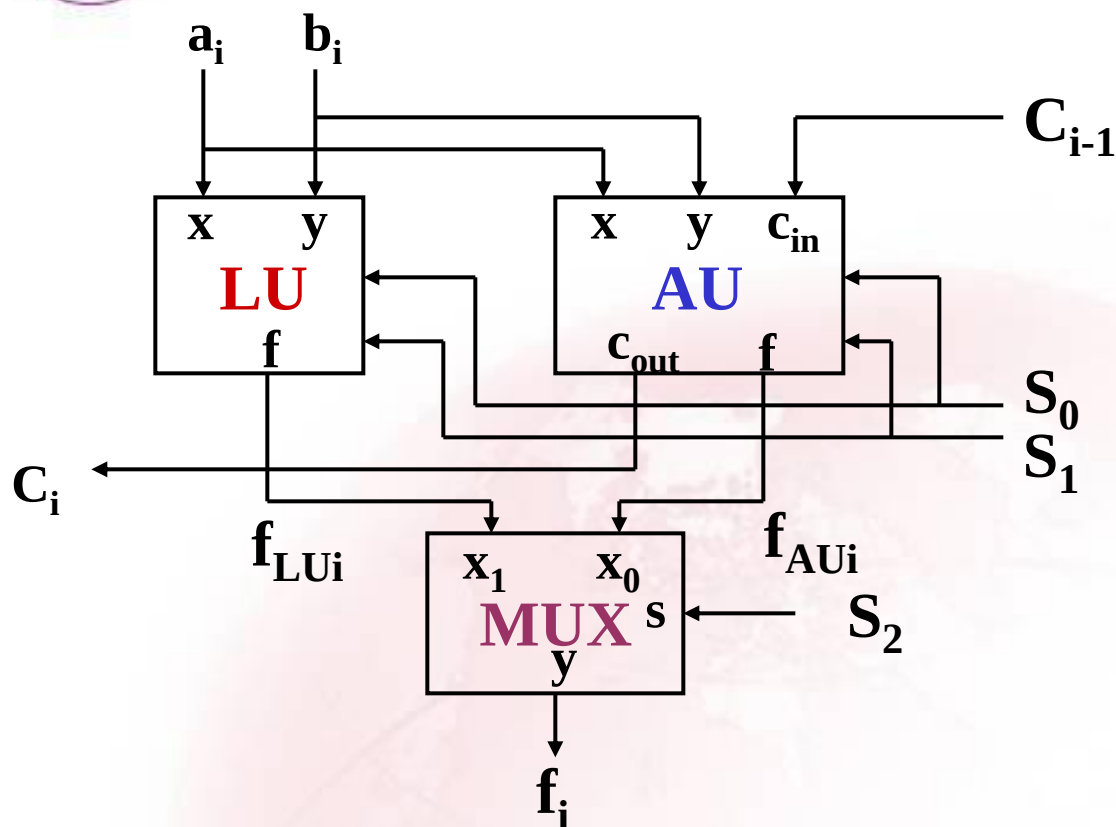


一位ALU:





一位ALU电路设计:



一位ALU分解图:

AU: 算术单元;

LU: 逻辑单元;

MUX: 数据选择器, 根据 S_2 的值, 对AU和LU的运算结果进行选择。

(1) MUX电路设计: 这里的MUX为二选一数据选择器, 设计方法前面已介绍;

(2) LU电路设计:



计算机中的逻辑运算是**位**操作(即对应**位**之间进行运算)。

LU单元功能表

S_1	S_0	功 能 (f_{LUi})
0	0	$a_i b_i$
0	1	$a_i + b_i$
1	0	$\bar{a}_i$
1	1	$a_i \oplus b_i$

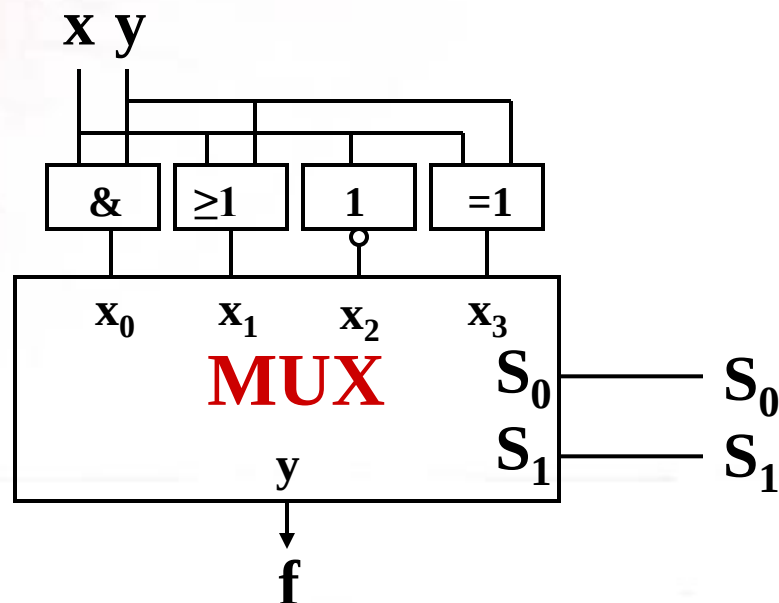
逻辑方程:

$$f = \bar{S}_1 \bar{S}_0 (xy) + \bar{S}_1 S_0 (x + y) + S_1 \bar{S}_0 (\bar{x}) + S_1 S_0 (x \oplus y)$$

逻辑方程化简为:

$$f = \bar{S}_1 xy + S_0 x\bar{y} + S_0 \bar{x}y + S_1 \bar{S}_0 \bar{x}$$

根据化简的逻辑方程，可用逻辑门实现LU功能。另外，也可直接根据功能表，用**数据选择器**实现。

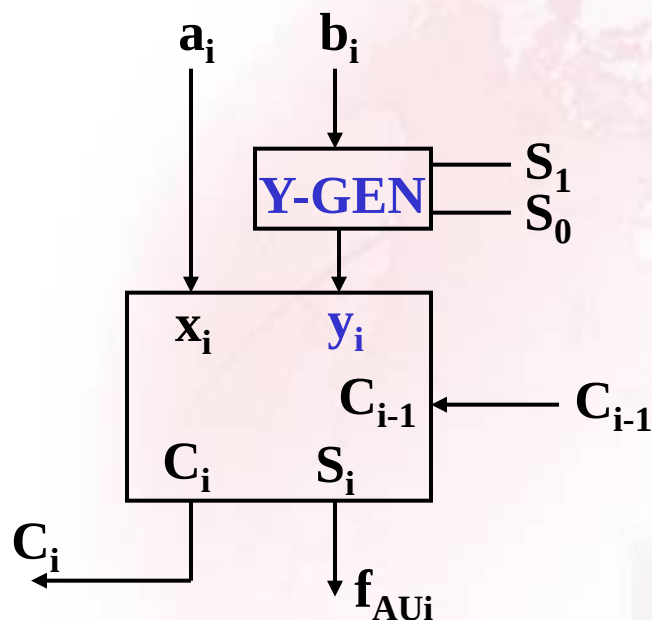




(3) AU电路设计:

算术单元要进行加、减、加1、减1等四种运算，当采用补码运算时，可利用全加器实现。

根据ALU功能表，可利用下表求得 y_i 和 C_{-1} 的表达式。



y_i 和 C_{-1} 取值表

功能	S_1	S_0	y_i	C_{-1}
加	0	0	b_i	0
减	0	1	$\bar{b}_i$	1
加1	1	0	0	1
减1	1	1	1	0



对**Y-GEN**,可写出 y_i 的表达式:

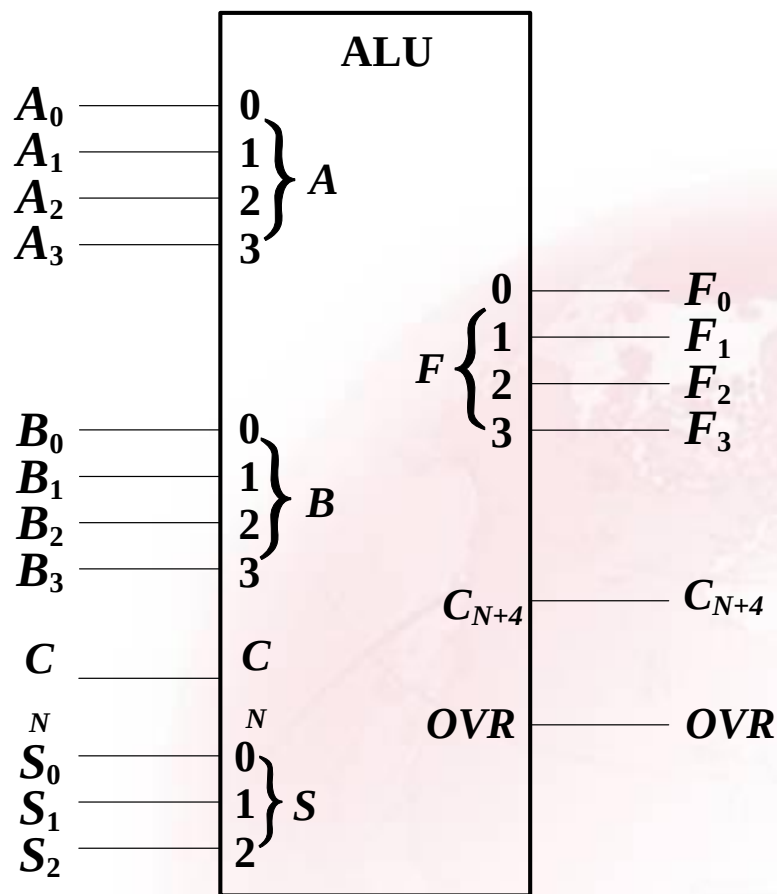
$$y_i = S_0 \oplus (\bar{S}_1 b_i)$$

对**C-GEN**可写出 C_{-1} 的表达式 : $C_{-1} = S_1 \oplus S_0$

将上述运算求得的MUX、LU、AU电路连接, 可得到一位ALU; 将多位ALU和C-GEN连接, 可完成多位ALU电路设计。



ALU集成电路74LS/HC382



逻辑符号

S_2	S_1	S_0	操 作	说 明
0	0	0	清零	$F_3F_2F_1F_0 = 0000$
0	0	1	$F = B - A$	} 要求 $C_N=1$
0	1	0	$F = A - B$	
0	1	1	$F = A + B$	
1	0	0	$F = A \oplus B$	异或
1	0	1	$F = A + B$	或
1	1	0	$F = A \cdot B$	与
1	1	1	预置	$F_3F_2F_1F_0 = 1111$

功能表



第4章 小结

- 自顶向下的模块化设计方法
- 组合逻辑电路的中规模功能模块
 - 编码器
 - 译码器
 - 数据分配器
 - 数据选择器
 - 加法器
 - 数值比较器